Tudományos számítások

Egy bevezető áttekintés — Átdolgozott második kiadás (magyar fordítás, folyamatban)

Michael T. Heath

Tartalomjegyzék

Előszó a Classics kiadáshoz **2**

Előszó **3**

Jelölések **7**

Tudományos számítások **9**

1.1 Bevezetés . . . 9
1.1.1 Számítási feladatok . . . 9
1.1.2 Általános stratégia . . . 11
1.2 Közelítések a tudományos számításokban . . . 12
1.2.1 A közelítés forrásai . . . 12
1.2.2 Abszolút hiba és relatív hiba . . . 13
1.2.3 Adathiba és számítási hiba . . . 14
1.2.4 Csonkítási hiba és kerekítési hiba . . . 16
1.2.5 Előreható hiba és hátraható hiba . . . 18
1.2.6 Érzékenység és kondicionáltság . . . 21
1.2.7 Stabilitás és pontosság . . . 24
1.3 Számítógépes aritmetika . . . 25
1.3.1 Lebegőpontos számok . . . 25
1.3.2 Normalizáltság . . . 26
1.3.3 A lebegőpontos rendszerek tulajdonságai . . . 27
1.3.4 Kerekítés . . . 28
1.3.5 Gépi pontosság . . . 29
1.3.6 Szubnormális számok és fokozatos alulcsordulás . . . 30
1.3.7 Kivételes értékek . . . 30
1.3.8 Lebegőpontos aritmetika . . . 31
1.3.9 Számjegykioltás . . . 33
1.3.10 Más aritmetikai rendszerek . . . 38
1.3.11 Komplex aritmetika . . . 40
1.4 Matematikai szoftver . . . 43
1.5 Történeti jegyzetek és további olvasnivaló . . . 47
Áttekintő kérdések . . . 48
Feladatok . . . 52

Számítógépes feladatok . . . 54

Lineáris egyenletrendszerek **61**
2.1 Lineáris rendszerek . . . 61
2.2 Létezés és egyértelműség . . . 63
2.3 Érzékenység és kondicionáltság . . . 65
2.3.1 Vektornormák . . . 65
2.3.2 Mátrixnormák . . . 67
2.3.3 Mátrix kondíciószáma . . . 69
2.3.4 Hibakorlátok . . . 73
2.3.5 Maradék . . . 76
2.4 Lineáris egyenletrendszerek megoldása . . . 78
2.4.1 Feladatátalakítások . . . 78
2.4.2 Háromszögű lineáris egyenletrendszerek . . . 79
2.4.3 Elemi eliminációs mátrixok . . . 81
2.4.4 Gauss-kiküszöbölés és LU-felbontás . . . 82
2.4.5 Főelemkiválasztás . . . 85
2.4.6 A Gauss-kiküszöbölés megvalósítása . . . 93
2.4.7 A lineáris egyenletrendszerek megoldásának komplexitása . . . 94
2.4.8 Gauss–Jordan-kiküszöbölés . . . 95
2.4.9 Módosított feladatok megoldása . . . 97
2.4.10 A pontosság javítása . . . 99
2.5 Lineáris egyenletrendszerek speciális típusai . . . 100
2.5.1 Szimmetrikus pozitív definit rendszerek . . . 101
2.5.2 Szimmetrikus indefinit rendszerek . . . 103
2.5.3 Sávos rendszerek . . . 104
2.6 Iterációs módszerek lineáris egyenletrendszerekhez . . . 105
2.7 Szoftverek lineáris egyenletrendszerekhez . . . 105
2.7.1 LINPACK és LAPACK . . . 106
2.7.2 Basic Linear Algebra Subprograms (BLAS) . . . 107
2.8 Történeti jegyzetek és további olvasnivaló . . . 108
Áttekintő kérdések . . . 109
Feladatok . . . 114
Számítógépes feladatok . . . 119

Előszó a Classics kiadáshoz

Megtiszteltetés számomra, hogy a SIAM a Classics sorozatában jelenteti meg a könyvemet. Ennek az új kiadásnak az előkészítése során jelentősen átdolgoztam a könyv leginkább frissítésre szoruló részeit, elsősorban a rendelkezésre álló szoftverekre vonatkozó hivatkozásokat és az irodalomjegyzéket. A könyv tartalmának nagyobbik része azonban időtállónak bizonyult, és nagyrészt érintetlen maradt. Szeretném azt hinni, hogy ez a könyv korszerű szemléletének köszönhető, de valószínűbb, hogy inkább az alapozó jellegéből adódik: a numerikus analízis és a tudományos számítások legalapvetőbb fogalmait és módszereit tárgyalja, amelyek viszonylag lassan változnak (Newton, Euler és Gauss nem publikált sokat az utóbbi időben!). A legjelentősebb technikai változás talán az, hogy a Lagrange-interpoláció baricentrikus alakját beépí-

tettem a törzsanyagba – korábban ez csak egy feladatban szerepelt.

A könyv eredeti megjelenése óta számos új irányzat bontakozott ki, amelyek közül néhányra ez az új kiadás utal is, míg mások túlmutatnak a könyv keretein. Pedagógiai szempontból talán a legfontosabb trend a Python mint programozási nyelv és számítási környezet térnyerése, amely az oktatásban és a szakmai gyakorlatban egyaránt jelentős hatást gyakorolt. Ennek megfelelően a könyv szoftverhivatkozásait kiegészítettem a vonatkozó Python-csomagokkal. A szoftverek területén egy másik figyelemre méltó újdonság a Chebfun, amely több hagyományos megközelítést ötvöz innovatív módon, és ezáltal új, mélyreható számítási lehetőségeket kínál. A nagyléptékű tudományos számításokban a párhuzamosság egyre erőteljesebb kihasználásának tendenciája töretlenül folytatódott, ám a párhuzamos algoritmusok komoly tárgyalása már egy egész újabb könyvet igényelne.

Egy másik trend a sztochasztikus módszerek növekvő alkalmazása – például a lineáris algebrában és az optimalizálásban –, amelyet gyakran a gyorsan fejlődő területek, mint a gépi tanulás és az adattudomány alkalmazásai motiválnak. A könyv már eredetileg is tartalmazott egy rövid fejezetet a sztochasztikus módszerekről, amely jó alapot nyújt a további, más forrásokra támaszkodó tanulmányokhoz. A bizonytalanságszámszerűsítés (UQ) iránti érdeklődés is ugrásszerűen megnőtt az utóbbi években. A létezés, egyértelműség, kondicionáltság és stabilitás következetes hangsúlyozásával a könyv szilárd fogalmi alapot nyújt a számítási eredmények megbízhatóságának megítéléséhez, de a legújabb UQ-technikák részletes tárgyalása ismét csak meghaladja a kereteinket.

A korábbi kiadás megjelenése óta eltelt időszak fontos újdonsága, hogy elérhetővé vált mintegy 100 oktatási modul, amelyeket több diákkal együttműködve készítettem. Ezek a Java-appletek dinamikusan szemléltetik a tudományos számítások számos fogalmát és módszerét, és a könyvet kísérő interaktív példákként szolgálnak. Alkalmasak tantermi demonstrációkra, illetve a diákok önálló tanulásához is. A modulok elérhetők webböngészőn keresztül, illetve letölthetők egy önálló változatban, amely a felhasználó saját számítógépén futtatható. A modulok szabadon beszerezhetők a www.siam.org/books/cl80 címen.

Az eredeti előszóban megköszönt két mentorom (Alston Householder és Gene Golub) mellett szeretném ezt az alkalmat arra is használni, hogy elismerésemet fejezzem ki több szerzőnek, akiknek világos és elegáns stílusát következetesen inspirálónak és követendőnek találtam: Nick Higham, Pete Stewart, Gil Strang, Nick Trefethen és Charlie Van Loan. Végül ismét köszönetet szeretnék mondani feleségem, Mona türelméért és megértéséért, aki mellett „nyugdíjazásom” ellenére továbbra is dolgozom – illő, hogy ezzel az új kiadással végre ő is saját ajánlóoldalt kapjon!

Michael T. Heath

Előszó

Ez a könyv átfogó áttekintést nyújt a numerikus módszerekről olyan hallgatók és a gyakorlatban dolgozó szakemberek számára, akiknek matematikai feladatokat kell megoldaniuk a számításigényes tudományterületeken. A hagyományos numerikus analízis

könyvektől abban különbözik, hogy a bemutatott algoritmusok mögött rejlő motivációra és gondolatokra helyezi a hangsúlyt, nem pedig azok részletes elemzésére. Igyekszem általános képet adni azokról a technikákról, amelyek az egyes nagyobb problémakategóriákban rendelkezésre állnak, ideértve a helyes feladatmegfogalmazást és az eredmények értelmezését, ugyanakkor ahol csak lehetséges, a professzionális matematikai szoftverek használatát szorgalmazom a feladatok megoldására. A könyv inkább a matematikai szoftverek potenciális felhasználóinak szól, mint az ilyen szoftverek lehetséges készítőinek. Remélem, hogy felhívhatom az olvasó figyelmét arra, hogy milyen kérdések lényegesek a megfelelő módszerek és szoftverek kiválasztásánál és körültekintő alkalmazásánál.

A University of Illinois-on ez a könyv egy átfogó, egyféléves numerikus módszerek kurzus tankönyveként szolgál, és több célt is betölt:

- Zárókurzusként végzős alapképzéses hallgatóknak, főként informatika, matematika és mérnöki szakokon
- Alapozó kurzusként olyan mesterképzéses hallgatóknak informatika és matematika szakon, akik tudományos számításokra kívánnak szakosodni
- Átfogó kurzusként olyan informatika mesterképzéses hallgatóknak, akik nem kívánnak tudományos számításokra szakosodni
- Gyakorlati képzésként olyan természettudományos és mérnöki mesterképzéses hallgatóknak, akiknek numerikus módszereket és szoftvereket kell használniuk a kutatásukban. Ez egy kötelező alapkurzus a Mérnöki Kar által támogatott interdiszciplináris Computational Science and Engineering mesterképzési programban.

Hogy ezt a sokszínű hallgatói kört kiszolgáljam, a kurzus és a könyv előkövetelményeit a lehető legalacsonyabb szintre szorítottam: alapvető jártasság a lineáris algebrában, a többváltozós analízisben és némi ismeret a differenciálegyenletekről. Numerikus módszerekben való előzetes jártasságot nem feltételezek. A könyv ugyanakkor meglehetősen kifinomult szemléletet alkalmaz, ezért a hallgatónál (vagy olvasónál) bizonyos fokú matematikai érettség ajánlott. Az akadémiai környezeten túl remélem, hogy a könyv kézikönyvként is hasznos lesz a gyakorló mérnökök és tudósok számára, akiknek gyors áttekintésre lehet szükségük egy adott számítási feladatról, valamint a megoldásához rendelkezésre álló módszerekről és szoftverekről.

Noha a könyv hangsúlyozza a matematikai szoftverek használatát, néhány más szoftverközpontú munkával ellentétben a kötet önmagában nem mellékel szoftvert, és nem összpontosít konkrét szoftvercsomagokra, könyvtárakra vagy környezetekre. Ehelyett minden problémakategóriához hivatkozásokat ad olyan konkrét rutinokra, amelyek nyilvánosan elérhető tárolókból, valamint a jelentősebb kereskedelmi könyvtárakból és csomagokból szerezhetők be. Sok akadémiai és ipari számítási környezetben az ilyen szoftverek már telepítve vannak, mindenesetre hivatkozások szerepelnek a nyilvánosan hozzáférhető, az interneten keresztül szabadon elérhető szoftverekre is. A könyvben szereplő számítógépes feladatok nem függenek egy konkrét szoftver vagy programozási nyelv megválasztásától.

A könyv felépítésének főbb elemei a következők:

Fejezetek: A könyv minden fejezete egy-egy jelentős számítási problémakört tárgyal. A könyv első fele főként algebrai feladatokkal, míg a második fele a deriváltakat és

integrálokat érintő analitikus feladatokkal foglalkozik. Az első két fejezet alapvető a könyv többi részéhez, a további fejezetek azonban az oktató preferenciája szerint különféle sorrendben tárgyalhatók. Pontosabban, a fejezetek közötti főbb függőségek nagyjából a következők:

Fejezet	Függ ettől	Fejezet	Függ ettől	Fejezet	Függ ettől
2	1	6	1, 2, 3, 5	10	1, 2, 5, 7, 9
3	1, 2	7	1, 2	11	1, 2, 7, 9, 10
4	1, 2, 3	8	1, 2, 7	12	1, 2, 7
5	1, 2	9	1, 2, 5, 7	13	1

Így a 7., 8., 12. és 13. fejezet sokkal korábban, míg a 3., 4. és 6. fejezet sokkal később is tárgyalhatók, mint ahogy a könyvben szerepelnek. Például a 3., 7. és 12. fejezet mindegyike valamilyen adatillesztést tartalmaz, így kívánatos lehet egy egységként tárgyalni őket. Egy másik példa: a lineáris egyenletrendszerek iterációs módszerei a parciális differenciálegyenletekről szóló 11. fejezetben találhatók, mert innen származnak a legfontosabb motivációs példák, de ennek az anyagnak nagy része közvetlenül a 2. fejezet lineáris egyenletrendszerekre vonatkozó direkt módszerei után is tárgyalható. Megjegyezzük, hogy a sajátértékek fogalmára a könyv többi részében széles körben építünk, így célszerű a 4. fejezetet viszonylag korán feldolgozni, hacsak a hallgatók már nem ismerik e téma alapjait.

A könyvben több mint elegendő anyag van egy teljes egyféléves kurzushoz, így egy egyféléves kurzusban körültekintő válogatásra lehet szükség. Például a véletlen számokról és a sztochasztikus szimulációról szóló 13. fejezet csak érintőlegesen kapcsolódik a könyv többi részéhez, és nyilvánvaló jelölt az elhagyásra (véletlenszám-generátorokat azonban a könyv számos feladatában használunk). A teljes könyv két negyedéves (quarter) vagy kétféléves kurzus alatt dolgozható fel.

Példák: Szinte minden bevezetett fogalmat és módszert egy vagy több példa illusztrál. Ezek a példák a viszonylag tömör általános tárgyalás kiegészítését szolgálják, és a szöveg lényeges részeként olvasandók. A példákat a lehető legegyszerűbben tartottam (akár a túlzott leegyszerűsítés kockázatát is vállalva), hogy az olvasó könnyen követhesse őket. Tapasztalatom szerint egy alaposan megértett egyszerű példa rendszerint többet segít, mint egy realisztikusabb, de nehezebben követhető.

Szoftverek: Az egyes problémakategóriákhoz tartozó, rendelkezésre álló szoftverek listája igyekszik kellően átfogó lenni. Nem kíséreltem meg kiemelni a „legjobb” szoftvert egy-egy adott feladathoz, részben azért, mert rendszerint egyetlen csomag sem múlja felül a többit minden tekintetben, részben pedig azért, hogy figyelembe vegyem a különböző olvasóknál fennálló eltérő szoftverelérhetőséget és programozásinyelvválasztást. Minden idézett szoftver legalábbis professzionálisan fejlesztett, némelyikük pedig egészen kiváló.

Feladatok: A könyv sok feladatot tartalmaz, amelyek három kategóriába sorolhatók:

- Áttekintő kérdések, amelyek rövid választ igényelnek, és az alapvető fogalmi megértés ellenőrzésére szolgálnak

- Gyakorló feladatok, amelyek valamivel több gondolkodást, hosszabb válaszokat és esetleg kézi számításokat igényelnek
- Számítógépes feladatok, amelyek némi programozást igényelnek, és gyakran meglévő szoftverek használatát is magukban foglalják.

Az áttekintő kérdések az olvasó öntesztelését szolgálják. Szándékosan tartalmaznak némi ismétlést, hogy rögzítsék a kulcsfontosságú gondolatokat, és megerősítsék a hallgatók magabiztosságát az anyag elsajátításában. A hosszabb feladatok írásbeli házi feladatok céljára alkalmasak. Némelyikük egyszerű példákkal való kézi számítást igényel, míg másokat úgy terveztem, hogy pótolják a törzsanyagból kimaradt levezetések és bizonyítások részleteit. Ez utóbbiak különösen hasznosak lehetnek, ha a könyvet elméletibb jellegű kurzushoz használjuk. A számítógépes feladatok lehetőséget biztosítanak a gyakorlati tapasztalatszerzésre az ajánlott szoftverek használatában, az egyes kategóriákhoz tartozó tipikus feladatok megoldására. Egyes feladatok általánosak, mások viszont közvetlenül kapcsolódnak a különböző tudományos és mérnöki diszciplínák konkrét alkalmazásaihoz.

Változások a második kiadásban. Minden fejezet most motiváló bevezetővel és egy vagy több szemléltető példával kezdődik, amelyet az adott típusú feladat megoldásainak létezését, egyértelműségét és kondicionáltságát vizsgáló rész követ. A cél az, hogy a hallgató jobban megértse, miért fontos az adott feladat, és hogyan ismerheti fel a „jó” vagy „rossz” feladatmegfogalmazást, mielőtt a megoldó algoritmusokat tárgyalnánk. A legfontosabb algoritmusokat formálisan is megadtam, és a könnyebb hivatkozhatóság érdekében sorszámmal láttam el. Az irodalomjegyzéket naprakésszé tettem, a történeti jegyzeteket pedig némileg kibővítettem. Az 1. fejezetben az előreható és hátraható hibákról, valamint azok kapcsolatáról szóló részt kibővítettem és pontosítottam. A szingulárisérték-felbontásról szóló anyag nagy részét a 4. fejezetből a 3. fejezetbe helyeztem át, ahol alkalmazásai jobban illeszkednek. A 4. fejezetben a sajátérték-algoritmusok tárgyalását kibővítettem, több motivációval és részlettel – különösen a QR-iterációról –, valamint néhány további módszerrel. A 6. fejezet feltételes optimalizálásról szóló elemzését lényegesen kibővítettem. A differenciálegyenletekről szóló fejezeteket némileg átszerveztem, a spektrális módszerek bemutatását pedig kiegészítettem. A gyors Fourier-transzformációról szóló 12. fejezetet átalakítottam, és letisztultabbá tettem néhány felesleges rész elhagyásával.

Szeretném megköszönni mentoraim hatását, akik először bevezettek a numerikus számítás szépségeibe: Alston Householdernek és Gene Golubnak. Hálás vagyok azért a bőséges visszajelzésért, amelyet a könyv első kiadását használó hallgatóktól és oktatóktól kaptam. Az első kiadás előzetes lektorai a következők voltak: Alan George, University of Waterloo; Dianne O'Leary, University of Maryland; James Ortega, University of Virginia; John Strikwerda, University of Wisconsin; és Lloyd N. Trefethen, Oxford University. A második kiadás előkészítésében segédkező lektorok a következők voltak: Thomas Coleman, Cornell University; Robert Funderlic, North Carolina State University; Thomas Hagstrom, University of New Mexico; Ramon Moore, Ohio State University; Mark Pernarowski, Montana State University; Linda Petzold, University of California at Santa Barbara; és Brian Suchomel, University of Minnesota. Köszönöm valamennyi lektor felbecsülhetetlen értékű javaslatait. Ezen felül külön köszönetet szeretnék mondani kollégáimnak – Joerg Liesennek, Paul Saylornak és Eric de Sturlernek, mindhármuknak a University of Illinois-ról –, akik mindegyike elolvasta

az átdolgozott kézirat egy részét vagy egészét, és rendkívül értékes visszajelzést adott. Köszönöm Melchior Franznak és Justin Winklernek a tördeléssel kapcsolatos hasznos tanácsait, amelyek elengedhetetlenek voltak a nyomdakész példány előkészítéséhez. Végül végtelenül hálás vagyok feleségemnek, Monának a türelméért és megértéséért e könyv megírásának és átdolgozásának számtalan órája alatt. Nagy örömmel és hálával ajánlom neki ezt a könyvet.

Michael T. Heath

A könyv honlapja: www.siam.org/books/cl80 A szerző honlapja: heath.cs.illinois.edu

Jelölések

A könyvben használt jelölésrendszer meglehetősen szabványos, és kevés magyarázatra szorul. Szabadon használunk vektor- és mátrixjelölést: mátrixokat általában nagybetűs félkövér szedéssel, vektorokat kisbetűs félkövér szedéssel, skalárokat pedig normál szedéssel jelölünk. Az iterációs és komponensindexeket alsó indexszel jelöljük, általában i-től n-ig. Például egy x vektornak és egy A mátrixnak rendre x_i és a_{ij} elemei vannak. Azokban a ritkább esetekben, amikor iterációs index és komponensindex egyaránt szükséges, az iterációt zárójeles felső index jelzi, például $x_i^{(k)}$ egy sorozat k-adik vektorának i-edik komponensét jelenti. Egyébként x_i az x vektor i-edik komponensét jelöli, míg x_k egy sorozat k-adik vektorát.

Az egyszerűség kedvéért elsősorban valós vektorokkal és mátrixokkal foglalkozunk, noha az általunk tárgyalt elmélet és algoritmusok nagy része kevés változtatással (vagy akár változtatás nélkül) átvihető a komplex esetre. A valós számok halmazát $\mathbb{R}$ jelöli, az n-dimenziós valós euklideszi teret $\mathbb{R}^n$, a valós $m \times n$ mátrixok halmazát pedig $\mathbb{R}^{m\times n}$. A komplex eset megfelelő jelölései rendre $\mathbb{C}$, $\mathbb{C}^n$ és $\mathbb{C}^{m\times n}$.

Egy vektor vagy mátrix transzponáltját felső T index jelzi, konjugált transzponáltját pedig felső H index (a Hermitian transpose rövidítése). Hacsak másként nem jelezzük, minden vektort oszlopvektornak tekintünk; egy sorvektort egy oszlopvektor explicit transzponálásával adunk meg. A tördelési kényelem kedvéért egy oszlopvektor komponenseit néha a megfelelő sorvektor transzponáltjaként jelöljük, például $\mathbf{x} = [x_1 \quad x_2]^T$. Két n-dimenziós $\mathbf{x}$ és $\mathbf{y}$ vektor belső szorzata (más néven skaláris szorzat) a mátrixszorzás speciális eseteként áll elő, ezért $\mathbf{x}^T\mathbf{y}$-nal jelöljük (vagy $\mathbf{x}^H\mathbf{y}$-nal a komplex esetben). Hasonlóképpen két n-dimenziós $\mathbf{x}$ és $\mathbf{y}$ vektor külső szorzata, amely egy $n \times n$-es mátrix, $\mathbf{x}\mathbf{y}^T$ alakban írható. Az n-edrendű egységmátrixot $\mathbf{I}_n$ jelöli (vagy csak $\mathbf{I}$, ha az n dimenzió a szövegkörnyezetből egyértelmű), i-edik oszlopát pedig $\mathbf{e}_i$. A zérusmátrixot $\mathbf{O}$, a zérusvektort $\mathbf{0}$, a zérusskalárt pedig 0 jelöli. A $d_1, \ldots, d_n$ átlóbeli elemekkel rendelkező diagonális mátrixot $\operatorname{diag}(d_1, \ldots, d_n)$ jelöli. Vektorok vagy mátrixok közötti egyenlőtlenségek elemenként értelmezendők. Az $m \times n$-es $\mathbf{A}$ mátrix oszlopai által kifeszített alteret $\mathbb{R}^m$-ben, azaz $\{\mathbf{A}\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\}$-t, $\operatorname{span}(\mathbf{A})$ jelöli.

Egyváltozós $f(t)$ függvény közönséges deriváltját df/dt vagy $f'(t)$ jelöli. Többváltozós függvények – például $u(x, y)$ – parciális deriváltjait $\partial u/\partial x$ alakban, vagy egyes szövegkörnyezetekben alsó indexszel, például u_x alakban jelöljük. A gradiensvektorok, a Jacobi-mátrixok és a Hesse-mátrixok jelölését szükség szerint vezetjük be. Minden logaritmus természetes alapú (az alap $e \approx 2{,}718$), hacsak más alapot kifejezetten

nem jelölünk. A $\approx$ szimbólumot hagyományos értelemben vett közelítő egyenlőségre, a $\cong$ szimbólumot pedig kifejezetten legkisebb négyzetes közelítésekre tartjuk fenn.

A numerikus algoritmusok számítási költségét, avagy számítási komplexitását általában a szükséges aritmetikai műveletek számával mérjük. Hagyományosan a numerikus analízisben csak a szorzásokat számolták (esetleg az osztásokat és négyzetgyökvonásokat is), mert a szorzás általában lényegesen drágább volt az összeadásnál vagy kivonásnál, és mert a legtöbb algoritmusban a szorzások jellemzően hasonló számú összeadással párosulnak (például két vektor belső szorzatának számításánál). Újabban az összeadás és a szorzás közötti költségkülönbség nagyrészt eltűnt (sőt, sok modern mikroprocesszor egyetlen szorzás-összeadás utasítással el tudja végezni az összevont szorzást és összeadást). A számítógépgyártók és -felhasználók szeretik a legmagasabb lehetséges teljesítményt hirdetni, ezért egyre gyakoribb, hogy minden egyes aritmetikai műveletet számolnak. Mivel azonban egyes műveletszámok nagyon jól ismertek a hagyományos gyakorlat szerint, ebben a könyvben rendszerint csak a szorzásokat számoljuk. Az egyértelműség kedvéért hozzátesszük a „és hasonló számú összeadás” fordulatot, vagy külön jelezzük, ha mindkét műveletet számoljuk.

A műveletszám és a közelítések pontosságának számszerűsítéséhez gyakran a „nagy O” jelölést használjuk egy függvény nagyságrendjének, illetve domináns tagjának megadására. Műveletszám esetén arra vagyunk kíváncsiak, hogyan viselkedik a függvény, ha a feladat mérete – mondjuk n – nagy lesz. Azt mondjuk, hogy

$$f(n) = \mathcal{O}(g(n))$$

(„f nagy O-ja g-nek”, vagy „f g nagyságrendű”), ha létezik olyan C pozitív állandó, amelyre

$$|f(n)| \leq C|g(n)|$$

minden elég nagy n-re teljesül. Például

$$2n^3 + 3n^2 + n = \mathcal{O}(n^3),$$

mert ahogy n nagy lesz, az n^3-nál kisebb rendű tagok relatíve jelentéktelenné válnak. Pontossági becslésnél egy olyan h mennyiség (például egy lépésköz vagy rácstávolság) kicsivé válását vizsgáljuk. Azt mondjuk, hogy

$$f(h) = \mathcal{O}(g(h)),$$

ha létezik olyan C pozitív állandó, amelyre

$$|f(h)| \leq C|g(h)|$$

minden elég kicsi h-ra teljesül. Például

$$\frac{1}{1-h} = 1 + h + h^2 + h^3 + \cdots = 1 + h + \mathcal{O}(h^2),$$

mert ahogy h kicsivé válik, a h^2-nél magasabb rendű elhagyott tagok relatíve jelentéktelenné válnak. Vegyük észre, hogy a két definíció ekvivalens, ha $h = 1/n$.

Tudományos számítások

1.1 Bevezetés

Ennek a könyvnek a tárgyát hagyományosan numerikus analízisnek nevezik. A numerikus analízis olyan algoritmusok tervezésével és elemzésével foglalkozik, amelyek számos területen – különösen a természettudományokban és a mérnöki tudományokban – felmerülő matematikai feladatokat oldanak meg. Ezért a numerikus analízist az utóbbi időben tudományos számításoknak is nevezik. A tudományos számításokat a számítástudomány legtöbb más ágától az különbözteti meg, hogy folytonos, nem pedig diszkrét mennyiségekkel foglalkozik. Olyan függvényekkel és egyenletekkel dolgozik, amelyek alapul szolgáló változói – idő, távolság, sebesség, hőmérséklet, sűrűség, nyomás, feszültség és hasonlók – természetüknél fogva folytonosak.

A folytonos matematika legtöbb feladata (például szinte bármely feladat, amely deriváltakat, integrálokat vagy nemlinearitást tartalmaz) még elvben sem oldható meg véges számú lépésben egzaktul, ezért ezeket egy (elméletileg végtelen) iteratív eljárással kell megoldani, amely végül a megoldáshoz konvergál. A gyakorlatban természetesen nem iterálunk örökké, hanem csak addig, amíg a válasz megközelítőleg helyes, azaz gyakorlati célokra „elég közel” van a kívánt eredményhez. Így a tudományos számítások egyik legfontosabb aspektusa a gyorsan konvergáló iteratív algoritmusok megtalálása és a kapott közelítés pontosságának felmérése. Ha a konvergencia kellően gyors, akkor még néhány olyan feladat is, amely véges algoritmussal megoldható – mint például a lineáris algebrai egyenletrendszerek –, bizonyos esetekben előnyösebben oldható meg iterációs módszerekkel, amint azt látni fogjuk.

Ebből kifolyólag a tudományos számítások egy másik megkülönböztető jellemzője, hogy foglalkozik a közelítések hatásaival. Sok megoldási technika különféle típusú közelítések egész sorozatát foglalja magában. Még a használt aritmetika is csak közelítő, hiszen a digitális számítógépek nem tudnak minden valós számot egzaktul ábrázolni. A jó algoritmusok szokásos tulajdonságain – például a hatékonyságon – túl a numerikus algoritmusoknak a lehető legmegbízhatóbbnak és legpontosabbnak kell lenniük az útközben alkalmazott különféle közelítések ellenére.

1.1.1 Számítási feladatok

Ahogy a név is sugallja, a tudományos számításokban felmerülő sok feladat a természettudományokból és a mérnöki tudományokból származik, ahol a végső cél valamely természeti jelenség megértése vagy egy eszköz tervezése. A számítógépes szimuláció egy fizikai rendszer vagy folyamat számítógépes ábrázolása és utánzása. A számítógépes szimuláció nagymértékben elmélyítheti a tudományos megértést, mert lehetővé teszi olyan helyzetek vizsgálatát, amelyeket kizárólag elméleti, megfigyelési vagy

kísérleti úton nehéz vagy lehetetlen volna vizsgálni. Az asztrofizikában például két ütköző fekete lyuk részletes viselkedése túl bonyolult ahhoz, hogy elméletileg meghatározzuk, és lehetetlen közvetlenül megfigyelni vagy laboratóriumban reprodukálni. Számítógépes szimulációjához mindössze egy megfelelő matematikai leírásra (ebben az esetben Einstein általános relativitáselméleti egyenleteire), az egyenletek numerikus megoldására szolgáló algoritmusra és egy elegendően nagy számítógépre van szükség, amelyen az algoritmus implementálható.

A számítógépes szimuláció nemcsak egzotikus vagy egyébként hozzáférhetetlen helyzetek feltárására hasznos, hanem a „normális” forgatókönyvek szélesebb körének vizsgálatát is lehetővé teszi, mint amennyit ésszerű költséggel és idővel egyébként vizsgálni lehetne. A mérnöki tervezésben a számítógépes szimuláció lehetővé teszi, hogy sok tervezési alternatívát sokkal gyorsabban, olcsóbban és biztonságosabban próbáljunk ki, mint a fizikai prototípusokat használó hagyományos „építsd meg és teszteld” módszerekkel. Ebben az értelemben a számítógépes szimuláció virtuális prototípuskészítés néven is ismertté vált. A gépjárműbiztonság javításánál például a töréstesztszámítógépen sokkal olcsóbb és veszélytelenebb, mint a valóságban, így a lehetséges tervezési paraméterek tere sokkal alaposabban feltárható egy optimális terv kidolgozásához.

A számítógépes szimuláció átfogó problémamegoldási folyamata általában a következő lépéseket tartalmazza:

- 1. Az érdeklődési körbe tartozó fizikai jelenség vagy rendszer matematikai modelljének kidolgozása – általában valamilyen típusú egyenletekkel megfogalmazva.
- 2. Algoritmusok kidolgozása az egyenletek numerikus megoldására.
- 3. Az algoritmusok implementálása számítógépes szoftverben.
- 4. A szoftver futtatása számítógépen, a fizikai folyamat numerikus szimulálására.
- 5. A számított eredmények valamilyen érthető formában – például grafikus megjelenítéssel – való bemutatása.
- 6. A számított eredmények értelmezése és validálása, szükség esetén az előző lépések bármelyikének megismétlésével.

Az 1. lépést gyakran matematikai modellezésnek nevezik. Ehhez az érintett tudományos vagy mérnöki terület speciális ismerete, valamint az alkalmazott matematikában való jártasság szükséges. A 2. és 3. lépés – numerikus algoritmusok és szoftverek tervezése, elemzése, implementálása és használata – a tudományos számítások fő témája, így e könyvé is. Noha a 2. és 3. lépésre összpontosítunk, lényeges, hogy mindezek a lépések – a probléma megfogalmazásától egészen az eredmények értelmezéséig és validálásáig – megfelelően történjenek, hogy az eredmények értelmesek és hasznosak legyenek. A tudományos számítások elveit és módszereit meglehetősen általános szinten lehet tanulmányozni, amint azt látni fogjuk, de a konkrét feladat konkrét forrását és az eredmények felhasználási módját mindig szem előtt kell tartani, mivel mindegyik szempont hat a többire és fordítva. Például az eredeti feladatmegfogalmazás erősen befolyásolhatja a numerikus eredmények pontosságát, ami viszont kihat az eredmények értelmezésére és validálására.

Egy matematikai feladatot jól kitűzöttnek mondunk, ha létezik megoldása, az egyér-

telmű, és folytonosan függ a feladat adataitól. Az utóbbi feltétel azt jelenti, hogy a feladat adatainak kis megváltozása nem okoz hirtelen, aránytalan változást a megoldásban; ez a tulajdonság különösen fontos a numerikus számításokban, ahol – amint hamarosan látni fogjuk – az ilyen perturbációk általában elkerülhetetlenek. A jól kitűzöttség erősen kívánatos a fizikai rendszerek matematikai modelljeiben, de ez nem mindig érhető el. Például egy fizikai rendszer belső szerkezetére csak külső megfigyelésekből való következtetés – mint a tomográfiában vagy a szeizmológiában – gyakran eredendően rosszul kitűzött matematikai feladatokra vezet, amelyekben egymástól jelentősen eltérő belső konfigurációk megkülönböztethetetlen külső megjelenéssel rendelkeznek.

Még ha egy feladat jól kitűzött is, a megoldás akkor is nagyon érzékenyen (bár folytonosan) reagálhat a feladat adataiban bekövetkező perturbációkra. Ahhoz, hogy felmérjük az ilyen perturbációk hatásait, a folytonosság kvalitatív fogalmán túl kell lépnünk, és meg kell határoznunk a feladat érzékenységének mennyiségi mértékét. Ezenfelül ügyelnünk kell arra is, hogy az adott feladat numerikus megoldására használt algoritmus ne tegye az eredményeket érzékenyebbé annál, mint ami az alapul szolgáló feladatban eleve benne rejlik (a „ne árts!” hippokratészi eskü a numerikus matematikusokra éppúgy vonatkozik, mint az orvosokra). Ez a követelmény vezet el a stabil algoritmus fogalmához. Ezeket az általános fogalmakat és kérdéseket ebben a fejezetben vezetjük be, majd a későbbi fejezetekben részletesen is tárgyaljuk őket a konkrét típusú számítási feladatok kapcsán.

1.1.2 Általános stratégia

Egy adott számítási feladat megoldása során – és ez a könyvön végigvonuló alapvető stratégia – az a célunk, hogy egy nehéz feladatot egy könnyebbre cseréljünk, amelynek ugyanaz a megoldása, vagy legalábbis nagyon közel áll ahhoz. Ennek a megközelítésnek a példái:

- Végtelendimenziós terek helyettesítése végesdimenziós terekkel
- Végtelen folyamatok helyettesítése véges folyamatokkal, például integrálok vagy végtelen sorok helyettesítése véges összegekkel, illetve a deriváltak helyettesítése véges differenciákkal
- Differenciálegyenletek helyettesítése algebrai egyenletekkel
- Nemlineáris feladatok helyettesítése lineáris feladatokkal
- Magasabb rendű rendszerek helyettesítése alacsonyabb rendű rendszerekkel
- Bonyolult függvények helyettesítése egyszerű függvényekkel, például polinomokkal
- Általános mátrixok helyettesítése egyszerűbb alakú mátrixokkal

Például egy nemlineáris differenciálegyenlet-rendszert először egy nemlineáris algebrai egyenletrendszerrel helyettesíthetünk, majd a nemlineáris algebrai rendszert egy lineáris algebrai rendszerrel, végül pedig a lineáris rendszer mátrixát olyan speciális alakú mátrixra cserélhetjük, amellyel a megoldás már könnyen kiszámítható. E folyamat minden lépésében meg kell győződnünk arról, hogy a megoldás nem változott meg, vagy legalábbis a valódi megoldáshoz képest egy előírt tűréshatáron belül marad.

Ahhoz, hogy ez az általános stratégia egy adott feladatra működjön, a következők

megléte szükséges:

- Egy olyan alternatív feladat vagy feladatosztály, amely könnyebben megoldható
- Az eredeti feladatnak egy olyan transzformációja ebbe az alternatív típusba, amely valamilyen értelemben megőrzi a megoldást

Így erőfeszítéseink nagy része olyan megfelelő feladatosztályok azonosítására irányul, amelyeknek egyszerű megoldásuk van, valamint az ilyen osztályokba vezető, megoldástartó transzformációk megtalálására.

Ideális esetben a transzformált feladat megoldása azonos az eredeti feladat megoldásával, de ez nem mindig lehetséges. Az utóbbi esetben a megoldás csak közelíti az eredeti feladatét, de a pontosság általában tetszőlegesen javítható további számítási munka és tárhely árán. Ezért elsődleges feladatunk az ilyen közelítő megoldás pontosságának becslése és annak igazolása, hogy határértékben a valódi megoldáshoz konvergál.

1.2 Közelítések a tudományos számításokban

1.2.1 A közelítés forrásai

A tudományos számításokban a közelítésnek, illetve a pontatlanságnak számos forrása van. Néhány közelítés már a számítás megkezdése előtt felléphet:

- Modellezés: A vizsgált feladat vagy rendszer bizonyos fizikai jellemzőit leegyszerűsíthetjük vagy elhagyhatjuk (pl. súrlódás, viszkozitás, légellenállás).
- Empirikus mérések: A laboratóriumi műszereknek véges a pontossága. Pontosságukat tovább korlátozhatja a kis mintaelemszám, vagy a leolvasott értékeket véletlen zaj, illetve szisztematikus torzítás is terhelheti. Például még a fontos fizikai állandók – mint a Newton-féle gravitációs állandó vagy a Planck-állandó – legalaposabb méréseiből is jellemzően legfeljebb nyolc vagy kilenc értékes jegyet kapunk, a legtöbb laboratóriumi mérés pedig ennél jóval pontatlanabb.
- Korábbi számítások: A bemenő adatok egy korábbi számítási lépésből származhatnak, amelynek eredményei csak közelítők voltak.

Az imént felsorolt közelítésekre általában nincs ráhatásunk, de fontos szerepet játszanak annak meghatározásában, hogy egy számítástól milyen pontosság várható. Figyelmünket elsősorban azokra a közelítésekre fordítjuk, amelyeket képesek vagyunk befolyásolni. Ezek a számítás közben fellépő szisztematikus közelítések a következők:

• Csonkítás vagy diszkretizáció: egy matematikai modell bizonyos jellemzőit elhagyhatjuk vagy leegyszerűsíthetjük (pl. deriváltakat véges differenciákkal helyettesítünk, vagy egy végtelen sornak csak véges sok tagját használjuk).

• Kerekítés: legyen szó kézi számításról, zsebszámológépről vagy digitális számítógépről, a valós számok és a rajtuk végzett aritmetikai műveletek ábrázolása végső soron véges pontosságra korlátozódik, így általában nem egzakt.

Egy számítás végeredményének pontossága ezen közelítések bármelyikéből, vagy akár mindegyikének együttes hatásából is fakadhat, a fellépő perturbációkat pedig a megoldandó feladat természete vagy a használt algoritmus – netán mindkettő – fel is erősítheti. Az ilyen közelítéseknek a numerikus algoritmusok pontosságára

és stabilitására gyakorolt hatásának vizsgálatát hagyományosan hibaelemzésnek nevezik.

1.1. Példa. Közelítések. A Föld felszínét egy r sugarú gömb felszínképletével, az

$$A = 4\pi r^2$$

összefüggéssel számíthatjuk ki. Ennek a képletnek a használata a számításban számos közelítést foglal magában:

- A Földet gömbként modellezzük, ami a tényleges alakjának idealizálása.
- Az $r \approx 6370$ km sugárérték empirikus mérések és korábbi számítások kombinációján alapul.
- A π értékét egy végtelen határérték-folyamat adja meg, amelyet valahol csonkolni kell.
- A bemenő adatok numerikus értékei, valamint a rajtuk végzett aritmetikai műveletek eredményei egy számítógépen vagy számológépen kerekítéssel állnak elő.

A számított eredmény pontossága mindezektől a közelítésektől függ.

1.2.2 Abszolút hiba és relatív hiba

Egy hiba jelentősége nyilvánvalóan összefügg a mért vagy számított mennyiség nagyságával. Például egy egységnyi hiba sokkal kisebb jelentőségű a Föld népességének számolásánál, mint egy telefonfülke lakóinak számolásánál. Ez motiválja az *abszolút hiba* és a *relatív hiba* fogalmait, amelyeket a következőképpen definiálunk:

$$\text{abszolút hiba} = \text{közelítő érték} - \text{valódi érték},$$

$$\text{relatív hiba} = \frac{\text{abszolút hiba}}{\text{valódi érték}}.$$

Egyes szerzők az abszolút hibát a fenti különbség abszolút értékeként definiálják, de mi az abszolút értéket (vagy vektorok és mátrixok esetén a normát) csak akkor alkalmazzuk, amikor kifejezetten a hiba nagyságára van szükségünk. Megjegyezzük, hogy a relatív hiba nincs definiálva, ha a valódi érték nulla.

A relatív hiba százalékban is kifejezhető, ami egyszerűen a relatív hiba 100-szorosa. Így például egy 0,1-es abszolút hiba a 10-es valódi értékhez viszonyítva 0,01, azaz 1 százalékos relatív hibának felel meg. Egy teljesen hibás közelítés legalább 1, azaz legalább 100 százalékos relatív hibának felelne meg, ami azt jelenti, hogy az abszolút hiba akkora, mint maga a valódi érték.

A relatív hiba egy másik értelmezése: ha egy közelítő érték relatív hibája körülbelül 10^{-p}, akkor tizedestört alakja körülbelül p helyes értékes jegyet tartalmaz (az értékes jegyek az első nem nulla számjegytől kezdve az azt követő összes jegyet jelentik). Ezzel kapcsolatban érdemes különbséget tenni a *precízió* és a *pontosság* között: a precízió azon jegyek számára utal, amelyekkel a számot kifejezzük, a pontosság pedig a kívánt mennyiség közelítésében helyes értékes jegyek számára (azaz a relatív hibára).

Például a 3,252603764690804 egy nagyon precíz szám, de a π közelítéseként nem túl pontos. Amint hamarosan látni fogjuk, az, hogy egy mennyiséget adott precízióval számítunk, önmagában nem jelenti, hogy az eredmény ugyanilyen mértékben lesz pontos.

Az abszolút és relatív hiba közötti összefüggés egy hasznos alakja a következő:

$$\text{közelítő érték} = (\text{valódi érték}) \times (1 + \text{relatív hiba}).$$

Természetesen általában nem ismerjük a valódi értéket; ha ismernénk, nem kellene bajlódnunk a közelítésével. Ezért a hibát rendszerint csak becsüljük vagy felső korlátot adunk rá, nem pedig egzaktul kiszámítjuk, hiszen a valódi érték ismeretlen. Ugyanezen okból a relatív hibát gyakran a közelítő értékhez, nem pedig a valódi értékhez viszonyítva számítják, eltérően a fenti definíciótól.

1.2.3 Adathiba és számítási hiba

Amint láttuk, egyes hibák a bemenő adatoknak tulajdoníthatók, míg mások a későbbi számítási folyamatok során keletkeznek. Bár ez a megkülönböztetés nem mindig egyértelmű (a kerekítés például érintheti a bemenő adatokat és a későbbi számítási eredményeket is), mégis segít megérteni a közelítéseknek a numerikus számításokra gyakorolt összhatását.

A legtöbb realisztikus feladat többdimenziós, de az egyszerűség kedvéért ebben a fejezetben csak egydimenziós feladatokat vizsgálunk; a definíciók és eredmények magasabb dimenziókra való kiterjesztése egyértelmű, általában csak az abszolút értékek megfelelő normákra cserélését igényli (lásd a 2.3.1. szakaszt). Egy tipikus egydimenziós feladat felfogható egy függvény – mondjuk $f : \mathbb{R} \to \mathbb{R}$ – értékének számításaként, amely egy adott bemenő értéket egy kimenő eredményhez rendel. Jelölje a bemenet valódi értékét x, így a kívánt valódi eredmény $f(x)$. Tegyük fel, hogy pontatlan bemenettel, mondjuk $\hat{x}$-szel kell dolgoznunk, és csak egy közelítést – mondjuk $\hat{f}$-et – tudunk számítani a függvényértékre. Ekkor azzal a jól ismert matematikai trükkel, hogy ugyanazt a mennyiséget hozzáadjuk és kivonjuk úgy, hogy az összeg változatlan maradjon, a következőt kapjuk:

$$\begin{aligned}
\text{teljes hiba} &= \hat{f}(\hat{x}) - f(x) \\
&= \big(\hat{f}(\hat{x}) - f(\hat{x})\big) + \big(f(\hat{x}) - f(x)\big) \\
&= \text{számítási hiba} + \text{továbbterjedő adathiba}.
\end{aligned}$$

Ebben az összegben az első tag a pontos és a közelítő függvény különbsége ugyanazon bemenet mellett, tehát tiszta *számítási hibának* tekinthető. A második tag a pontos függvényértékek különbsége a bemenet hibája miatt, tehát tiszta *továbbterjedő adathibának* tekinthető. Megjegyezzük, hogy az algoritmus megválasztása nincs hatással a továbbterjedő adathibára.

1.2. Példa. Adathiba és számítási hiba. Tegyük fel, hogy nincs számítógépünk vagy számológépünk, és „gyors, de durva” közelítésre van szükségünk a $\sin(\pi/8)$

értékére. Először a π valamilyen értékére van szükségünk a bemenethez. Röviden fontolóra vesszük a $\pi \approx 22/7$ klasszikus, iskoláskorunkból megmaradt közelítést, de úgy döntünk, túl sok munka volna átváltani a kívánt tizedestört alakba, és inkább a $\pi \approx 3$ egyszerű „bibliai” közelítésnél maradunk, így a tényleges bemenetünk $3/8$. A függvényérték kiszámításához felidézzük az analízisből, hogy kis argumentumokra jó közelítést ad a Taylor-sor első tagja, ami $\sin(x)$ esetén egyszerűen x. Végeredményünk tehát

$$\sin(\pi/8) \approx \sin(3/8) \approx 3/8 = 0{,}3750.$$

A fenti közelítések közül az első – egy perturbált $\hat{x} = 3/8$ bemenet használata a valódi $x = \pi/8$ helyett – továbbterjedő adathibát okoz: még ha egzaktul számítanánk is ki a szinuszfüggvényt, hibás eredményt kapnánk, mert hibás bemenetet használtunk. A második közelítés egy számítási hibát jelent, noha a „számításunk” ebben az esetben csupán a bemenet lemásolása volt! (A számítási hibák gyakran ilyen „elhagyásból eredő” hibák, bár általában nem ennyire szélsőségesek.) A korábban bevezetett jelölésben az $\hat{f}(x) = x$ csonkolt matematikai kifejezést használtuk a valódi $f(x) = \sin(x)$ függvény helyett, ami azt jelenti, hogy még helyes bemenet esetén is helytelen eredményt kaptunk volna. A végeredmény pontosságát e két közelítés kombinációja határozza meg.

Később – miután hozzáférünk egy zsebszámológéphez – megállapítjuk, hogy négy tizedesjegy pontossággal a helyes válasz

$$\sin(\pi/8) \approx 0{,}3827,$$

így a teljes hiba

$$\hat{f}(\hat{x}) - f(x) \approx 0{,}3750 - 0{,}3827 = -0{,}0077.$$

Észrevéve, hogy a perturbált bemenetre a helyes válasz

$$f(\hat{x}) = \sin(3/8) \approx 0{,}3663,$$

látjuk, hogy a pontatlan bemenet használata által okozott továbbterjedő adathiba

$$f(\hat{x}) - f(x) = \sin(3/8) - \sin(\pi/8) \approx 0{,}3663 - 0{,}3827 = -0{,}0164.$$

A végtelen sor csonkolásából adódó számítási hiba pedig

$$\hat{f}(\hat{x}) - f(\hat{x}) = 3/8 - \sin(3/8) \approx 0{,}3750 - 0{,}3663 = 0{,}0087.$$

E két hiba összege adja a megfigyelt teljes hibát. Ebben a konkrét példában a két hibának ellentétes az előjele, tehát részben kioltják egymást; más körülmények között azonos előjelűek is lehetnek, és ekkor felerősítik egymást. Ennél a bemenetnél

a továbbterjedő adathiba és a számítási hiba nagyságrendileg hasonló – körülbelül kétszeres szorzóval tér el –, de bármelyik hibaforrás dominánssá válhat más bemenő értékek esetén. Ha a π-re és a $\sin(x)$-re ugyanezeket a közelítéseket használjuk, a továbbterjedő adathiba sokkal kisebb bemeneteknél lenne domináns, míg a számítási hiba sokkal nagyobb bemeneteknél (Miért?). A teljes hiba csökkentésére használhatnánk pontosabb értéket π-re (ami csökkentené a továbbterjedő adathibát), vagy pontosabb matematikai leírást $\sin(x)$-re (pl. a végtelen sor több tagját), ami csökkentené a számítási hibát.

1.2.4 Csonkítási hiba és kerekítési hiba

A számítási hiba (azaz a számítás során keletkező hiba) tovább bontható *csonkítási* (vagy *diszkretizációs*) *hibára* és *kerekítési hibára*:

- A csonkítási hiba a valódi eredmény (a tényleges bemenet mellett) és az adott algoritmus egzakt aritmetikával kapott eredménye közötti különbség. Olyan közelítésekből ered, mint egy végtelen sor csonkolása, a deriváltak végesdifferencia-formulákkal való helyettesítése, vagy egy iterációs eljárás konvergencia előtti megszakítása.
- A kerekítési hiba az adott algoritmus egzakt aritmetikával kapott eredménye és ugyanezen algoritmus véges precíziójú, kerekített aritmetikával számított eredménye közötti különbség. A valós számok és a rajtuk végzett aritmetikai műveletek ábrázolásának pontatlanságából ered, amelyet az 1.3. szakaszban részletesen is tárgyalunk.

A definíció szerint tehát a számítási hiba egyszerűen a csonkítási hiba és a kerekítési hiba összege. Az 1.2. példában a bemenetet kerekítettük, de a számítás során nem volt kerekítési hiba, így a számítási hiba kizárólag a végtelen sor egyetlen tagjának használatából eredő csonkítási hibából állt. Több tag használata a sorban csökkentette volna a csonkítási hibát, de valószínűleg némi kerekítési hibát is bevezetett volna a sor kiszámításához szükséges aritmetikába. A csonkítási és a kerekítési hibák közötti ehhez hasonló kompromisszumok egyáltalán nem szokatlanok.

1.3. Példa. Végesdifferencia-közelítés. Egy differenciálható $f : \mathbb{R} \to \mathbb{R}$ függvényre tekintsük az első derivált végesdifferencia-közelítését:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}.$$

A Taylor-tétel szerint

$$f(x+h) = f(x) + f'(x)h + f''(\theta)h^2/2$$

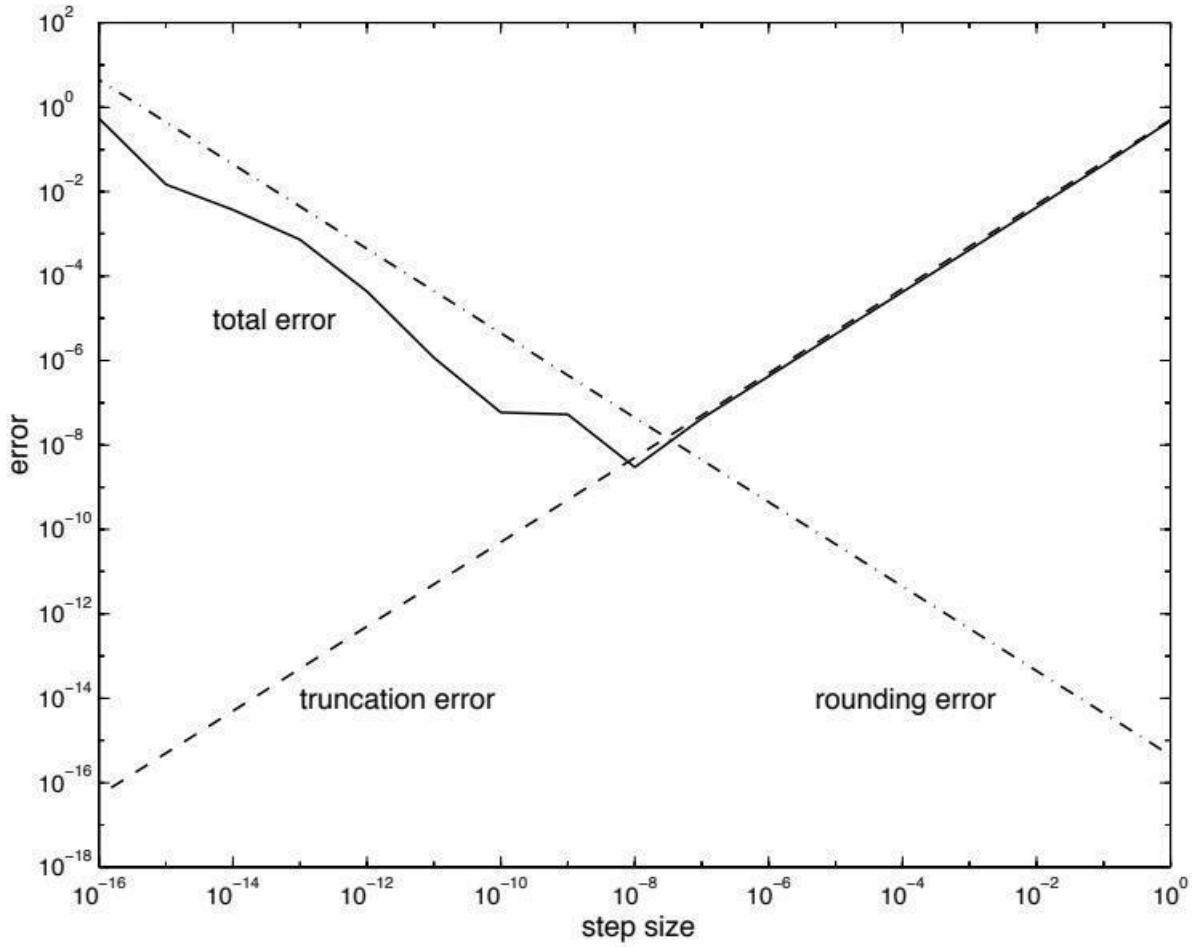
valamely $\theta \in [x, x+h]$ esetén, így a végesdifferencia-közelítés csonkítási hibája felülről korlátozható $Mh/2$-vel, ahol M egy $|f''(t)|$-re adott felső korlát az x környezetében. Feltéve, hogy a függvényértékek hibáját ϵ korlátozza, a végesdifferencia-képlet kiszámításának kerekítési hibája $2\epsilon/h$-val becsülhető. A teljes számítási hiba így felülről korlátozható két tag összegével:

$$\frac{Mh}{2} + \frac{2\epsilon}{h},$$

ahol az első tag h csökkenésével egyre kisebb lesz, a második pedig egyre nő. Ezért a h lépésköz megválasztásánál kompromisszum áll fenn a csonkítási hiba és a kerekítési hiba között. Ezt a kifejezést h szerint deriválva és a deriváltat nullával egyenlővé téve látható, hogy a teljes számítási hibára adott korlát a minimumát

$$h = 2\sqrt{\epsilon/M}$$

esetén veszi fel. Egy tipikus példa az 1.1. ábrán látható, ahol a végesdifferencia-közelítés teljes számítási hibáját – a csonkítási és kerekítési hibákra adott külön-külön korlátokkal együtt – az $f(x) = \sin(x)$ függvényre $x = 1$-ben ábrázoltuk a h lépésköz függvényében, $M = 1$ mellett és egy $\epsilon \approx 10^{-16}$ precíziójú számítógépen. Láthatjuk, hogy a teljes hiba valóban $h \approx 10^{-8} \approx \sqrt{\epsilon}$ környékén éri el a minimumát. A teljes hiba a túl nagy h értékek esetén a növekvő csonkítási hiba miatt, a túl kicsi h értékek esetén pedig a növekvő kerekítési hiba miatt indul növekedésnek.



1.1. ábra: A végesdifferencia-közelítés számítási hibája adott lépésközre.

A csonkítási hiba csökkenthető lenne pontosabb végesdifferencia-formula használatával, például a középpontos differencia-közelítéssel (lásd a 8.6.1. szakaszt):

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

A kerekítési hiba magasabb precíziójú aritmetikával csökkenthető, ha rendelkezésre áll.

Bár egy adott számításban a csonkítási hiba és a kerekítési hiba egyaránt fontos szerepet játszhat, általában egyik vagy másik válik a teljes számítási hiba domináns tényezőjévé. Nagyjából elmondható, hogy a kerekítési hiba rendszerint a tisztán algebrai, véges megoldási algoritmusú feladatokban dominál, míg a csonkítási hiba az integrálokat, deriváltakat vagy nemlinearitást tartalmazó feladatokban, amelyek gyakran elméletileg végtelen megoldási folyamatot igényelnek.

A különböző hibatípusok között tett megkülönböztetéseink fontosak a numerikus algoritmusok viselkedésének és a pontosságukat befolyásoló tényezők megértéséhez, de az egyes hibatípusokat általában nem szükséges – és nem is mindig lehetséges – pontosan kvantifikálni. Valójában, amint hamarosan látni fogjuk, gyakran előnyös, ha az összes hibát összevonjuk és a bemenő adatok hibájára visszavezetjük.

1.2.5 Előreható hiba és hátraható hiba

Egy számított eredmény minősége egyéb tényezők mellett a bemenő adatok minőségétől is függ. Ha például a bemenő adatok csak – mondjuk – négy értékes jegyre pontosak, akkor legfeljebb négy értékes jegyet várhatunk a számított eredményben is, bárhogy is végezzük a számítást. A jól ismert számítástechnikai alapigazság, a „szemét megy be, szemét jön ki” (garbage in, garbage out) ezt a megfigyelést viszi a logikus végletekig. Ezért a számított eredmények minőségének értékelésekor nem szabad figyelmen kívül hagynunk a bemenő adatok – bizonytalansági szintjükön belüli – perturbációinak lehetséges hatásait.

1.4. Példa. Az adathiba hatásai. Tegyük fel, hogy egy ország népességét szeretnénk előrejelezni tíz év múlva. Először egy matematikai modellre van szükségünk, amely leírja a népesség időbeli változásait. Feltéve, hogy a születések és a halálozások egyaránt arányosak a mindenkori népességgel, a

$$P(t + \Delta t) = P(t) + (B - D)P(t)\Delta t$$

egyszerű modellhez jutunk, ahol $P(t)$ a népességet jelöli a t időpontban, Δt egy adott időintervallum (mondjuk egy év), B és D pedig a születési és halálozási ráta (például a nettó növekedési ráta $B - D = 0{,}04$, azaz évi 4 százalék lehet). Használhatjuk ezt a diszkrét modellt egy rögzített Δt időintervallummal, vagy vehetjük a $\Delta t \to 0$ határátmenetet, hogy a

$$dP(t)/dt = (B - D)P(t)$$

differenciálegyenlethez jussunk, amelynek megoldása a jól ismert exponenciális növekedési törvény:

$$P(t) = P(0)\exp\big((B-D)t\big).$$

Mindkét esetben a modell nyilvánvalóan csak közelítése a valóságnak; például elhanyagoltuk a bevándorlást és a korlátozott eltartóképesség (kapacitás) esetleges hatásait. Bár e szemléltetésben az ilyen modellezési hibákat figyelmen kívül hagyjuk, szinte minden valós tudományos feladatban való jelenlétük miatt mérsékelnünk kell a további számításokban elérhető pontossággal szembeni elvárásainkat. Akár a diszkrét, akár a folytonos modellt használjuk népesség-előrejelzéshez, ismernünk kell a jelenlegi népességet, valamint a születési és halálozási rátákat. Közismert, hogy egy népszámlálásnál nehéz mindenkit megszámolni anélkül, hogy valakit kihagynánk, így a kiindulási népesség csak korlátozott pontossággal ismert. Ennek megfelelően a kiindulási népességet rendszerint „kerek számokkal” (azaz csak néhány értékes jeggyel) adják meg, utalva az érték bizonytalansági szintjére. Ezt nem szabad kerekítési hibának tekinteni, mert a kapott érték éppúgy lehet helyes, mint egy precízebbnek tűnő érték, hiszen az elhagyott jegyek kétségesek vagy értelmetlenek voltak. Hasonlóképpen a születési és halálozási ráták is – amelyek sok diszkrét eseményre vett átlagok – csak korlátozott pontossággal ismertek.

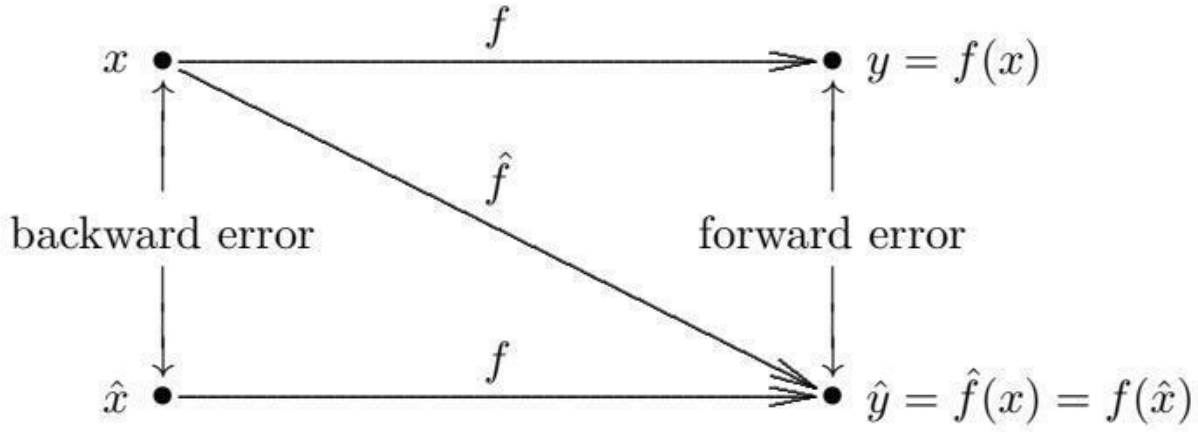
A modellünk bemenetében lévő bizonytalanságok szükségszerűen bizonyos bizonytalanságot okoznak a kapott népesség-előrejelzésben is. Következésképpen úgy tekinthetjük a modellünket, hogy a bemeneti tér egy „elmosódott” tartományát a kimeneti tér egy „elmosódott” tartományára képezi le (ez utóbbi az összes lehetséges bemenetkombináció bizonytalansági tartományán belül adódó összes lehetséges eredmény halmaza). A modell implementálása során némi számítási hiba (csonkítási vagy kerekítési hiba) léphet fel, de amíg az előállított eredmény a bemenő adatok bizonytalanságának megfelelő kimeneti tér elmosódott tartományában marad, addig az eredmény aligha kifogásolható. Másként fogalmazva: bármely eredmény – bárhogy is kaptuk –, amely egzakt eredménye egy ugyanolyan valószínűséggel helyes bemenetnek, mint amilyet ténylegesen használtunk, annyira jó válasz, amennyire csak joggal elvárhatunk.

Most formalizáljuk ezeket a fogalmakat, ismét az egyszerűség kedvéért egydimenziós feladatokra összpontosítva. Tegyük fel, hogy egy $y = f(x)$ függvényérték kiszámítása a cél, ahol $f : \mathbb{R} \to \mathbb{R}$, de helyette egy közelítő $\widehat{y}$ értéket kapunk. A számított és a valódi érték közötti eltérést, $\Delta y = \widehat{y} - y$, *előreható hibának* nevezzük. A számított eredmény minősége értékelésének egyik módja, hogy megpróbáljuk megbecsülni ennek az előreható hibának a relatív nagyságát, ami a konkrét körülményektől függően lehet könnyű vagy nehéz. Általában azonban egy számításban a hibák előrefelé történő terjedésének (propagálásának) elemzése – a későbbiekben látható okokból – gyakran nehéz. Sőt, a minden szakaszban feltételezett legrosszabb esetek gyakran nagyon pesszimista korláthoz vezetnek a teljes hibára nézve.

Egy alternatív megközelítés, ha a kapott közelítő megoldást úgy tekintjük, mint egy módosított feladat egzakt megoldását, és megkérdezzük: mekkora módosítás szükséges az eredeti feladathoz ahhoz, hogy éppen a ténylegesen kapott eredményt adja? Más szóval: mekkora adathibára volna szükség a kiindulási bemenetben, hogy megmagyarázza a végső számított eredmény összes hibáját? Formálisabban a $\Delta x = \widehat{x} - x$

mennyiséget – ahol $f(\hat{x}) = \hat{y}$ – *hátraható hibának* nevezzük, amelynek relatív nagyságát a hátraható hibaelemzésben próbáljuk megbecsülni. Ebből a nézőpontból egy adott feladatra adott közelítő megoldás akkor jó, ha egy „közeli” feladat egzakt megoldása (azaz a relatív hátraható hiba kicsi). Ha pedig a közeli feladat a bemenő adatok bizonytalanságán belül van, akkor a $\hat{y}$ számított megoldás valójában lehet „a” valódi megoldás, amennyire tudhatjuk (vagy tudhatnánk, tekintve a bemenet bizonytalanságát), tehát aligha kifogásolható.

Ezeket az összefüggéseket sematikusan (nem méretarányosan) az 1.2. ábra szemlélteti, ahol x és f rendre az egzakt bemenet és az egzakt függvény, $\hat{f}$ a ténylegesen számított közelítő függvény, $\hat{x}$ pedig egy olyan bemeneti érték, amelyre az egzakt függvény éppen ezt a számított eredményt adná. Megjegyezzük, hogy a $\hat{f}(x) = f(\hat{x})$ egyenlőség éppen $\hat{x}$ választásából adódik; valójában ez a követelmény definiálja $\hat{x}$-et. Az 1.2.6. szakaszban számszerűsítjük az előreható és a hátraható hiba közötti kapcsolatot.



1.2. ábra: Sematikus ábra az előreható és a hátraható hiba szemléltetésére.

1.5. Példa. Előreható és hátraható hiba. A $y = \sqrt{2}$ közelítéseként a $\hat{y} = 1{,}4$ érték abszolút előreható hibája

$$|\Delta y| = |\hat{y} - y| = |1{,}4 - 1{,}41421\ldots| \approx 0{,}0142,$$

azaz körülbelül 1 százalékos relatív előreható hiba. A hátraható hiba meghatározásához megfigyeljük, hogy $\sqrt{1{,}96} = 1{,}4$, tehát az abszolút hátraható hiba

$$|\Delta x| = |\hat{x} - x| = |1{,}96 - 2| = 0{,}04,$$

azaz 2 százalékos relatív hátraható hiba.

1.6. Példa. Hátraható hibaelemzés. Tegyük fel, hogy egyszerű közelítést keresünk a $y = f(x) = \cos(x)$ koszinuszfüggvényre az $x = 1$ helyen. A koszinuszfüggvényt a

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots$$

végtelen sor adja meg, így fontolóra vehetjük a sor csonkolását, mondjuk két tag után, hogy a

$$\hat{y} = \hat{f}(x) = 1 - x^2/2$$

közelítést kapjuk. Ekkor a közelítés előreható hibája

$$\Delta y = \hat{y} - y = \hat{f}(x) - f(x) = 1 - x^2/2 - \cos(x).$$

A hátraható hiba meghatározásához meg kell találnunk azt a $\hat{x}$ bemenő értéket f-re, amely a ténylegesen kapott $\hat{y}$ kimenő értéket adja, azaz amelyre $\hat{f}(x) = f(\hat{x})$. A koszinuszfüggvény esetén ezt az értéket

$$\hat{x} = \arccos(\hat{f}(x)) = \arccos(\hat{y})$$

adja meg. Így $x = 1$ esetén:

$$y = f(1) = \cos(1) \approx 0{,}5403,$$

$$\hat{y} = \hat{f}(1) = 1 - 1^2/2 = 0{,}5,$$

$$\hat{x} = \arccos(\hat{y}) = \arccos(0{,}5) \approx 1{,}0472,$$

$$\text{előreható hiba} = \Delta y = \hat{y} - y \approx 0{,}5 - 0{,}5403 = -0{,}0403,$$

$$\text{hátraható hiba} = \Delta x = \hat{x} - x \approx 1{,}0472 - 1 = 0{,}0472.$$

Az előreható hiba arra utal, hogy a pontosság meglehetősen jó, mert a kimenet közel van ahhoz, amit számítani szerettünk volna; a hátraható hiba pedig arra utal, hogy a pontosság jó, mert a kapott kimenet helyes egy csak kissé perturbált bemenetre. A következőkben látni fogjuk, hogyan kapcsolódik az előreható és a hátraható hiba egymáshoz számszerűen.

1.2.6 Érzékenység és kondicionáltság

Egy pontatlan megoldás nem feltétlenül egy rosszul megtervezett algoritmus következménye; a pontatlanság a megoldandó feladat eredendő tulajdonsága is lehet. Még egzakt számítás esetén is előfordulhat, hogy a feladat megoldása rendkívül érzékeny a bemenő adatok perturbációira. Az érzékenység kvalitatív fogalma és annak kvantitatív mértéke – a kondicionáltság – a továbbterjedő adathibával foglalkozik, azaz a bemenő adatok perturbációinak a megoldásra gyakorolt hatását vizsgálja.

Egy feladatot érzéketlennek, vagy *jól kondicionáltnak* nevezünk, ha a bemenő adatok egy adott relatív változása a megoldásban azzal összemérhető (vele arányos) relatív változást okoz. Egy feladatot érzékenynek, vagy *rosszul kondicionáltnak* nevezünk, ha a megoldásban bekövetkező relatív változás nagyságrendekkel nagyobb lehet, mint a bemenő adatokban bekövetkező változás. Aki valaha tapasztalta már, hogy a zuhany vize a hőmérséklet-szabályozó legkisebb elállítására is fagyosból forróvá (vagy fordítva) vált, az első kézből tapasztalhatta meg, milyen egy érzékeny rendszer: a hatás aránytalanul nagy a kiváltó okhoz képest.

Pontosabban, egy feladat *kondíciószámát* a megoldás relatív változásának és a bemenet relatív változásának hányadosaként definiáljuk. Egy feladat akkor rosszul kondicionált (azaz érzékeny), ha a kondíciószáma sokkal nagyobb 1-nél. Egy függvény kiszámításának korábbi példájában használt jelölésekkel:

$$\text{kondíciószám} = \frac{\big|(f(\hat{x}) - f(x))/f(x)\big|}{\big|(\hat{x} - x)/x\big|} = \frac{|(\hat{y} - y)/y|}{|(\hat{x} - x)/x|} = \frac{|\Delta y/y|}{|\Delta x/x|}.$$

A tört számlálójában és nevezőjében rendre a relatív előreható, illetve a relatív hátraható hibát ismerhetjük fel, így az összefüggés a következőképpen is felírható:

$$\big|\text{relatív előreható hiba}\big| = \text{kondíciószám} \times \big|\text{relatív hátraható hiba}\big|.$$

A kondíciószám tehát egyfajta „erősítési tényezőként” értelmezhető, amely összekapcsolja az előreható és a hátraható hibát. Ha egy feladat rosszul kondicionált (azaz a kondíciószáma nagy), akkor a relatív előreható hiba (a megoldás relatív perturbációja) akkor is nagy lehet, ha a relatív hátraható hiba (a bemenet relatív perturbációja) csekély.

A kondíciószám általában a bemenettől függően változik, és a gyakorlatban ritkán ismerjük a pontos értékét. Ezért gyakran meg kell elégednünk a kondíciószám durva becslésével vagy egy adott bemeneti tartományra vonatkozó felső korlátjával; emiatt a hátraható és az előreható hiba közötti összefüggés közelítő egyenlőtlenséggé válik:

$$\big|\text{relatív előreható hiba}\big| \lesssim \text{kondíciószám} \times \big|\text{relatív hátraható hiba}\big|,$$

amely felülről korlátozza a legrosszabb esetben fellépő előreható hibát, de nem feltétlenül éles (nem feltétlenül valósul meg) minden bemenet esetén. Ezen összefüggés alapján a kondíciószám lehetővé teszi a (számunkra általában legfontosabb) előreható hiba becslését a (gyakran könnyebben meghatározható) hátraható hiba függvényében.

Egy differenciálható $f : \mathbb{R} \to \mathbb{R}$ függvény kiszámításának kondíciószámát az analízis eszközeivel közelíthetjük:

$$\text{abszolút előreható hiba} = f(x + \Delta x) - f(x) \approx f'(x)\Delta x,$$

így

$$\text{relatív előreható hiba} = \frac{f(x+\Delta x) - f(x)}{f(x)} \approx \frac{f'(x)\Delta x}{f(x)},$$

és ezért

$$\text{kondíciószám} \approx \left| \frac{f'(x)\Delta x / f(x)}{\Delta x / x} \right| = \left| \frac{x\, f'(x)}{f(x)} \right|.$$

Így a kimenő függvényérték relatív hibája sokkal nagyobb, de akár sokkal kisebb is lehet a bemenet relatív hibájánál, a szóban forgó függvény tulajdonságaitól és a bemenet konkrét értékétől függően.

Egy adott feladat *inverz feladata* annak meghatározása, hogy melyik bemenet eredményezne egy adott kimenetet. Egy $y = f(x)$ függvény esetében az inverz feladat – jelölésben $x = f^{-1}(y)$ – az, hogy egy adott y értékhez meghatározzuk azt az x értéket, amelyre $f(x) = y$. A definícióból jól látható, hogy az inverz feladat kondíciószáma az eredeti feladat kondíciószámának reciproka. Következésképpen, ha a kondíciószám közel van 1-hez, akkor mind a feladat, mind annak inverze jól kondicionált. Ha azonban a kondíciószám sokkal nagyobb vagy sokkal kisebb 1-nél, akkor rendre a feladat vagy az inverze rosszul kondicionált.

Felidézve az analízisből tanultakat, ha g az inverz függvény, azaz $g(y) = f^{-1}(y)$, és x és y olyan értékek, amelyekre $y = f(x)$, akkor $g'(y) = 1/f'(x)$, feltéve, hogy $f'(x) \neq 0$. Így a g inverz függvény kondíciószáma:

$$\text{kondíciószám} \approx \left| \frac{y\, g'(y)}{g(y)} \right| = \left| \frac{f(x)\,(1/f'(x))}{x} \right| = \left| \frac{f(x)}{x\, f'(x)} \right|,$$

ami az eredeti f függvény kondíciószámának reciproka.

1.7. Példa. Kondíciószám. Tekintsük az $f(x) = \sqrt{x}$ függvényt. Mivel $f'(x) = 1/(2\sqrt{x})$,

$$\text{kondíciószám} \approx \left| \frac{x f'(x)}{f(x)} \right| = \left| \frac{x/(2\sqrt{x})}{\sqrt{x}} \right| = \frac{1}{2}.$$

Ez azt jelenti, hogy a bemenet adott relatív változása a kimenetben körülbelül fele akkora relatív változást okoz. Vagyis a relatív előreható hiba megközelítőleg fele a relatív hátraható hibának, ahogy ezt az 1.5. példában is láttuk ugyanezen feladat kapcsán. Így a négyzetgyökvonás feladata kifejezetten jól kondicionált. Megjegyezzük, hogy az inverz feladat, a $g(y) = y^2$ kondíciószáma $|y\, g'(y)/g(y)| = |y(2y)/y^2| = 2$, ami a négyzetgyökvonás kondíciószámának reciproka, ahogy vártuk.

1.8. Példa. Érzékenység. Tekintsük a tangensfüggvényt, $f(x) = \tan(x)$. Mivel $f'(x) = \sec^2(x) = 1 + \tan^2(x)$,

$$\text{kondíciószám} \approx \left|\frac{xf'(x)}{f(x)}\right| = \left|\frac{x(1+\tan^2(x))}{\tan(x)}\right| = \left|x\left(\frac{1}{\tan(x)} + \tan(x)\right)\right|.$$

Így a $\tan(x)$ nagyon érzékeny a $\pi/2$ bármely páratlan többszörösének közelében, ahol a függvényérték a végtelenhez tart. Például az $x = 1{,}57079$ helyen a kondíciószám körülbelül $2{,}48275 \times 10^5$. Ennek hatását úgy szemléltethetjük, ha a függvényt két közeli pontban kiszámítjuk:

$$\tan(1{,}57079) \approx 1{,}58058 \times 10^5, \qquad \tan(1{,}57078) \approx 6{,}12490 \times 10^4.$$

Jól látható, hogy a kimenet relatív változása – körülbelül $1{,}58$ – nagyjából negyedmillió-szorosa a bemenet relatív változásának, ami körülbelül $6{,}37 \times 10^{-6}$. Másfelől a $g(y) = \arctan(y)$ inverz függvény kondíciószáma $|y\, g'(y)/g(y)| = |y\,(1/(1+y^2))/\arctan(y)|$. Az $y = 1{,}58058 \times 10^5$ értékre ez a kondíciószám körülbelül $4{,}0278 \times 10^{-6}$, ami a tangensfüggvény megfelelő x-re vett kondíciószámának reciproka. Így az $\arctan(y)$ rendkívül érzéketlen (jól kondicionált) ebben a pontban.

Az így definiált kondíciószámot néha *relatív* kondíciószámnak nevezik, mivel a bemenet és a kimenet relatív változásaira, illetve relatív hibáira nézve van definiálva. Általában ez a legmegfelelőbb megközelítés, de nincs értelmezve, ha a bemeneti x vagy a kimeneti y érték nulla. Ilyen esetekben az *abszolút* kondíciószám – a megoldás abszolút változásának és a bemenet abszolút változásának hányadosa, $|\Delta y|/|\Delta x|$ – az érzékenység megfelelő mértéke. Ilyen helyzet merül fel például a *gyökkeresés* során: egy $f : \mathbb{R} \to \mathbb{R}$ függvényhez olyan x^* értéket keresünk, amelyre $f(x^*) = 0$ (lásd az 5. fejezetet). Az $f(x)$ függvény egy ilyen x^* gyök közelében vett abszolút kondíciószáma:

$$\frac{|\Delta y|}{|\Delta x|} \approx |f'(x^*)|,$$

a gyök meghatározására vonatkozó inverz feladat – azaz egy olyan x bemeneti érték keresése, amely az $y = f(x) = 0$ kimenetet adja – abszolút kondíciószáma pedig $1/|f'(x^*)|$, feltéve, hogy $f'(x^*) \neq 0$.

1.2.7 Stabilitás és pontosság

Egy számítási algoritmus *stabilitásának* fogalma analóg egy matematikai feladat kondicionáltságával abban az értelemben, hogy mindkettő a perturbációk hatásairól szól. A különbség az, hogy a stabilitás a számítási hibának a számított eredményre gyakorolt hatására vonatkozik, míg a kondicionáltság az adathibának a feladat megoldására gyakorolt hatását írja le. Egy algoritmus *stabil*, ha a kiszámított eredmény viszonylag érzéketlen a számítás során alkalmazott közelítések miatti perturbációkra. A hátrahatható hibaelemzés szemszögéből egy algoritmus akkor stabil, ha az általa adott eredmény egy közeli feladat egzakt megoldása, azaz a számítás során fellépő perturbációk hatása nem rosszabb, mint amilyen hatással egy kis mennyiségű adathiba lenne az eredeti feladat bemenetében. Ezen definíció értelmében egy stabil algoritmus egzaktul a helyes eredményt adja egy közelítőleg helyes (kissé perturbált) feladatra. Sok – de nem

minden – hasznos algoritmus stabil ebben az erős értelemben. Bizonyos kontextusokban hasznos egy gyengébb stabilitási fogalom is: az, hogy az algoritmus közelítőleg helyes eredményt ad egy közelítőleg helyes feladatra.

A *pontosság* a számított megoldás és a vizsgált feladat valódi megoldása közötti közelséget jelenti. Egy algoritmus stabilitása önmagában nem garantálja, hogy a számított eredmény pontos is lesz: a pontosság egyszerre függ a feladat kondicionáltságától és az algoritmus stabilitásától. A stabilitás azt mondja meg, hogy a kapott megoldás egzakt egy közeli feladatra nézve, de ennek a közeli feladatnak a megoldása nem feltétlenül van közel az eredeti feladat megoldásához, hacsak a feladat nem jól kondicionált. Így pontatlanság eredhet akár egy stabil algoritmusnak egy rosszul kondicionált feladatra való alkalmazásából, akár egy instabil algoritmusnak egy jól kondicionált feladatra való alkalmazásából. Ezzel szemben, ha egy stabil algoritmust egy jól kondicionált feladatra alkalmazunk, akkor pontos megoldáshoz jutunk.

1.3 Számítógépes aritmetika

Ahogy már említettük, a tudományos számításokban az egyik elkerülhetetlen közelítés a valós számok számítógépes ábrázolásában rejlik. Ebben a szakaszban részletesebben megvizsgáljuk azokat a véges precíziójú aritmetikai rendszereket, amelyeket a digitális számítógépeken végzett tudományos számítások túlnyomó többségében használunk.

1.3.1 Lebegőpontos számok

Egy digitális számítógépen a matematikai $\mathbb{R}$ valós számrendszert egy *lebegőpontos számrendszerrel* ábrázoljuk. Az alapötlet a tudományos jelölésmódra (normálalakra) hasonlít, amelyben egy nagyon nagy vagy nagyon kicsi abszolút értékű számot egy mérsékelt méretű szám és a tíz megfelelő hatványának szorzataként fejezünk ki. Például a 2347 és a 0,0007396 rendre $2{,}347 \times 10^3$ és $7{,}396 \times 10^{-4}$ alakban írható. Ebben a formátumban a tizedespont mozog, vagyis „lebeg”, ahogy a 10 hatványa változik. Formálisan egy F lebegőpontos számrendszert négy egész szám jellemez:

$$\begin{aligned} \beta &\quad \text{a számrendszer alapja (radix)} \\ p &\quad \text{precízió (a mantissza hossza)} \\ [L, U] &\quad \text{exponens-tartomány} \end{aligned}$$

Bármely $x \in F$ lebegőpontos szám alakja

$$x = \pm \left(d_0 + \frac{d_1}{\beta} + \frac{d_2}{\beta^2} + \cdots + \frac{d_{p-1}}{\beta^{p-1}} \right) \beta^E,$$

ahol a d_i olyan egész szám, amelyre

$$0 \leq d_i \leq \beta - 1, \quad i = 0, \ldots, p-1,$$

E pedig olyan egész szám, amelyre

$$L \leq E \leq U.$$

A zárójelben lévő részt, amelyet a $d_0 d_1 \cdots d_{p-1}$ p darab β-alapú számjegyből álló sorozat ábrázol, *mantisszának* vagy *szignifikándnak* nevezzük, E-t pedig az x lebegőpontos szám *exponensének* vagy *karakterisztikájának*. A mantissza $d_1 d_2 \cdots d_{p-1}$ része a *törtrész*. Egy számítógépen az előjelet, az exponenst és a mantisszát egy adott lebegőpontos szó külön mezőiben tárolják, amelyek mindegyike rögzített szélességű. A nulla egyedi ábrázolást igényel, például úgy, hogy mind a mantisszája, mind az exponense nulla, vagy úgy, hogy a törtrésze nulla és az exponens egy speciális értéket vesz fel.

Manapság a legtöbb számítógép bináris ($\beta = 2$) aritmetikát használ, de más alapok is előfordultak a múltban: például hexadecimális ($\beta = 16$) az IBM mainframe-eken, illetve $\beta = 3$ egy balsorsú orosz számítógépen. Az oktális ($\beta = 8$) és a hexadecimális jelölést gyakran használjuk kényelmes rövidítésként a bináris számok hármas, illetve négyes bitcsoportokba rendezett leírására. Nyilvánvaló okokból a decimális ($\beta = 10$) aritmetika népszerű a zsebszámológépekben. Az emberi interakció megkönnyítése érdekében egy számítógép általában a bevitt számértékeket decimális jelölésről belső ábrázolásra, a kimeneten pedig vissza decimális jelölésre alakítja át, függetlenül attól, milyen alapot használ belsőleg. Néhány tipikus lebegőpontos rendszer paramétereit az 1.1. táblázat tartalmazza, amely jól illusztrálja a mezőszélességekből adódó kompromisszumot a precízió és az exponens-tartomány között. Például ugyanolyan 64 bites szóhosszúság mellett a Cray rendszer szélesebb exponens-tartománnyal rendelkezik, mint az IEEE dupla pontosságú ábrázolás, cserébe viszont kisebb pontossággal (precízióval) fizet.

Az IEEE szabvány szerinti egyszeres pontosságú (SP) és dupla pontosságú (DP) bináris lebegőpontos rendszerek ma messze a legfontosabbak. Szinte egyetemesen elterjedtek a személyi számítógépekben és munkaállomásokon, valamint számos mainframe- és szuperszámítógépen is. Az IEEE szabványt gondosan úgy tervezték, hogy kiküszöbölje a korábbi, gyártóspecifikus lebegőpontos implementációk számos anomáliáját és kétértelműségét, és ezzel nagyban elősegítette a hordozható és megbízható numerikus szoftverek fejlesztését. Emellett lehetővé teszi a kivételes helyzetek – például a nullával való osztás – értelmes és következetes kezelését.

Rendszer	β	p	L	U
IEEE SP	2	24	-126	127
IEEE DP	2	53	-1022	1023
Cray	2	48	$-16\,383$	$16\,384$
HP számológép	10	12	-499	499
IBM mainframe	16	6	-64	63

1.1. táblázat: Tipikus lebegőpontos rendszerek paraméterei.

1.3.2 Normalizáltság

Egy lebegőpontos rendszert *normalizáltnak* nevezünk, ha a vezető d_0 számjegy mindig nullától különböző, kivéve, ha az ábrázolt szám nulla. Így egy normalizált lebegő-

pontos rendszerben egy adott, nem nulla lebegőpontos szám m mantisszájára mindig fennáll, hogy

$$1 \leq m < \beta.$$

(Egy másik konvenció szerint d_0 mindig nulla, ebben az esetben egy lebegőpontos szám akkor normalizált, ha $d_1 \neq 0$, és ekkor $\beta^{-1} \leq m < 1$.) A lebegőpontos rendszereket általában normalizálják, mert:

- Így minden szám ábrázolása egyértelmű.
- Nem pazarolunk el számjegyeket vezető nullákra, és ezzel maximalizáljuk a precíziót.
- Egy bináris ($\beta = 2$) rendszerben a vezető bit mindig 1, így nem kell tárolni, és ezáltal egy adott mezőszélességen belül egy bittel nagyobb precíziót nyerünk.

1.3.3 A lebegőpontos rendszerek tulajdonságai

Egy lebegőpontos számrendszer véges és diszkrét. Egy adott lebegőpontos rendszerben a normalizált lebegőpontos számok száma

$$2(\beta - 1)\,\beta^{p-1}\,(U - L + 1) + 1,$$

mivel az előjelre két választási lehetőség van, a mantissza vezető számjegyére $\beta - 1$, a mantissza további $p - 1$ számjegyének mindegyikére β, az exponensre pedig $U - L + 1$ lehetséges érték. A $+1$-et azért adjuk hozzá, mert a szám lehet nulla is.

Létezik egy legkisebb pozitív normalizált lebegőpontos szám,

$$\text{UFL (alulcsordulási szint)} = \beta^L,$$

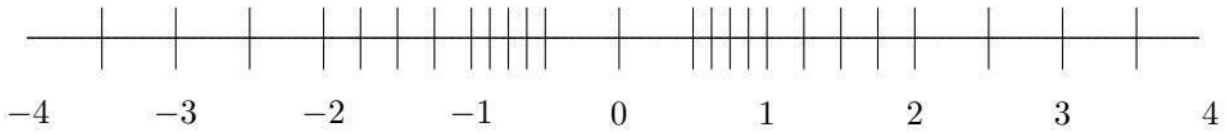
amelyben a vezető számjegy 1, a mantissza többi számjegye 0, az exponens pedig a legkisebb lehetséges érték. Létezik egy legnagyobb lebegőpontos szám,

$$\text{OFL (túlcsordulási szint)} = \beta^{U+1}\big(1 - \beta^{-p}\big),$$

amelyben a mantissza minden számjegye $\beta - 1$, az exponens pedig a legnagyobb lehetséges érték. Az adott lebegőpontos rendszerben nem ábrázolható az OFL-nél nagyobb szám, sem az UFL-nél kisebb pozitív szám. A lebegőpontos számok nem egyenletesen oszlanak el az értékkészletükben, hanem csak az egymást követő β-hatványok között egyenletes a távolságuk.

1.9. Példa. Lebegőpontos rendszer. Egy példa lebegőpontos rendszert szemléltet az 1.3. ábra, ahol a skálajelek a $\beta = 2$, $p = 3$, $L = -1$, $U = 1$ paraméterű rendszer mind a 25 lebegőpontos számát jelzik. Ebben a rendszerben a legnagyobb szám $\text{OFL} = (1{,}11)_2 \times 2^1 = (3{,}5)_{10}$, a legkisebb pozitív normalizált szám pedig $\text{UFL} = (1{,}00)_2 \times 2^{-1} = (0{,}5)_{10}$. Ez egy nagyon apró, szemléltető célú „játékrendszer”, mégis jellemző a lebegőpontos rendszerekre általában: kellően nagy nagyítás

mellett minden normalizált lebegőpontos rendszer lényegében így néz ki – szemcsés és egyenetlen eloszlású.



1.3. ábra: Példa lebegőpontos számrendszer.

1.3.4 Kerekítés

Azokat a valós számokat, amelyek egy adott lebegőpontos rendszerben pontosan ábrázolhatók, *gépi számoknak* nevezzük. Ha egy adott x valós szám nem ábrázolható pontosan lebegőpontos számként, akkor valamilyen „közeli” lebegőpontos számmal kell közelítenünk. Egy x valós szám lebegőpontos közelítését $\mathrm{fl}(x)$-szel jelöljük. Azt a folyamatot, amelynek során egy adott x valós számhoz egy közeli $\mathrm{fl}(x)$ lebegőpontos számot választunk, *kerekítésnek* nevezzük, az ilyen közelítés által bevezetett hibát pedig *kerekítési hibának*. Két gyakran használt kerekítési szabály:

- Csonkítás: az x szám β-alapú kifejtését a $(p-1)$-edik számjegy után elvágjuk. Mivel az $\mathrm{fl}(x)$ a következő lebegőpontos szám x-től a nulla felé haladva, ezt a szabályt néha *nulla felé kerekítésnek* is nevezik.
- Kerekítés a legközelebbihez: $\mathrm{fl}(x)$ az x-hez legközelebbi lebegőpontos szám; döntetlen esetén azt a lebegőpontos számot választjuk, amelynek az utolsó tárolt számjegye páros. Ez utóbbi tulajdonság miatt ezt a szabályt *páros felé kerekítésnek* is nevezik.

A legközelebbihez való kerekítés a legpontosabb és torzítatlan, de helyes megvalósítása valamivel költségesebb. Egyes régebbi rendszerek olcsóbban megvalósítható szabályokat használtak, például csonkítást, de az IEEE szabványos rendszerekben a legközelebbihez való kerekítés az alapértelmezett szabály.

1.10. Példa. Kerekítési szabályok. Az alábbi decimális számok két számjegyre kerekítése mindkét szabály szerint a következő eredményt adja:

Szám	Csonkítás	Legközelebbi	Szám	Csonkítás	Legközelebbi
1,649	1,6	1,6	1,749	1,7	1,7
1,650	1,6	1,6	1,750	1,7	1,8
1,651	1,6	1,7	1,751	1,7	1,8
1,699	1,6	1,7	1,799	1,7	1,8

Egy további, gyakran figyelmen kívül hagyott hibaforrás a lebegőpontos számok bevitelekor és kiírásakor szokásosan elvégzett decimális-bináris és bináris-decimális átváltásokban rejlik. Ezekre az átváltásokra az IEEE szabvány nem terjed ki; a szabvány csak a belső aritmetikai műveleteket rögzíti. A helyesen kerekített bevitel és kiírás ésszerű költséggel megvalósítható, de nem minden számítógépes rendszerben alkalmazzák. A helyesen kerekített bináris-decimális és decimális-bináris átváltásokra ha-

tékony, hordozható rutinok – a `dtoa` és az `strtod` – érhetők el a Netlibről (lásd az 1.4.1. szakaszt).

1.3.5 Gépi pontosság

Egy lebegőpontos rendszer pontosságát olyan mennyiség jellemzi, amelyet *egységnyi kerekítési hibának* (unit roundoff), *gépi pontosságnak* vagy *gépi epszilonnak* is nevezznek. Ennek értéke – amelyet ϵ_{mach}-kal jelölünk – az alkalmazott kerekítési szabálytól függ. Csonkítás esetén

$$\epsilon_{\text{mach}} = \beta^{1-p},$$

a legközelebbihez való kerekítés esetén pedig

$$\epsilon_{\text{mach}} = \tfrac{1}{2}\,\beta^{1-p}.$$

Az egységnyi kerekítési hiba azért fontos, mert felső korlátot ad arra a *relatív hibára*, amellyel egy lebegőpontos rendszer normalizált tartományában bármely nem nulla x valós szám ábrázolható:

$$\left|\frac{\text{fl}(x) - x}{x}\right| \leq \epsilon_{\text{mach}}.$$

Az egységnyi kerekítési hiba egy másik megfogalmazása, amellyel néha találkozhatunk, hogy ϵ_{mach} a legkisebb olyan ϵ szám, amelyre

$$\text{fl}(1+\epsilon) > 1,$$

ám ez nem teljesen ekvivalens az előbbi definícióval, ha a páros felé kerekítés szabályát alkalmazzuk. Egy további, esetenként használt definíció szerint ϵ_{mach} az 1 és az azt követő lebegőpontos szám közötti távolság, ám ez is eltérhet az előző két definíciótól. Részleteikben különbözhetnek, de az ϵ_{mach} mindhárom definíciójának ugyanaz az alapvető célja: a lebegőpontos rendszer relatív szemcsézettségének mérése.

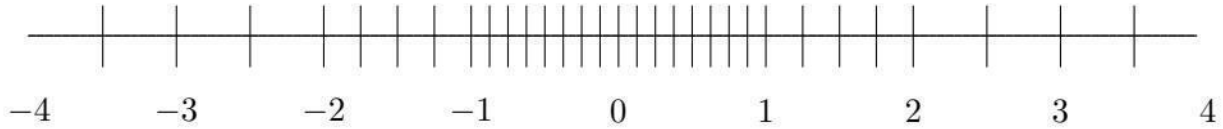
Az 1.9. példa szemléltető játékrendszerében csonkítás esetén $\epsilon_{\text{mach}} = 0{,}25$, a legközelebbihez való kerekítés esetén pedig $\epsilon_{\text{mach}} = 0{,}125$. Az IEEE bináris lebegőpontos rendszerekben $\epsilon_{\text{mach}} = 2^{-24} \approx 10^{-7}$ egyszeres pontosság mellett, és $\epsilon_{\text{mach}} = 2^{-53} \approx 10^{-16}$ dupla pontosság mellett. Ezért azt mondjuk, hogy az IEEE egyszeres, illetve dupla pontosságú lebegőpontos rendszerek körülbelül 7, illetve 16 decimális értékes jegy pontosságúak.

Bár mindkettő „kicsi”, az egységnyi kerekítési hibát nem szabad összetéveszteni az alulcsordulási szinttel. Az ϵ_{mach} egységnyi kerekítési hibát a lebegőpontos rendszer mantisszamezejének hossza (számjegyeinek száma) határozza meg, míg az UFL alulcsordulási szintet az exponensmező hossza. Minden gyakorlatban használt lebegőpontos rendszerben fennáll, hogy

$$0 < \text{UFL} < \epsilon_{\text{mach}} < \text{OFL}.$$

1.3.6 Szubnormális számok és fokozatos alulcsordulás

Az 1.3. ábrán bemutatott szemléltető lebegőpontos rendszerben észrevehető rés van a nulla körül. Ez a rés – amely valamilyen mértékben minden lebegőpontos rendszerben jelen van – a normalizáltság következménye: a legkisebb lehetséges mantissza $1{,}00\ldots$, a legkisebb lehetséges exponens pedig L, így nincs lebegőpontos szám a nulla és β^L között. Ha feladjuk a normalizáltsági követelményt, és megengedjük, hogy a vezető számjegy nulla legyen (de csak akkor, amikor az exponens a minimális értékén áll), akkor a nulla körüli rést további lebegőpontos számokkal „tölthetjük ki”. A szemléltető játékrendszerünkben ez az enyhítés hat további lebegőpontos számot eredményez, amelyek közül a legkisebb pozitív a $(0{,}01)_2 \times 2^{-1} = (0{,}125)_{10}$, ahogy azt az 1.4. ábra mutatja.



1.4. ábra: Példa lebegőpontos rendszer szubnormális számokkal.

Az így a rendszerhez hozzáadott további számokat *szubnormális* vagy *denormalizált* lebegőpontos számoknak nevezzük. Noha hasznosan kiterjesztik az ábrázolható abszolút értékek tartományát, a szubnormális számoknak eredendően kisebb a pontosságuk, mint a normalizált számokénak, mert a törtrészükben kevesebb értékes jegy szerepel. Különösen fontos megjegyezni, hogy a tartomány ilyen módon történő kiterjesztése nem csökkenti az ϵ_{mach} egységnyi kerekítési hibát.

Az ilyen módon kiterjesztett lebegőpontos rendszerről azt mondjuk, hogy *fokozatos alulcsordulást* (gradual underflow) valósít meg, mivel az ábrázolható abszolút értékek alsó tartományát kiterjeszti, ahelyett, hogy a minimális exponensérték alá csökkenéskor hirtelen nullára csordulna (flush to zero). Az IEEE szabvány előírja a szubnormális számok használatát és a fokozatos alulcsordulást. A fokozatos alulcsordulást az exponensmező egy speciális, lefoglalt értékén keresztül valósítják meg, mivel a vezető bináris számjegyet fizikailag nem tárolják, így azt nem lehet a denormalizáltság jelzésére felhasználni.

1.3.7 Kivételes értékek

Az IEEE lebegőpontos szabvány két további speciális értéket biztosít a kivételes helyzetek jelzésére:

- `Inf`, amely a végtelent jelöli, és egy véges szám nullával való osztásakor áll elő, például $1/0$.
- `NaN`, amely a *nem szám* (not a number) rövidítése, és definiálatlan vagy határozatlan művelet eredménye, például $0/0$, $0 \cdot \text{Inf}$ vagy Inf/Inf.

Az `Inf` és a `NaN` az IEEE aritmetikában az exponensmező speciális, fenntartott értékeinek segítségével valósul meg.

Az, hogy az `Inf` és a `NaN` támogatott-e felhasználói szinten egy adott számítási környezetben, a nyelvtől, a fordítótól és a futásidejű rendszertől függ. Ha elérhetők, akkor ezek a mennyiségek segíthetnek olyan szoftvert tervezni, amely a kivételes helyzeteket elegánsan kezeli, és nem szakítja meg hirtelen a programot. A MATLAB-ban (lásd az 1.4.2. szakaszt) például, ha `Inf` vagy `NaN` lép fel, a rendszer azokat értelemszerűen továbbviszi a számítás során (pl. $1 + \text{Inf} = \text{Inf}$). Ennek ellenére érdemes, ha lehetséges, teljesen elkerülni az ilyen kivételes helyzeteket. Amellett, hogy felhívják a felhasználó figyelmét az aritmetikai kivételekre, ezek a speciális értékek jelzőként (flag) is hasznosak lehetnek, mivel egyetlen érvényes numerikus értékkel sem téveszthetők össze. Például a `NaN` használható egy tömb még definiálatlan részének megjelölésére.

1.3.8 Lebegőpontos aritmetika

Két lebegőpontos szám összeadásánál vagy kivonásánál a mantisszák összeadása vagy kivonása előtt az exponenseknek meg kell egyezniük. Ha kezdetben nem egyeznek meg, akkor az egyik szám mantisszáját addig kell eltolni, amíg az exponensek azonosak nem lesznek. Egy ilyen eltolás során az (abszolút értékben) kisebb szám utolsó számjegyei a mantisszamező rögzített szélességén túlra kerülhetnek, így az aritmetikai művelet helyes eredménye nem ábrázolható pontosan a lebegőpontos rendszerben. Sőt, ha az abszolút értékek közötti különbség túl nagy, akkor a kisebb szám teljes mantisszája kicsúszhat a mezőszélességből, és így az eredmény egyszerűen a két operandus közül a nagyobbik lesz. Másképpen fogalmazva: ha két p számjegyű szám valódi összege p-nél több számjegyet tartalmaz, akkor a p számjegyre kerekítéskor a felesleges számjegyek elvesznek, és a legrosszabb esetben a kisebb abszolút értékű operandus teljesen eltűnhet.

Két lebegőpontos szám szorzásához nem szükséges, hogy exponenseik megegyezzenek – az exponenseket egyszerűen összeadjuk, a mantisszákat pedig összeszorozzuk. Két p számjegyű mantissza szorzata azonban általában akár $2p$ számjegyet is tartalmazhat, így a helyes eredmény ismételten nem ábrázolható pontosan a lebegőpontos rendszerben, ezért kerekíteni kell.

1.11. Példa. Lebegőpontos aritmetika. Tekintsünk egy $\beta = 10$ és $p = 6$ paraméterű lebegőpontos rendszert. Ha $x = 1{,}92403 \times 10^2$ és $y = 6{,}35782 \times 10^{-1}$, akkor a lebegőpontos összeadás – a legközelebbihez való kerekítés mellett – az $x + y = 1{,}93039 \times 10^2$ eredményt adja. Vegyük észre, hogy y utolsó két számjegye nem befolyásolja az eredményt. Még kisebb exponens esetén y egyáltalán nem is hatott volna az eredményre. Hasonlóképpen, a lebegőpontos szorzás az $x \cdot y = 1{,}22326 \times 10^2$ eredményt adja, ami a valódi szorzat számjegyeinek a felét eldobja.

Két lebegőpontos szám osztása is adhat olyan eredményt, amely nem ábrázolható pontosan. Például az 1 és a 10 pontosan ábrázolható bináris lebegőpontos számként, de hányadosuk, az $1/10$ nem véges bináris kifejtésű, így nem ábrázolható pontosan bináris lebegőpontos számként.

Minden fent említett esetben egy lebegőpontos aritmetikai művelet eredménye eltérhet attól, amit a megfelelő valós aritmetikai művelet adna ugyanazokra az operandusokra, egyszerűen azért, mert a helyes valós eredmény ábrázolásához nincs elegendő pontosság. A valós eredmény azért is lehet nem ábrázolható, mert az exponense a lebegőpontos rendszer rendelkezésre álló tartományán kívül esik (túlcsordulás vagy

alulcsordulás). A túlcsordulás általában súlyosabb probléma, mint az alulcsordulás, abban az értelemben, hogy egy lebegőpontos rendszerben nincs jó közelítés a tetszőlegesen nagy számokra, míg a nulla gyakran ésszerű közelítés a tetszőlegesen kis számokra. Ezért sok számítógépes rendszerben a túlcsordulás bekövetkezése végzetes hibával megszakítja a programot, az alulcsordulást viszont a rendszer csendben nullára állíthatja a végrehajtás megzavarása nélkül.

1.12. Példa. Sor összegzése. E kérdések szemléltetéseként a

$$\sum_{n=1}^{\infty} \frac{1}{n}$$

végtelen sornak véges az összege lebegőpontos aritmetikában, noha a valós sor divergens. Első ránézésre azt gondolhatnánk, hogy ez azért van, mert az $1/n$ előbb-utóbb alulcsordul, vagy a részletösszeg előbb-utóbb túlcsordul – ami végül valóban meg is történik. De mielőtt ezek bármelyike bekövetkezne, a részletösszeg már megszűnik változni, amint az $1/n$ a részletösszeghez képest elhanyagolhatóvá válik, azaz amikor $1/n < \epsilon_{\text{mach}} \sum_{k=1}^{n-1}(1/k)$, így az összeg véges marad (lásd az 1.8. számítógépes feladatot).

Amint megjegyeztük, két lebegőpontos számon elvégzett valós aritmetikai művelet eredménye nem feltétlenül egy újabb, pontos lebegőpontos szám. Ha a számítógépbe olyan szám kerül, amely lebegőpontos számként nem ábrázolható pontosan, vagy egy későbbi aritmetikai művelet ilyet állít elő, akkor azt a korábban megadott kerekítési szabályok valamelyikével kerekíteni kell, hogy lebegőpontos számot kapjunk. Mivel a lebegőpontos számok nem egyenletes eloszlásúak, az ilyen közelítésben fellépő abszolút hiba nem egyenletes, a relatív hibát azonban az ϵ_{mach} egységnyi kerekítési hiba korlátozza.

Ideális esetben $x \operatorname{flop} y = \text{fl}(x \operatorname{op} y)$ (azaz a lebegőpontos aritmetikai műveletek helyesen kerekített eredményt adnak), és sok számítógép – például az IEEE lebegőpontos szabványnak megfelelőek – el is éri ezt az ideált, amíg az $x \operatorname{op} y$ a lebegőpontos rendszer tartományán belül marad. Ennek ellenére a valós aritmetika néhány alapvető törvénye nem feltétlenül érvényes a lebegőpontos rendszerekben. Érdemes kiemelni, hogy a lebegőpontos összeadás és szorzás kommutatív, de nem asszociatív. Például ha ϵ az ϵ_{mach} egységnyi kerekítési hibánál egy picit kisebb pozitív lebegőpontos szám, akkor $(1+\epsilon)+\epsilon = 1$, de $1+(\epsilon+\epsilon) > 1$. Az, hogy a lebegőpontos aritmetika nem teljesíti a valós aritmetika szokásos törvényeit, az egyik fő oka annak, hogy az előreható hibaelemzés nehéz feladat. A hátraható hibaelemzés egyik nagy előnye éppen az, hogy az elemzés során lehetővé teszi a valós aritmetika szabályainak használatát.

A kerekítési hibaelemzés rendszerint a következő szabványos modellre épül a lebegőpontos aritmetikára vonatkozóan:

$$\text{fl}(x \operatorname{op} y) = (x \operatorname{op} y)(1+\delta),$$

ahol $|\delta| \leq \epsilon_{\text{mach}}$, és az op a $+$, $-$, $\times$, $/$ szabványos aritmetikai műveletek valamelyike. Átrendezve láthatjuk, hogy ez a modell értelmezhető a relatív előreható hibára

vonatkozó állításként is:

$$\frac{|\mathrm{fl}(x \text{ op } y) - (x \text{ op } y)|}{|(x \text{ op } y)|} = |\delta| \leq \epsilon_{\text{mach}}.$$

De ugyanígy értelmezhető a hátraható hibára vonatkozó állításként is. Például op $= +$ esetén

$$\mathrm{fl}(x+y) = (x+y)(1+\delta) = x(1+\delta) + y(1+\delta),$$

ami azt jelenti, hogy a lebegőpontos összeadás eredménye egzakt olyan x és y operandusokra nézve, amelyek mindegyike $|\delta|$ relatív mértékben perturbált.

1.13. Példa. Kerekítési hibaelemzés. Tekintsük az $x(y+z)$ egyszerű számítást. Lebegőpontos aritmetikában

$$\mathrm{fl}(y+z) = (y+z)(1+\delta_1), \quad |\delta_1| \leq \epsilon_{\text{mach}},$$

és így

$$\begin{aligned}
\mathrm{fl}\big(x(y+z)\big) &= \big(x\big((y+z)(1+\delta_1)\big)\big)(1+\delta_2), \quad |\delta_2| \leq \epsilon_{\text{mach}} \\
&= x(y+z)(1+\delta_1+\delta_2+\delta_1\delta_2) \\
&\approx x(y+z)(1+\delta_1+\delta_2) \\
&= x(y+z)(1+\delta), \quad |\delta| = |\delta_1+\delta_2| \leq 2\epsilon_{\text{mach}}.
\end{aligned}$$

Ahogy korábban láttuk, ez a korlát egyaránt értelmezhető az előreható hiba (a számított és a kívánt eredmény közötti eltérés) vagy a hátraható hiba (operandusperturbációk) szempontjából is, de meglehetősen pesszimista lehet. Például δ_1 és δ_2 lehet ellentétes előjelű, és így ki is olthatják egymást, aminek eredményeként a teljes hiba sokkal kisebb lehet, mint amit a legrosszabb esetre vonatkozó elemzés sugall. Bonyolultabb számítások hasonló elemzése általában az ϵ_{mach} ennek megfelelő, nagyobb többszöröseit tartalmazó hibakorlátokhoz vezet. Szerencsére az ϵ_{mach} olyannyira pici, hogy a gyakorlatban az eredmény rendkívül ritkán romlik le súlyosan pusztán a számítási lépések nagy száma miatt.

1.3.9 Számjegykioltás

A kerekítés nem az egyetlen szükséges rossz a véges precíziójú aritmetikában. Két azonos előjelű és hasonló abszolút értékű p számjegyű szám kivonása p-nél kevesebb értékes számjegyet tartalmazó eredményt ad, ezért az mindig pontosan ábrázolható (feltéve, hogy a két szám abszolút értéke nem több mint kétszeresen tér el egymástól). Ennek oka, hogy a két szám vezető számjegyei kioltják egymást (azaz különbségük nulla). Például ismét a $\beta = 10$ és $p = 6$ értékeket véve, ha $x = 1{,}92403 \times 10^2$ és $z = 1{,}92275 \times 10^2$, akkor $x - z = 1{,}28000 \times 10^{-1}$ adódik, amely mindössze három értékes jegyével pontosan ábrázolható.

Bár az eredmény pontos, az ilyen számjegykioltás gyakran mégis jelentős információvesztést jelent. A probléma az, hogy az operandusok a kerekítés vagy más korábbi hibák miatt gyakran eleve bizonytalanok, ezért a különbségben megjelenő relatív bizonytalanság nagyon nagy lehet. Más szóval: ha két közel azonos szám csak a kerekítési hiba pontosságáig helyes, akkor a különbségük már kizárólag csak a kerekítési hibát fogja tartalmazni.

Egyszerű példaként: ha ϵ az ϵ_{mach} egységnyi kerekítési hibánál kicsit kisebb pozitív szám, akkor $(1+\epsilon)-(1-\epsilon)=1-1=0$ lebegőpontos aritmetikában, ami az utolsó kivonás tényleges operandusaira nézve helyes, de a teljes számítás valódi eredménye, a 2ϵ, teljesen elveszik. Maga a kivonás nem hibás: csupán felszínre hozza a már korábban bekövetkezett információvesztést.

Természetesen az információvesztés nem mindig teljes, de tény marad, hogy a számjegykioltás során elveszett számjegyek a legfontosabb, vezető számjegyek, míg a kerekítés során elveszett számjegyek a legkevésbé fontos, utolsó számjegyek. Emiatt a hatás miatt egy kis mennyiség kiszámítása két nagy mennyiség különbségeként általában rossz ötlet, mert a kerekítési hiba valószínűleg dominálni fogja a végeredményt. Például egy váltakozó előjelű sor összegzése, mint az

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$$

$x < 0$ esetén, a katasztrofális számjegykioltás miatt végzetesen rossz eredményt adhat (lásd az 1.9. számítógépes feladatot).

1.14. Példa. Számjegykioltás. A számjegykioltás nemcsak a számítógépes aritmetikában okoz problémát, hanem bármely olyan helyzetben, ahol csak korlátozott pontosság érhető el – például empirikus mérések vagy laboratóriumi kísérletek során. Például Manhattan és Staten Island közötti távolság meghatározása a két hely Los Angelestől mért távolságainak különbségeként nagyon rossz eredményt adna, hacsak ez utóbbi távolságokat nem ismerjük rendkívül nagy pontossággal.

Másik példaként a fizikusok évek óta próbálják a héliumatom teljes energiáját első elvekből kiszámítani Monte-Carlo-módszerekkel. E számítások pontosságát nagyrészt a felhasznált véletlen minták száma határozza meg. Ahogy a gyorsabb számítógépek elérhetővé válnak, és a számítási technikák finomodnak, úgy javul az elérhető pontosság is. A teljes energia a mozgási és a potenciális energia összege, amelyeket külön-külön számítanak ki, és ellentétes előjelűek. Így a teljes energia valójában különbségként áll elő, és számjegykioltástól szenved. Az 1.2. táblázat az évek során kapott értékek sorozatát mutatja be (ezeket az adatokat Dr. Robert Panoff bocsátotta rendelkezésre). Ebben az időszakban a mozgási és a potenciális energiára számított értékek legfeljebb 6 százalékkal változtak, a teljes energiára adott becslés azonban 144 százalékkal. A korábbi számítások egy-két értékes jegye a későbbi kivonás során teljesen elveszett.

Év	Mozgási	Potenciális	Teljes
1971	13,0	−14,0	−1,0
1977	12,76	−14,02	−1,26
1980	12,22	−14,35	−2,13

Év	Mozgási	Potenciális	Teljes
1985	12,28	$-14{,}65$	$-2{,}37$
1988	12,40	$-14{,}84$	$-2{,}44$

1.2. táblázat: A héliumatom teljes energiájára számított értékek.

1.15. Példa. Másodfokú megoldóképlet. A számjegykioltás és egyéb numerikus buktatók nem feltétlenül igényelnek hosszú számítássorozatot. Például a másodfokú egyenlet gyökeire szolgáló szabványos képlet használata tele van numerikus csapdákkal. Ahogy minden iskolás megtanulja, az

$$ax^2 + bx + c = 0$$

másodfokú egyenlet két megoldását a megoldóképlet adja:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

Az együtthatók bizonyos értékei esetén e képlet naiv alkalmazása a lebegőpontos aritmetikában túlcsordulást, alulcsordulást vagy katasztrofális számjegykioltást okozhat.

Ha például az együtthatók nagyon nagyok vagy nagyon kicsik, akkor a b^2 vagy a $4ac$ túlcsordulhat vagy alulcsordulhat. A túlcsordulás lehetősége az együtthatók átskálázásával elkerülhető, például úgy, hogy mindhárom együtthatót elosztjuk a legnagyobb abszolút értékűvel. Ez az átskálázás nem változtatja meg a másodfokú egyenlet gyökeit, ám a legnagyobb együttható mostantól 1, így a b^2 vagy a $4ac$ számításakor nem léphet fel túlcsordulás. A művelet ugyan nem szünteti meg teljesen az alulcsordulás lehetőségét, de megelőzi a felesleges alulcsordulást, amely egyébként bekövetkezhetne, ha mindhárom együttható nagyon kicsi lenne.

A $-b$ és a négyzetgyök közötti számjegykioltás elkerülhető, ha az egyik gyököt az

$$x = \frac{2c}{-b \mp \sqrt{b^2 - 4ac}}$$

alternatív képlet szerint számoljuk, amelyben az előjelek a szabványos képlet ellentettjei. A négyzetgyök alatti kioltás azonban nehezen kerülhető el nagyobb pontosság használata nélkül (ha a diszkrimináns az együtthatókhoz képest kicsi, akkor a két gyök nagyon közel van egymáshoz, és a feladat eredendően rosszul kondicionált).

Szemléltetésképpen négy számjegyű decimális aritmetikát – a legközelebbihez való kerekítéssel – használunk az $a = 0{,}05010$, $b = -98{,}78$, $c = 5{,}015$ együtthatójú másodfokú egyenlet gyökeinek számítására. Összehasonlításul a helyes gyökök, tíz értékes jegyre kerekítve:

$$1971{,}605916 \quad \text{és} \quad 0{,}05077069387.$$

A diszkrimináns négy számjegyű aritmetikában

$$b^2 - 4ac = 9757 - 1{,}005 = 9756,$$

így

$$\sqrt{b^2 - 4ac} = 98{,}77.$$

A szabványos megoldóképlet ekkor a

$$\frac{98{,}78 \pm 98{,}77}{0{,}1002} = 1972 \quad \text{és} \quad 0{,}09980$$

gyököket adja. Az első gyök a helyesen kerekített négy számjegyű eredmény, a másik viszont teljesen hibás, körülbelül 100 százalékos hibával. A probléma forrása a számjegykioltás – nem abban az értelemben, hogy a végső kivonás helytelen volna (sőt, az teljesen pontos), hanem abban, hogy a vezető számjegyek kioltása csak a korábbi kerekítési hibákat hagyta meg. Az alternatív megoldóképlet a

$$\frac{10{,}03}{98{,}78 \mp 98{,}77} = 1003 \quad \text{és} \quad 0{,}05077$$

gyököket adja. Ismét egy teljesen pontos és egy teljesen hibás gyököt kaptunk, de most éppen a másik gyök lett pontatlan. A magyarázat ismét a számjegykioltás, de az eltérő előjelminta miatt az ellenkező gyök szennyeződik. Általában az egyes gyökök számításához azt a képletet kell választanunk, amelyik – b előjelétől függően – elkerüli a kioltást.

1.16. Példa. Szórás. Egy véges x_i ($i = 1, \dots, n$) valós sorozat átlagának definíciója

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i,$$

a szórásé pedig

$$\sigma = \left[\frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2 \right]^{1/2}.$$

Ezen képletek használata azt jelenti, hogy kétszer kell végigmenni az adatokon: egyszer az átlag, másszor pedig a szórás kiszámításához. A hatékonyság érdekében csábító lehet a matematikailag ekvivalens

$$\sigma = \left[\frac{1}{n-1} \left(\sum_{i=1}^{n} x_i^2 - n\bar{x}^2 \right) \right]^{1/2}$$

képletet használni a szórás számítására, hiszen így az összeg és a négyzetösszeg egyetlen lépésben (menetben) számítható.

Sajnos az egylépéses képlet végén fellépő egyetlen számjegykioltás numerikusan gyakran sokkal rombolóbb, mint a kétlépéses képletben fellépő összes kerekítési hiba együttvéve. A probléma az, hogy az egylépéses képletben kivonandó két mennyiség általában viszonylag nagy és közel azonos, így a különbségükben fellépő relatív hiba hatalmas lehet (sőt, a kerekítések miatt az eredmény akár negatív is lehet, és ilyenkor a négyzetgyökvonás is meghiúsul).

1.17. Példa. Maradékok számítása. Tegyük fel, hogy az $ax = b$ skaláris lineáris egyenletet oldjuk meg az x ismeretlenre, és kaptunk egy $\hat{x}$ közelítő megoldást. A válaszunk minőségének egyik mérőszámaként kiszámítjuk az $r = b - a\hat{x}$ maradékot. Lebegőpontos aritmetikában

$$\mathrm{fl}(a\hat{x}) = a\hat{x}(1+\delta_1), \quad |\delta_1| \le \epsilon_{\text{mach}},$$

így

$$\begin{aligned}
\mathrm{fl}(b - a\hat{x}) &= (b - a\hat{x}(1+\delta_1))(1+\delta_2), \quad |\delta_2| \le \epsilon_{\text{mach}} \\
&= (r - a\hat{x}\delta_1)(1+\delta_2) \\
&= r(1+\delta_2) - a\hat{x}\delta_1 - a\hat{x}\delta_1\delta_2 \\
&\approx r(1+\delta_2) - \delta_1 b.
\end{aligned}$$

Ám a $\delta_1 b$ tag akár $\epsilon_{\text{mach}} b$ nagyságrendű is lehet, ami viszont már megegyezhet r nagyságrendjével – ez azt jelenti, hogy a maradék kiszámításában fellépő hiba elérheti a 100 százalékot, vagy akár annál is nagyobb lehet. Ezért az r maradék értelmes (pontos) kiszámításához sokszor kétszeres pontosság szükséges.

Szemléltetésként tegyük fel, hogy három számjegyű decimális aritmetikát használunk, és legyen $a = 2{,}78$, $b = 3{,}14$, $\hat{x} = 1{,}13$. Három számjegyű aritmetikában $2{,}78 \times 1{,}13 = 3{,}14$, így a három számjegyű maradék 0. Ez az eredmény egyfelől megnyugtató, mert azt mutatja, hogy a felhasznált precízió mellett a lehető legjobbat hoztuk ki a feladatból, másfelől viszont nem tartalmaz egyetlen helyes értékes jegyet sem a valódi maradékból, amely amúgy $3{,}14 - 2{,}78 \times 1{,}13 = 3{,}14 - 3{,}1414 = -0{,}0014$. Ami történt: b és $a\hat{x}$ jellegükből fakadóan csaknem egyenlők, így a különbségük már csak a kerekítési hibát tartalmazza. A feladat korábbi elemzése szerint

$$3{,}14 = \mathrm{fl}(a\hat{x}) = a\hat{x}(1+\delta_1) = 3{,}1414(1+\delta_1),$$

amelyből $\delta_1 = -0{,}0014/3{,}1414 \approx -0{,}00044566$. A későbbi kivonásban nincs kerekítési hiba (mint a kioltás esetén általában), így $\delta_2 = 0$. A kapott számított maradék

$$\mathrm{fl}(b - a\hat{x}) \approx r(1+\delta_2) - \delta_1 b = r - \delta_1 b \approx -0{,}0014 - (-0{,}0014) = 0.$$

Általánosságban a maradék pontos kiszámításához a munka során használt precíziónak akár a kétszeresére is szükség lehet.

1.3.10 Más aritmetikai rendszerek

Manapság a tudományos számítások túlnyomó többségében az IEEE szabvány szerinti lebegőpontos aritmetikát használjuk. Mivel szinte minden modern mikroprocesszor hardveresen támogatja, az IEEE lebegőpontos aritmetika rendkívül gyors, a dupla pontosságú ábrázolás pedig a legtöbb gyakorlati célra több mint elegendő precíziót biztosít. Időnként előfordulhatnak azonban olyan esetek, amikor ennél nagyobb pontosságra van szükség – például egy elkerülhetetlenül nagyon érzékeny (rosszul kondicionált) feladat megoldásakor, vagy esetleg a számítási műveletek puszta mennyisége miatt (bár ehhez rendszerint valóban hatalmas számítási volumen szükséges). Ilyenkor a rendelkezésre álló pontosság kiterjesztésére több alternatíva is adódik, ám sietve hozzá kell tennünk, hogy a kiterjesztett pontosság használata nem helyettesítheti a feladatok gondos megfogalmazását és a megbízható megoldási technikákat.

Néhány számítógépes rendszer hardveresen is biztosít négyszeres pontosságot, és ez valószínűleg egyre elterjedtebbé válik, ahogy a processzorok gyorsulnak, a memóriák pedig egyre nagyobbak lesznek. Számos olyan szoftvercsomag is elérhető, amely többszörös pontosságú lebegőpontos aritmetikát kínál (lásd az 1.4.3. szakaszt). Egyes szimbolikus számítási környezetek – például a Maple és a Mathematica (lásd az 1.4.2. szakaszt) – tetszőleges pontosságú aritmetikát biztosítanak abban az értelemben, hogy a pontosság szükség szerint növelhető, így a számítás során végig megőrizhetők az egzakt eredmények. Az ilyen szoftveres megközelítéseknek azonban jelentős hátrányaik vannak a sebesség és a memóriaigény terén: a szoftveresen végzett aritmetika nagyságrendekkel lassabb lehet, mint a megfelelő hardveres lebegőpontos műveletek, és nagy mennyiségű memóriára lehet szükség a kapott hosszú számjegysorozatok tárolásához. Még ha a pontosság abban az értelemben tetszőleges is, hogy dinamikusan változhat, végső soron az adott számítógépen rendelkezésre álló véges memória mindig határt szab neki.

Bármilyen pontosságot is használunk, a numerikus analízis egyik fő kérdése, hogy egy adott számításban valójában milyen pontosságot érünk el. Ahogy ebben a fejezetben láttuk – és a könyv hátralévő részében is látni fogjuk –, a hagyományos hibaelemzés a megoldandó feladat, a használt algoritmus, valamint azon aritmetika tulajdonságainak matematikai vizsgálatán nyugszik, amelyen az algoritmust végrehajtjuk. Az *intervallumanalízis* egy olyan alternatív megközelítés, amely a bizonytalanságok továbbterjedését magába az aritmetikai rendszerbe építi be, és ezzel legalábbis elvben automatikusan meghatározza a pontosságot. Az alapötlet az, hogy minden numerikus mennyiséget egy zárt intervallumként – mondjuk $[a, b]$ – ábrázolunk, amelyet az adott mennyiség összes lehetséges értéke halmazának tekintünk. Az intervallum hossza ekkor a mennyiség bizonytalanságát, azaz a lehetséges hibáját képviseli (ha nincs bizonytalanság, akkor $a = b$).

Két intervallumon végzett aritmetikai művelet eredménye a megfelelő valós aritmetikai művelet összes lehetséges eredményének halmaza, ha az operandusokat a két

intervallumból vesszük:

$$[a,b] \text{ op } [c,d] = \{x \text{ op } y : x \in [a,b] \text{ és } y \in [c,d]\},$$

ahol op a $+$, $-$, $\times$, $/$ műveletek valamelyike. Az eredményül kapott halmaz maga is intervallum, amelynek végpontjai a következő szabályok szerint számíthatók ki:

$$\begin{aligned}
[a,b] + [c,d] &= [a+c,\ b+d],\\
[a,b] - [c,d] &= [a-d,\ b-c],\\
[a,b] \times [c,d] &= \big[\min(ac,ad,bc,bd),\ \max(ac,ad,bc,bd)\big],\\
[a,b]/[c,d] &= [a,b] \times [1/d,\ 1/c], \quad \text{feltéve, hogy } 0 \notin [c,d].
\end{aligned}$$

Lebegőpontos aritmetikában való megvalósításkor az intervallum végpontjainak kiszámítását megfelelően kell kerekíteni annak érdekében, hogy a számított intervallum biztosan tartalmazza a megfelelő egzakt intervallumot (a bal végpontot $-\infty$ felé, a jobb végpontot $+\infty$ felé kerekítjük; ez az úgynevezett *irányított kerekítés* az IEEE aritmetikában opcióként elérhető).

A bizonytalanságok (köztük a kezdeti bemenet bizonytalanságának) továbbterjesztése a számítási folyamaton keresztül egyszerű módszernek tűnhet az automatikus hibaelemzéshez, a gyakorlatban azonban a kapott intervallumok gyakran túl szélesek ahhoz, hogy hasznosak legyenek – nagyjából ugyanabból az okból kifolyólag, amiért az előreható hibaelemzés is gyakran túlzottan pesszimista korlátokat ad. Ha a számítást rögzített precízióval dolgozó, egymás után alkalmazott függvények kompozíciójának tekintjük, akkor a végső intervallum szélessége az összes érintett függvény kondíciószámának szorzatával arányos. Ha sok ilyen függvény van, vagy ha rosszul kondicionáltak, akkor a végső intervallum valószínűleg túl széles lesz ahhoz, hogy érdemi információt nyújtson. További probléma, hogy az intervallumanalízis hajlamos figyelmen kívül hagyni a számítási változók közötti függőségeket – például, ha ugyanaz a változó többször is megjelenik egy számításban –, és ezeket független bizonytalansági forrásokként kezeli, ami ismét feleslegesen széles intervallumokhoz vezethet. Egy triviális példa: $[a,b] - [a,b] = [a-b,\ b-a]$ adódik a várt $[0,0]$ helyett.

Az intervallumaritmetika által kapott intervallumszélességek kordában tarthatók, ha a rendszert változó precízióval valósítjuk meg. Az úgynevezett *tartomány-aritmetikában* (range arithmetic) például a szokásos lebegőpontos aritmetikát két módon is kibővítjük: egyrészt a mantissza tetszőleges hosszúságú lehet egy előre meghatározott precíziókorlátig, másrészt egy további r *tartományjegyet* tartunk fenn, amely a mantissza bizonytalanságát ($\pm r$) jelöli. A tartományszámokon végzett aritmetikai műveletek a korábban megadott szokásos intervallumaritmetikai szabályokat követik. A számítások előrehaladtával, ha egy adott érték bizonytalansága nő, akkor a mantisszájának hosszát ennek megfelelően csökkentik. Ha a végső eredmény mantisszája túl kevés számjegyet tartalmaz a pontossági követelmény teljesítéséhez, akkor a precíziókorlátot megemelik, és az egész számítást annyiszor ismétlik meg, ahányszor csak szükséges. Ily módon sok (de nem minden) számítási feladat megoldható tetszőlegesen előírt pontossággal, ám gyakran megjósolhatatlan (és potenciálisan hatalmas) futásidő- és memóriaigény árán.

Több olyan szoftvercsomag is elérhető, amely intervallum- vagy tartomány-aritmetikát valósít meg (lásd az 1.4.3. szakaszt). Az említett hátrányok korlátozták e módszerek széles körű elterjedését az általános tudományos számításokban, ám az intervallumos módszerek ennek ellenére a kevés jól használható alternatíva egyikét nyújtják bizonyos speciális feladatoknál – ilyenek például a nemlineáris egyenletrendszerek többdimenziós garantált megoldási módszerei és a globális optimalizálási feladatok. Bár az intervallumos módszerek részletes bemutatása meghaladja e könyv kereteit, a megfelelő helyeken jelezni fogjuk az intervallum-alapú szoftverek elérhetőségét.

1.3.11 Komplex aritmetika

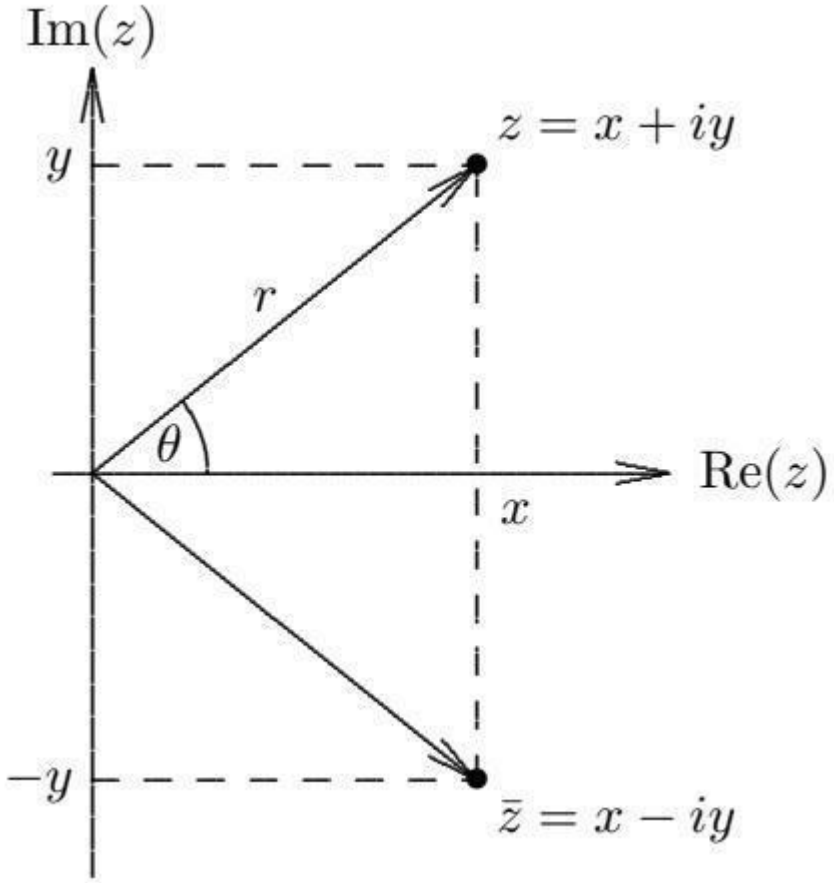
Ahogyan az m/n alakú törtek – ahol m és n egész számok – az egész számok halmazát bővítve a racionális számokat alkotják, az algebrai számok (például $\sqrt{2}$) és a transzcendens számok (például π) pedig a racionális számokat bővítve a valós számokat, úgy a valós számok $\sqrt{-1}$-gyel való bővítése a *komplex* számokat eredményezi. Ebben a könyvben a komplex számokat kifejezetten csak a sajátérték-feladatok (4. fejezet) és a Fourier-transzformációk (12. fejezet) tárgyalásánál használjuk. Máshol az egyszerűség kedvéért nagyrészt a valós esetre szorítkozunk, ám az általunk tárgyalt algoritmusok többsége a komplex esetre is alkalmazható.

Ahogy a matematikában szokásos, ebben a könyvben is a $\sqrt{-1}$ képzetes egységet i-vel jelöljük, noha egyes mérnöki tudományágak ehelyett j-t használnak. Egy adott z komplex szám felírható a

$$z = x + iy$$

alakban, ahol az $x = \mathrm{Re}(z)$ és $y = \mathrm{Im}(z)$ valós számok rendre a z valós és képzetes részei. Az összes komplex szám halmazát a $\mathbb{C} = \{x + iy : x, y \in \mathbb{R}\}$ jelöli. Egy z komplex szám *valós*, ha $\mathrm{Im}(z) = 0$, és *képzetes* (vagy tisztán képzetes), ha $\mathrm{Re}(z) = 0$.

$\mathbb{C}$-t elképzelhetjük egy kétdimenziós Descartes-féle síkként – ez a *komplex számsík* (vagy más néven *Argand-diagram*) –, amelynek két valós koordinátáját egy adott komplex szám valós és képzetes része adja meg. Konvenció szerint a valós rész a vízszintes, a képzetes rész pedig a függőleges tengelyen szerepel a komplex számsíkban (lásd az 1.5. ábrát). Az $i = \sqrt{-1}$-vel való szorzás a komplex számsíkban $\pi/2$-vel való elforgatásnak felel meg. Hasonlóképpen az $i^2 = -1$-gyel való szorzás π-vel való elforgatásnak, az $i^4 = 1$-gyel való szorzás pedig 2π-vel való elforgatásnak (azaz az 1, ahogy várható, a multiplikatív egység).



1.5. ábra: Komplex számsík a z komplex szám és $\bar{z}$ komplex konjugáltjának Descartes- és polárkoordinátás ábrázolásával.

Két $z_1 = x_1 + iy_1$ és $z_2 = x_2 + iy_2$ komplex szám szabványos aritmetikai műveletei a következőképpen definiáltak:

$$\begin{aligned}
z_1 + z_2 &= (x_1 + x_2) + i(y_1 + y_2),\\
z_1 - z_2 &= (x_1 - x_2) + i(y_1 - y_2),\\
z_1 \times z_2 &= (x_1x_2 - y_1y_2) + i(x_1y_2 + x_2y_1),\\
z_1/z_2 &= \left(\frac{x_1x_2 + y_1y_2}{x_2^2 + y_2^2}\right) + i\left(\frac{x_2y_1 - x_1y_2}{x_2^2 + y_2^2}\right), \quad \text{feltéve, hogy } z_2 \neq 0.
\end{aligned}$$

Megjegyezzük, hogy két komplex számon végzett aritmetikai művelet kettőtől tizenegy valós aritmetikai műveletet igényel – a konkrét komplex művelettől függően –, így a komplex aritmetika lényegesen költségesebb lehet, mint a valós aritmetika.

Egy $z = x + iy$ komplex szám *komplex konjugáltját* $\bar{z}$ jelöli, és

$$\bar{z} = x - iy.$$

Egy komplex szám és komplex konjugáltja tehát a valós tengelyre nézve szimmetrikus (egymás tükörképei) (lásd az 1.5. ábrát). Bármely $z = x + iy$ komplex számra

$$z\bar{z} = (x + iy)(x - iy) = x^2 + y^2$$

mindig valós és nemnegatív, így egy $z = x + iy$ komplex szám *modulusa* (más néven *abszolút értéke*) definiálható:

$$|z| = \sqrt{z\bar{z}} = \sqrt{x^2 + y^2},$$

amely éppen az (x, y) pont és az origó $(0, 0)$ közötti szokásos euklideszi távolság a komplex számsíkban.

Egy z komplex szám egy másik hasznos ábrázolása a polárkoordinátás alak:

$$z = x + iy = r(\cos\theta + i\sin\theta),$$

ahol $r = |z|$ és $\theta = \arctan(y/x)$ (lásd az 1.5. ábrát). Az Euler-azonosságot használva,

$$e^{i\theta} = \cos\theta + i\sin\theta,$$

megkapjuk az exponenciális ábrázolást:

$$z = re^{i\theta}.$$

Ez utóbbi különösen fontos lesz a Fourier-transzformációk tárgyalásánál (lásd a 12. fejezetet). Vegyük észre, hogy az exponenciális ábrázolásban a komplex szorzás és osztás egyszerűbb alakú:

$$\begin{aligned} z_1 \times z_2 &= r_1 r_2\, e^{i(\theta_1+\theta_2)}, \\ z_1/z_2 &= (r_1/r_2)\, e^{i(\theta_1-\theta_2)}. \end{aligned}$$

Egy számítógépen egy komplex számot jellemzően két szokásos lebegőpontos értékkel ábrázolunk, amelyek a valós és a képzetes résznek felelnek meg. A komplex aritmetika támogatása nyelvenként és rendszerenként változó; a Fortran minden változata például kifejezett támogatást ad a komplex adattípusokhoz és aritmetikához, míg a C és leszármazottai jellemzően nem. A MATLAB (lásd az 1.4.2. szakaszt) zökkenőmentesen támogatja a komplex aritmetikát; sőt a MATLAB elsődleges numerikus adattípusa komplex. A komplex aritmetikát a Python is támogatja – a képzetes számok `j` utótagúak, az ezekhez tartozó függvények pedig a `cmath` modulban találhatók.

1.4 Matematikai szoftver

Ez a könyv a numerikus analízis és a tudományos számítások témáinak széles körét tárgyalja. Bár minden témának érintjük az alapvető aspektusait, arra nincs lehetőségünk, hogy bármelyiket is kimerítően tárgyaljuk. Ahhoz, hogy érdekes számítási feladatokat tudjunk megoldani, gyakran szakemberek által fejlesztett matematikai szoftverekre támaszkodunk. Ha az algoritmikus részleteket az ilyen szoftverekre bízzuk, figyelmünket a helyes feladatmegfogalmazásra és az eredmények értelmezésére összpontosíthatjuk. Minden feladattípusnál csak a legalapvetőbb algoritmusokat tekintjük át, elsősorban az általuk nyújtott betekintés kedvéért – hogy megalapozottan tudjunk megfelelő módszert választani, és azt körültekintően alkalmazni. Elsődleges célunk az, hogy a matematikai szoftverek értő felhasználói legyünk, ne pedig az alkotói.

Mielőtt rátérnénk a jó matematikai szoftverek konkrét forrásaira, foglaljuk össze azokat a kívánatos tulajdonságokat, amelyekkel az ilyen szoftvereknek rendelkezniük kell (a felsorolás nem tükröz fontossági sorrendet):

- **Megbízhatóság:** könnyű feladatok esetén mindig helyesen működik.
- **Robusztusság:** nehéz feladatok esetén is rendszerint működik, ám amikor mégis kudarcot vall, azt elegánsan és informatív módon teszi.
- **Pontosság:** a feladat és a bemenő adatok által indokolt mértékben pontos eredményt ad, lehetőleg az elért pontosság becslésével együtt.
- **Hatékonyság:** a végrehajtási idő és a memóriaigény megközelíti a megoldandó feladat elméleti minimumát.
- **Hordozhatóság:** kis módosítással vagy anélkül is adaptálható az új számítási környezetekhez.
- **Karbantarthatóság:** könnyen érthető és módosítható.
- **Használhatóság:** kényelmes és jól dokumentált felhasználói felülettel rendelkezik.
- **Alkalmazhatóság:** feladatok széles körét képes megoldani.

Nyilvánvaló, hogy ezek a tulajdonságok gyakran ütköznek egymással, és ritka az olyan szoftver, amely mindegyiknek maradéktalanul megfelel. Ez a lista azonban segít a matematikai szoftverek felhasználóinak felismerni, milyen tulajdonságokat érdemes keresniük, a fejlesztők számára pedig kijelöl néhány elérendő célt.

1.4.1 Matematikai szoftverkönyvtárak Itt sorolunk fel néhány széles körben elérhető, általános célú matematikai szoftverforrást. Ennek a könyvnek minden fejezete végén konkrét rutinokat ismertetünk adott típusú feladatokhoz – mind ezekből az általános könyvtárakból, mind specializáltabb csomagokból.

- **FMM:** Forsythe, Malcolm és Moler *Computer Methods for Mathematical Computations* című könyvéhez [152] tartozó szoftver. Elérhető a Netlibről.
- **GAMS:** *Guide to Available Mathematical Software*, amelyet a National Institute of Standards and Technology (NIST) állít össze. `gams.nist.gov`
- **GSL (GNU Scientific Library):** Átfogó matematikai szoftverkönyvtár C és C++ nyelvhez. `www.gnu.org/software/gsl/`
- **HSL (Harwell Subroutine Library):** Az angliai Harwell Laboratóriumban kifejlesztett szoftver. `www.hsl.rl.ac.uk`

- **IMSL (International Mathematical and Statistical Libraries):** Átfogó matematikai szoftverkönyvtár. A Rogue Wave Software, Inc. kereskedelmi terméke. `www.roguewave.com/products-services/imsl-numerical-libraries`
- **KMN:** Kahaner, Moler és Nash *Numerical Methods and Software* [262] könyvéhez tartozó szoftver.
- **NAG (Numerical Algorithms Group):** Átfogó matematikai szoftverkönyvtár. A NAG Inc. kereskedelmi terméke. `www.nag.com`
- **NAPACK:** Hager *Applied Numerical Linear Algebra* [220] könyvéhez tartozó szoftver. A lineáris algebrán kívül rutinokat tartalmaz nemlineáris egyenletekhez, feltétel nélküli optimalizáláshoz és gyors Fourier-transzformációkhoz is. Elérhető a Netlibről.
- **Netlib:** Sokféle forrásból származó ingyenes szoftver. `www.netlib.org`
- **NR (Numerical Recipes):** Press, Teukolsky, Vetterling és Flannery *Numerical Recipes* [377] könyvéhez tartozó szoftver. Elérhető Fortran-, C- és C++-változatban. `numerical.recipes`
- **NUMAL:** Az amszterdami Mathematisch Centrumban kifejlesztett szoftver. Algol- és Fortran-változatban is létezik, de leginkább Lau *A Numerical Library for Scientists and Engineers* [297, 298] könyveiből érhető el C és Java változatban.
- **NUMERALGO:** A *Numerical Algorithms* folyóiratban megjelent szoftverek. Elérhető a Netlibről.
- **PORT:** A Bell Laboratories-ben kifejlesztett szoftver. Egyes rutinok szabadon elérhetők a Netlibről, más részek csak kereskedelmi licenc alatt.
- **SciPy/NumPy:** Átfogó, nyílt forráskódú matematikai szoftverkönyvtár Pythonhoz. `www.scipy.org`
- **SLATEC:** Egy USA-beli kormányzati laboratóriumokból álló konzorcium által összeállított szoftver. Elérhető a Netlibről.
- **SOL:** Optimalizálási és kapcsolódó feladatokhoz készült szoftver a Stanford University Systems Optimization Laboratoryjából. `web.stanford.edu/~saunders/brochure/broch`
- **TOMS:** Az *ACM Transactions on Mathematical Software* (korábban *Collected Algorithms of the ACM*) folyóiratban megjelent szoftverek. Az algoritmusokat szám (megjelenési sorrendben) és név szerint egyaránt azonosítjuk. Elérhető a Netlibről.

1.4.2 Tudományos számítási környezetek Az imént felsorolt szoftverkönyvtárak olyan szubrutinokat tartalmaznak, amelyeket a felhasználó által írt – rendszerint egy hagyományos programozási nyelvben, például Fortranban vagy C-ben készült – programoknak kell meghívniuk. A tudományos számítások egyre népszerűbb alternatívája az olyan interaktív környezetek használata, amelyek erőteljes, kényelmesen elérhető, beépített matematikai képességeket kínálnak, gyakran kifinomult grafikával és egy nagyon magas szintű programozási nyelvvel párosítva, amely új algoritmusok gyors prototipizálására alkalmas.

Az egyik leggyakrabban használt ilyen számítási környezet a MATLAB, amely a The MathWorks, Inc. (`www.mathworks.com`) kereskedelmi terméke. A MATLAB (MATrix LABoratory) egy interaktív rendszer, amely kiterjedt matematikai képességeket – különösen lineáris algebrában –, nagy teljesítményű tudományos vizualizációt, magas szintű programozási nyelvet és többféle választható „toolboxot” ötvöz, amelyek spe-

ciális képességeket nyújtanak egy-egy alkalmazási területhez, például jelfeldolgozáshoz, képfeldolgozáshoz, szabályozáshoz, rendszeridentifikációhoz, optimalizáláshoz vagy statisztikához. A MATLAB sokféle számítógépre elérhető, szakmai és hallgatói kiadásban egyaránt. Hasonló képességeket nyújt az Octave, amely szabadon elérhető (`www.gnu.org/software/octave`). További, egyre népszerűbb alternatíva a Python, amely általános célú magas szintű programozási nyelvet, interaktív végrehajtási környezetet, kiterjedt matematikai szoftverkönyvtárakat (pl. SciPy) és átfogó ábrázolási képességeket (Matplotlib) kínál.

Az interaktív számítási környezetek egy másik családja elsősorban szimbolikus (nem pedig numerikus) számításon alapul, amit gyakran *komputeralgebrának* neveznek. Ezek a csomagok – az Axiom, Derive, Magma, Maple, Mathematica, Maxima és Reduce – sok ugyanolyan matematikai és grafikus képességet nyújtanak, és ezenfelül szimbolikus differenciálást, integrálást, egyenletmegoldást, polinommanipulációt és hasonlókat kínálnak, valamint tetszőleges pontosságú aritmetikát is. A MATLAB egy „Symbolic Math” toolbox révén szintén rendelkezik szimbolikus számítási képességekkel, Pythonhoz pedig a SymPy könyvtár kínál hasonló lehetőségeket.

Mivel a MATLAB és a Python ezek közül a környezetek közül a leggyakrabban használtak a könyvben tárgyalt feladattípusokhoz, az egyes feladatkategóriákhoz rendelkezésre álló szoftverek összegzéseiben a hagyományos szoftverkönyvtárak mellett konkrét MATLAB- és Python-függvényeket – akár az alapkörnyezetből, akár a kiegészítő toolboxokból és könyvtárakból – is felsorolunk.

1.4.3 Kiterjesztett aritmetikai csomagok A többszörös pontosságú aritmetikát biztosító szoftvercsomagok közé tartoznak a TOMS-ból származó MP(#524), FM(#693), MPFUN(#719) és ZM(#786); a CLN (Class Library for Numbers); a GMP (GNU Multiple Precision Arithmetic Library); a Python `mpmath` könyvtára; valamint az NR [377] és [378] rutinjai. Intervallum- vagy tartomány-aritmetikát megvalósító szoftvercsomagok és rendszerek többek között a TOMS-ból származó INTLIB(#737, #763); a PROFIL/BIAS [278]; az Arb (`arblib.org`); a MATLAB-hoz készült INTLAB eszköztár; valamint az [1]-hez tartozó szoftver.

1.4.4 Gyakorlati tanácsok a szoftverhasználathoz Ez a szakasz gyakorlati tanácsokat tartalmaz a könyvben hivatkozott szoftverek beszerzésére és használatára, különös tekintettel az egyes fejezetek végén található számítógépes feladatok alapján készíthető programozási házi feladatokra. A számítógépes feladatok egyik konkrét szoftvertől vagy programozási nyelvtől sem függnek, így több lehetőség is rendelkezésre áll. Egy adott esetben a legjobb választás a felhasználó tapasztalatától, erőforrásaitól és céljaitól függ.

A hivatkozott szoftverek sokféle forrásból származnak: nagy kereskedelmi könyvtárakból, például az IMSL-ből és a NAG-ból; nyilvános, szabadon hozzáférhető tárolókból, például a Netlibről; valamint tudományos számítási környezetekből, például a MATLAB-ból és a Pythonból. Sok akadémiai és ipari számítási központban és laboratóriumban az ilyen szoftverek reprezentatív mintája már telepítve és használatra kész állapotban rendelkezésre áll. Mindenesetre e könyv összes számítógépes feladatához bőséges mennyiségű szoftver szerezhető be ingyenesen az interneten keresztül, vagy jelképes összegért egyéb forrásokból (például a kísérő tankönyvekből). A megfelelő

szoftver megtalálása, letöltése és telepítése hasznos, valós tapasztalatot jelent, és az ennek során megszerzett készségek fontos gyakorlati kiegészítői a könyvben tanultaknak.

Talán a legfontosabb döntés a programozási nyelv megválasztása. Ahogy az 1.4.2. szakaszban már tárgyaltuk, a MATLAB (vagy az Octave) és a Python környezetei nemcsak magas szintű programozási nyelvet nyújtanak, hanem viszonylag zökkenőmentes hozzáférést adnak kiterjedt matematikai szoftverkönyvtárakhoz is. Az ilyen környezetek felhasználói több előnyt is élvezhetnek. A felhasználói programok általában sokkal rövidebbek lesznek az explicit deklarációk, a memóriakezelés és sok ciklus mellőzhetősége miatt. További bónusz a beépített grafika, amely feleslegessé teszi az adatok külön utófeldolgozási fázisban történő ábrázolását.

A hagyományos programozási nyelvek közül a Fortran volt a tudományos számítások eredeti nyelve, és a korábbi szoftverkönyvtárak és alkalmazási kódok nagy része Fortranban íródott. E könyv használata során a Fortran-felhasználók előnyére válik, hogy ebben a nyelvben áll rendelkezésre a legszélesebb szoftverválaszték, és kompatibilis a létező alkalmazási kódok túlnyomó többségével. Emellett, mivel a Fortran viszonylag szigorú nyelv, és a hozzá tartozó fordítók sok év optimalizálási munkáján vannak túl, a Fortran-fordítók egyes számítógépes rendszereken gyakran hatékonyabb futtatható kódot állítanak elő.

A C sokoldalúbb és kifejezőbb nyelv, mint a Fortran, és a C (valamint rokon nyelvei, például a C++ és a Java) gyakrabban szerepel a kezdő programozási tárgyak tananyagában. Ha ezzel a könyvvel C-t szeretnél használni, bőséges mennyiségű szoftver áll rendelkezésedre. Ezenfelül sok számítógépes rendszeren viszonylag egyszerű Fortran-rutinokat C-programokból meghívni. A főbb eltérések, amelyekre figyelni kell: a rutinok nevei kissé módosulhatnak (gyakran a szokásos név elé vagy után kerül egy aláhúzás); minden argumentumot a Fortran-szubrutinoknak cím szerint (azaz mutatóként C-ben) kell átadni; továbbá a C és a Fortran ellentétes konvenciókat használ a tömbök tárolására (a C a mátrixokat sorfolytonosan, a Fortran pedig oszlopfolytonosan tárolja). Végül a Fortran-forráskód automatikusan át is alakítható C-kódra a Netlibről elérhető `f2c` konverterrel, így az – például az interneten – beszerzett Fortran-rutinok könnyen használhatók C-programokkal.

A könyv egyes számítógépes feladatai grafikus kimenetet igényelnek. A szükséges ábrák előállítására több lehetőség is rendelkezésre áll, a használt számítási környezettől függően. Az olyan interaktív környezetekben, mint a MATLAB vagy a Python, kiterjedt grafikai képességek automatikusan elérhetők. A hagyományos programozási nyelvekkel való használatra szabadon elérhető grafikai csomagok többek között a `gnuplot` (`www.gnuplot.info`), a `pgplot` (`www.astro.caltech.edu/~tjp/pgplot`) és a `plplot` (`plplot.sourceforge.net`). Kifinomultabb és erőteljesebb tudományos vizualizációs rendszerek is léteznek, ám ezek képességei messze meghaladják a könyv számítógépes feladataihoz szükséges, viszonylag egyszerű ábrázolásokat.

Egy másik fontos programozási szempont a teljesítmény. A mai mikroprocesszoralapú számítógépes rendszerek teljesítménye gyakran kritikusan függ a memóriahierarchia (regiszterek, gyorsítótár, RAM, lemez stb.) megfontolt kihasználásától, mind a programozó, mind az optimalizáló fordító részéről. Ezért nemcsak a megfelelő algoritmust kell megválasztani, hanem gondosan meg is kell azt valósítani, hogy maximali-

záljuk az adatok újrafelhasználását, miközben azok a memóriahierarchia leggyorsabb elérési idejű részeiben vannak. A programozás teljesítményre vonatkozó szempontjairól lásd [121, 188, 201]. Szerencsére az ilyen szintű programozás részletei a könyvben ajánlott könyvtári rutinok belsejében általában rejtve maradnak a felhasználó elől. Ez csak egyike azon számos előnynek, amely a meglévő, szakemberek által írt tudományos szoftverek – ahol csak lehetséges – használatából fakad.

Ha olyan tudományos számítási környezetet használsz, mint a MATLAB vagy a Python, tudnod kell, hogy jelentős teljesítménykülönbségek lehetnek a beépített – általában nagyon gyors – műveletek, és a felhasználó által explicit módon megírt kódok között, amelyek az értelmező (interpreter) üzemmód és a memóriakezelés többletköltsége miatt általában sokkal lassabbak. Ezért ilyen körülmények között óvatosnak kell lenni a teljesítmény-összehasonlításokkal. Előfordulhat például, hogy egy algoritmus elvileg rosszabb egy másiknál, a gyakorlatban mégis jobban teljesít, pusztán azért, mert hatékonyabban hasznosítja a gyors, beépített műveleteket.

1.5 Történeti jegyzetek és további olvasnivaló

A ma numerikus analízisnek vagy tudományos számításoknak nevezett tudományág jóval megelőzi a modern számítógépek megjelenését. A ma is használatos fogalmak és módszerek közül sokat a huszadik század előtti nagyok – Newton (1642–1727), Euler (1707–1783), Lagrange (1736–1813), Laplace (1749–1827), Legendre (1752–1833), Gauss (1777–1855), Cauchy (1789–1857), Jacobi (1804–1851), Adams (1819–1892), Csebisev (1821–1894), Hermite (1822–1901), Laguerre (1834–1886) és mások – fogalmaztak meg először, és nevük újra meg újra felbukkan ebben a könyvben. A fő feladat akkoriban is, akárcsak ma, a fizikában, a csillagászatban, a geodéziában és más tudományterületeken felmerülő matematikai feladatok közelítő megoldására szolgáló hatékony módszerek megtalálása volt. Sőt, a számítási erőforrások hatékony felhasználása még kritikusabb volt, amikor ceruzával, papírral és emberi agyerővel (esetleg asztali számológéppel) dolgoztak, mint ma egy modern, nagy sebességű számítógéppel.

Nagyrészt a modern számítógépek egyszerűen megnövelték a megoldható feladatok méretét. Emellett szükségessé tették a kerekítési hibák gondosabb elemzését és kontrollját is, mert a számítást már nem ember végzi, aki szükség szerint további pontossággal dolgozhat. Nem kétséges azonban, hogy a digitális számítógépek kifejlődése adta a lökést a numerikus analízis virágzó, termékeny és lendületesen fejlődő területté válásához, amely lehetővé tette, hogy a számítás ma a modern természettudomány és mérnöki gyakorlat minden területén mindenütt jelen legyen. Sőt a számítás az elmélet és a kísérlet mellett a tudományos ismeretek és a mérnöki gyakorlat előrehaladásának egyenrangú és nélkülözhetetlen partnerévé vált [267].

A numerikus analízis korai történetéről lásd [193]-at; a tudományos számításoknak a számítógépes korszakban való újabb fejlődéséről lásd [57, 63, 346]-ot. A numerikus analízis irodalma – a tankönyvektől a kutatási monográfiákon át a folyóiratokig – jóval terjedelmesebb annál, semhogy itt megfelelően tárgyalni lehetne. Ebben a szövegben megpróbáljuk megfelelően megadni az elismerést a bemutatott fő gondolatok eredetének (legalábbis azoknak, amelyek nem nyilvánvalók már a névről), és (rendszerint másodlagos) forrásokat idézünk a további olvasáshoz, ám ezek a hivatkozások

és ajánlások semmiképpen sem teljesek. Túl sok kiváló általános numerikus analízis tankönyv létezik ahhoz, hogy mindegyiket megemlítsük, de közülük sok ma is érdemes olvasmány (beleértve a régebbieket is, amelyek közül sokat olcsó utánnyomásokban is újra kiadtak). Csak a tárgyalásunkhoz leginkább közvetlenül kapcsolódókat idézzük.

A legtöbb numerikus analízis tankönyv általánosan tárgyalja a hibaelemzést. A kerekítési hibák elemzésének alapművei közé tartozik az [500] – értékes meglátások kincsesháza. Szerzője, James H. Wilkinson kulcsszerepet játszott a hátraható hibaelemzés fogalmának kidolgozásában és elterjesztésében, és számos híres „számítási ellenpéldáért" is felelős, amelyek váratlan feladatokban és algoritmusokban fellépő numerikus instabilitásokat tárnak fel [502, 503]. Hasonló szellemben készült újabb munka a [237]. A hibaelemzés automatizálására – beleértve az intervallumaritmetikát – különféle megközelítésekről lásd [1, 6, 213, 254, 329, 337, 338, 348, 469]-et. Hibaelemzésre szolgáló MATLAB toolboxot tárgyal [69]. A kondicionáltság általános fogalmának egy korai tárgyalása [385]-ben található.

A számítógépes aritmetika általános tárgyalásai közé tartoznak [54, 282, 283, 287, 356, 363]. A lebegőpontos aritmetika részletes tárgyalásához lásd [345, 423]-at; rövidebb áttekintéshez [279]-et. A lebegőpontos aritmetika szabványosítására irányuló erőfeszítést és a létrejött szabvány magas színvonalát nagyrészt William Kahan inspirálta, aki számos ismert számítási ellenpéldáért is felelős. Az IEEE lebegőpontos szabvány [247]-ben található. A lebegőpontos aritmetikáról és az IEEE szabványról szóló hasznos bemutatók közé tartozik [189, 361].

A matematikai szoftver mint a numerikus analízis és az informatika részterületének megjelenéséről lásd a [80] áttekintést és a [42, 88, 131, 230, 251, 386, 387] gyűjteményeket. A szakmai minőségű szoftvereket (nem csak szemléltető kódrészleteket) tartalmazó talán legkorábbi numerikus módszerek tankönyv a [406] volt, amely hangvételében, stílusában és tartalmában hasonló Forsythe, Malcolm és Moler nagy hatású könyvéhez [152], amely ezt a megközelítést népszerűsítette. További, bevezető szintű tárgyalást és példákat nyújtó tankönyvek közé tartoznak [13, 18, 85, 123, 208, 214, 240, 262, 331, 353, 380, 407, 428, 472, 481]. Haladóbb, általános tankönyvek közé tartoznak [17, 87, 92, 93, 111, 172, 196, 248, 275, 358, 371, 379, 402, 429, 434, 448, 459, 474]. A numerikus számítás buktatóinak felismeréséhez és elkerüléséhez gyakorlati tanácsokat nyújt [3, 4, 127, 151, 422].

Strang könyvei [435, 439, 440, 441] kiváló hátteret és rálátást nyújtanak az alkalmazott matematika számos olyan területére, amelyek általában is – és különösen e könyv szinte minden fejezete szempontjából – fontosak a numerikus számításhoz. A tudományos programozás elemi bevezetéseihez lásd [265, 480, 482, 517, 518]-at. Numerikus szoftverek tervezéséhez, megvalósításához és teszteléséhez – nem pusztán a használatukhoz – lásd [43, 80, 127, 328, 354]-et. További számítógépes gyakorlatok és projektek találhatók [89, 94, 130, 155, 167, 170, 200, 203, 205, 294, 353, 418]-ban.

Áttekintő kérdések

- **1.1.** Igaz vagy hamis: Egy feladat rosszul kondicionált, ha a megoldása nagyon érzékeny a feladat adatainak kis változásaira.
- **1.2.** Igaz vagy hamis: Nagyobb precíziójú aritmetika használata egy rosszul kondicionált feladatot jobban kondicionálttá tesz.

- **1.3.** Igaz vagy hamis: Egy feladat kondicionáltsága függ a megoldására használt algoritmustól.
- **1.4.** Igaz vagy hamis: Egy jó algoritmus a feladat kondíciójától függetlenül pontos megoldást ad.
- **1.5.** Igaz vagy hamis: Egy feladat megoldására választott algoritmusnak nincs hatása a továbbterjedő adathibára.
- **1.6.** Igaz vagy hamis: Egy stabil algoritmus jól kondicionált feladatra alkalmazva szükségképpen pontos megoldást ad.
- **1.7.** Igaz vagy hamis: Ha két valós szám lebegőpontos számként pontosan ábrázolható, akkor a rajtuk végzett valós aritmetikai művelet eredménye is ábrázolható lebegőpontos számként.
- **1.8.** Igaz vagy hamis: A lebegőpontos számok egyenletesen oszlanak el az értékkészletükön.
- **1.9.** Igaz vagy hamis: A lebegőpontos összeadás asszociatív, de nem kommutatív.
- **1.10.** Igaz vagy hamis: Egy lebegőpontos számrendszerben az alulcsordulási szint az a legkisebb pozitív szám, amely az 1-hez hozzáadva perturbálja az 1-et.
- **1.11.** Igaz vagy hamis: Az IEEE dupla pontosságú lebegőpontos aritmetika mantisszája pontosan kétszer olyan hosszú, mint az IEEE egyszeres pontosságúé.
- **1.12.** Mely három tulajdonság jellemzi a jól kitűzött feladatot?
- **1.13.** Sorolj fel három hibaforrást a tudományos számításokban.
- **1.14.** Fejtsd ki a különbséget a csonkítási (vagy diszkretizációs) hiba és a kerekítési hiba között.
- **1.15.** Fejtsd ki a különbséget az abszolút és a relatív hiba között.
- **1.16.** Fejtsd ki a különbséget a számítási hiba és a továbbterjedő adathiba között.
- **1.17.** Fejtsd ki a különbséget a precízió és a pontosság között.
- **1.18.** (a) Mit értünk egy feladat kondicionáltságán? (b) Függ-e ez a megoldásra használt algoritmustól? (c) Függ-e ez a megoldáshoz használt aritmetika precíziójától?
- **1.19.** Ha egy számítási feladat kondíciószáma 1, ez jó vagy rossz? Miért?
- **1.20.** Fejtsd ki a különbséget a relatív és az abszolút kondíciószám között.
- **1.21.** Mi az inverz feladat? Hogyan viszonyul egymáshoz egy feladat és az inverz feladat kondicionáltsága?
- **1.22.** (a) Mit értünk egy számított eredmény hátraható hibáján? (b) A hátraható hibaelemzés szemszögéből mikor tekinthető jónak egy adott feladat közelítő megoldása?
- **1.23.** Tegyük fel, hogy egy adott feladatot egy adott algoritmussal oldunk meg. Az alábbiak mindegyikéről mondd meg, hogy függ-e az algoritmus stabilitásától, és miért: (a) továbbterjedő adathiba; (b) a számított eredmény pontossága; (c) a feladat kondicionáltsága.
- **1.24.** (a) Fejtsd ki a különbséget az előreható és a hátraható hiba között. (b) Hogyan kapcsolódik az előreható és a hátraható hiba mennyiségileg egymáshoz?
- **1.25.** Adott lebegőpontos számrendszer esetén írd le szavakkal a gépi számok eloszlását a valós egyenes mentén.
- **1.26.** Lebegőpontos aritmetikában általában melyik károsabb: az alulcsordulás vagy a túlcsordulás? Miért?
- **1.27.** Lebegőpontos aritmetikában az alábbi műveletek közül, két pozitív lebegőpontos operandusra, melyek okozhatnak túlcsordulást? (a) összeadás; (b) kivonás; (c) szorzás; (d) osztás.

- **1.28.** Lebegőpontos aritmetikában az alábbi műveletek közül, két pozitív lebegőpontos operandusra, melyek okozhatnak alulcsordulást? (a) összeadás; (b) kivonás; (c) szorzás; (d) osztás.
- **1.29.** Sorolj fel két okot, amiért a lebegőpontos számrendszereket rendszerint normalizálják.
- **1.30.** Egy lebegőpontos rendszerben mely mennyiség határozza meg egy adott valós szám gépi számmal való ábrázolásakor fellépő maximális relatív hibát?
- **1.31.** (a) Fejtsd ki a különbséget a „nulla felé kerekítés” és a „legközelebbihez kerekítés” szabályai között egy lebegőpontos rendszerben. (b) E két szabály közül melyik pontosabb? (c) Mekkora számszerű különbséget okoz ez az ϵ_{mach} egységnyi kerekítési hibában?
- **1.32.** Egy p számjegyű bináris lebegőpontos rendszerben legközelebbihez kerekítés mellett mekkora az ϵ_{mach} egységnyi kerekítési hiba értéke?
- **1.33.** Egy fokozatos alulcsordulást (szubnormális számokat) engedélyező lebegőpontos rendszerben minden szám ábrázolása egyértelmű marad-e? Miért?
- **1.34.** Egy lebegőpontos rendszerben két gépi szám szorzata rendszerint pontosan ábrázolható a lebegőpontos rendszerben? Miért?
- **1.35.** Egy lebegőpontos rendszerben két nem nulla gépi szám hányadosa mindig pontosan ábrázolható a lebegőpontos rendszerben? Miért?
- **1.36.** (a) Adj példát, amely megmutatja, hogy a lebegőpontos összeadás nem feltétlenül asszociatív. (b) Adj példát, amely megmutatja, hogy a lebegőpontos szorzás nem feltétlenül asszociatív.
- **1.37.** (a) Milyen körülmények között lép fel számjegykioltás egy lebegőpontos rendszerben? (b) A kioltás fellépése azt jelenti-e, hogy az azt kiváltó konkrét művelet valódi eredménye nem ábrázolható pontosan a lebegőpontos rendszerben? Miért? (c) Miért általában rossz a kioltás?
- **1.38.** Adj példát olyan számra, amelynek a tízes számrendszerbeli alakja véges (azaz véges számú nem nulla számjegyet tartalmaz), de a bináris alakja nem.
- **1.39.** Adj példákat olyan lebegőpontos aritmetikai műveletekre, amelyek a `Inf` és a `NaN` kivételes értékek mindegyikét előállítják.
- **1.40.** Egy β alapú, p precíziójú, legközelebbihez kerekítést használó lebegőpontos rendszerben mi a maximális relatív hiba bármely nem nulla valós szám ábrázolásában, a rendszer tartományán belül?
- **1.41.** Fejtsd ki, miért gyakran rossz a két hasonló abszolút értékű szám kivonásakor fellépő kioltás, annak ellenére, hogy az eredmény az operandusokra nézve pontosan helyes lehet.
- **1.42.** Tegyük fel, hogy egy tízes alapú lebegőpontos rendszerrel dolgozunk, amelynek gépi precíziója $\epsilon_{\text{mach}} = 10^{-5}$, exponens-tartománya pedig ± 20. Mi az alábbi lebegőpontos aritmetikai műveletek eredménye? (a) $1 + 10^{-7}$; (b) $1 + 10^{3}$; (c) $1 + 10^{7}$; (d) $10^{10} + 10^{3}$; (e) $10^{10}/10^{-15}$; (f) $10^{-10} \times 10^{-15}$.
- **1.43.** Egy olyan lebegőpontos rendszerben, amelynek alulcsordulási szintje $\text{UFL} = 10^{-38}$, az alábbi számítások közül melyekben lép fel alulcsordulás?
  (a) $a = \sqrt{b^2 + c^2}$, ahol $b = 1$, $c = 10^{-25}$.
  (b) $a = \sqrt{b^2 + c^2}$, ahol $b = c = 10^{-25}$.
  (c) $u = (v \times w)/(y \times z)$, ahol $v = 10^{-15}$, $w = 10^{-30}$, $y = 10^{-20}$, $z = 10^{-25}$. Azokban az esetekben, ahol alulcsordulás lép fel, ésszerű-e egyszerűen nullára állítani az alulcsorduló mennyiséget?

- **1.44.** (a) Fejtsd ki szavakkal a különbséget egy lebegőpontos rendszer ϵ_{mach} egységnyi kerekítési hibája és UFL alulcsordulási szintje között. E két mennyiség közül (b) melyik függ csak a mantissza mezőjében található számjegyek számától? (c) melyik függ csak az exponens mezőjében található számjegyek számától? (d) melyik nem függ a használt kerekítési szabálytól? (e) melyiket nem befolyásolja a szubnormális számok engedélyezése?
- **1.45.** Legyen x_k pozitív számok szigorúan csökkenő, véges sorozata (azaz $x_k > x_{k+1}$ minden k-ra). Ha a számokat tetszőleges általunk választott sorrendben vehetjük, milyen sorrendben kellene összegezni a sorozatot a kerekítési hiba minimalizálása érdekében?
- **1.46.** Példa-e a kioltás a kerekítési hibára? Miért?
- **1.47.** (a) Magyarázd meg, miért lehet véges az összege lebegőpontos aritmetikában egy olyan divergens végtelen sornak, mint

$$\sum_{n=1}^{\infty} \frac{1}{n}.$$

(b) Melyik ponton fognak a részletösszegek megszűnni változni?

- **1.48.** Lebegőpontos aritmetikában, ha egy pozitív tagú konvergens végtelen sor

$$S = \sum_{i=1}^{\infty} x_i$$

összegét számítod természetes sorrendben, milyen megállási kritériumot alkalmaznál a lehető legnagyobb pontosság eléréséhez a legkevesebb taggal? - **1.49.** Fejtsd ki, miért nehéz lebegőpontos aritmetikában pontosan kiértékelni egy váltakozó előjelű végtelen sort, például

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$$

$x < 0$ esetén. - **1.50.** Ha f egy valós változós valós értékű függvény, akkor a deriváltjának

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

véges differencia közelítésében a csonkítási hiba $h \to 0$ esetén nullához tart. Ha lebegőpontos aritmetikát használunk, sorolj fel két tényezőt, amely korlátozza, mennyire kicsi h értéket tudunk a gyakorlatban használni. - **1.51.** Sorolj fel legalább két módot, ahogy a

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

másodfokú megoldóképlet kiértékelése lebegőpontos aritmetikában numerikus nehézségekbe ütközhet.

Feladatok

- **1.1.** Az átlagos emberi testhőmérsékletet szokás $98{,}6$ Fahrenheit-fokként megadni, amiről azt gondolhatnánk, hogy egy nagy populáció átlaga, három értékes jegyre kerekítve. Valójában azonban a $98{,}6$ egyszerűen a 37 Celsius-fokos érték Fahrenheit-megfelelője, amely csak két értékes jegyre pontos.
  (a) Mennyi a maximális relatív hiba az elfogadott értékben, feltéve, hogy az $\pm 0{,}05\,^\circ$F pontosságú?
  (b) Mennyi a maximális relatív hiba az elfogadott értékben, feltéve, hogy az $\pm 0{,}5\,^\circ$C pontosságú?
- **1.2.** Mennyi az abszolút és a relatív hiba, ha a π értékét a következő mennyiségekkel közelítjük?
  (a) 3; (b) $3{,}14$; (c) $22/7$.
- **1.3.** Ha az a a t pontos (valódi) értékű mennyiség közelítése, és a relatív hibája r, akkor a fogalmak definícióiból kiindulva bizonyítsd be, hogy $a = t(1 + r)$.
- **1.4.** Tekintsük a $\sin(x)$ függvény kiszámításának feladatát – különösen a továbbterjedő adathibát, azaz a függvényérték hibáját, amit az x argumentum h nagyságú perturbációja okoz.
  (a) Becsüld meg a $\sin(x)$ kiszámításának abszolút hibáját!
  (b) Becsüld meg a $\sin(x)$ kiszámításának relatív hibáját!
  (c) Becsüld meg a feladat kondíciószámát!
  (d) Az x argumentum mely értékeire nagyon érzékeny ez a feladat?
- **1.5.** Tekintsük az $f : \mathbb{R}^2 \to \mathbb{R}$ függvényt, amelyet az $f(x, y) = x - y$ definiál. A bemenet (x, y) nagyságát $|x| + |y|$-nal mérve, és feltételezve, hogy $|x| + |y| \approx 1$ és $x - y \approx \epsilon$, mutasd meg, hogy $\mathrm{cond}(f) \approx 1/\epsilon$. Mit mondhatsz a kivonás érzékenységéről?
- **1.6.** A szinuszfüggvényt a

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

végtelen sor adja meg. (a) Mennyi az előreható és a hátraható hiba, ha a szinuszfüggvényt a sor első tagjával közelítjük – azaz $\sin(x) \approx x$ – $x = 0{,}1$, $0{,}5$ és $1{,}0$ esetén? (b) Mennyi az előreható és a hátraható hiba, ha a szinuszfüggvényt a sor első két tagjával közelítjük – azaz $\sin(x) \approx x - x^3/6$ – $x = 0{,}1$, $0{,}5$ és $1{,}0$ esetén? - **1.7.** Egy lebegőpontos számrendszert négy egész szám jellemez: a β alap, a p precízió, valamint az exponenstartomány L alsó és U felső korlátja. (a) Ha $\beta = 10$, mik a p és az U legkisebb, illetve L legnagyobb értékei ahhoz, hogy a $2365{,}27$ és a $0{,}0000512$ egyaránt pontosan ábrázolható legyen egy normalizált lebegőpontos rendszerben? (b) Hogyan változna a válaszod, ha a rendszer nem normalizált, azaz fokozatos alulcsordulást is engedélyezünk? - **1.8.** Egy tízes alapú, $p = 6$ precíziójú lebegőpontos rendszerben legyen $x = 1{,}23456$ és $y = 1{,}23579$. (a) Hány értékes jegyet tartalmaz az $y - x$ különbség? (b) Ha a lebegőpontos rendszer normalizált, mi a minimális exponenstartomány, amely esetén az x, az y és az $y - x$ mindegyike pontosan ábrázolható? (c)

Az exponenstartománytól függetlenül pontosan ábrázolható-e az $y - x$ különbség, ha fokozatos alulcsordulás megengedett? Miért? - **1.9.** (a) Négy értékes jeggyel számoló decimális aritmetikával és az 1.1. példában adott képlettel számítsd ki a Föld felszínét $r = 6370$ km mellett! (b) Ugyanezzel a képlettel és precízióval számítsd ki a felszín megváltozását, ha a sugár értékét 1 km-rel növeljük! (c) Mivel $dA/dr = 8\pi r$, a felszín változása $8\pi rh$-val közelíthető, ahol h a sugár változása. Ezzel a képlettel – továbbra is négy számjegyű aritmetikával – számítsd ki a felszín megváltozását a sugár 1 km-es növekedése miatt! Hogyan viszonyul ez a közelítő képlettel kapott érték a (b) feladatrészben az „egzakt" képlettel számított értékhez? (d) Döntsd el, hogy az előző két eredmény közül melyik áll közelebb a helyeshez úgy, hogy mindkét számítást magasabb precízióval – mondjuk hat számjegyű decimális aritmetikával – megismétled! (e) Magyarázd meg az (a)–(d) pontokban kapott eredményeket! (f) Próbáld ki ezt a feladatot számítógépen is! Milyen pici h sugárváltoztatás esetén lép fel ugyanez a jelenség? Próbáld ki egyszeres és dupla pontossággal is, ha rendelkezésre áll! - **1.10.** Tekintsük az

$$\frac{1}{1-x} - \frac{1}{1+x}$$

kifejezést, feltételezve, hogy $x \neq \pm 1$. (a) Az x mely tartományában nehéz ezt a kifejezést pontosan kiszámítani lebegőpontos aritmetikában? (b) Add meg a tagok olyan átrendezését, amely az (a)-beli x-tartományban lebegőpontos aritmetikával pontosabb eredményt ad! - **1.11.** Ha $x \approx y$, akkor azt várnánk, hogy a $\log(x) - \log(y)$ kiszámítása során számjegykioltás lép fel. Másfelől $\log(x) - \log(y) = \log(x/y)$, és az utóbbiban nem lép fel számjegykioltás. Jelenti-e ez azt, hogy a $\log(x/y)$ kiszámítása valószínűleg jobb eredményt ad? (*Útmutatás:* Milyen értékre érzékeny a logaritmusfüggvény?) - **1.12.** (a) A matematikailag ekvivalens

$$x^2 - y^2 \quad \text{és} \quad (x-y)(x+y)$$

kifejezések közül melyik számítható ki pontosabban lebegőpontos aritmetikában? Miért? (b) Az x és y mely – egymáshoz viszonyított – értékeire van lényeges különbség a két kifejezés pontosságában? - **1.13.** Egy n-dimenziós x vektor euklideszi normáját az

$$\|x\|_2 = \left(\sum_{i=1}^{n} x_i^2 \right)^{1/2}$$

képlet definiálja. Hogyan kerülheted el a túlcsordulást és a káros alulcsordulást ebben a számításban? - **1.14.** Egy $[a, b]$ intervallum m középpontjának lebegőpontos aritmetikával történő kiszámítására a következő két képlet közül melyik jobb, és miért? Mikor? (*Útmutatás:* Találj ki példákat, amelyekben a képlet által adott „középpont" az $[a, b]$ intervallumon kívülre esik.) (a) $m = (a+b)/2{,}0$ (b) $m = a + (b-a)/2{,}0$ - **1.15.** Adj konkrét példákat, amelyek megmutatják, hogy a lebegőpontos összeadás

nem asszociatív, a következő lebegőpontos rendszerek mindegyikében: (a) az 1.9. példában szereplő „játék” lebegőpontos rendszer; (b) az IEEE egyszeres pontosságú lebegőpontos aritmetika. - **1.16.** Magyarázd meg, hogyan különbözhetnek a gyakorlatban az ϵ_{mach} egységnyi kerekítési hiba 1.3.5. szakaszban adott különféle definíciói! (*Útmutatás:* Tekintsd az 1.9. példa „játék” lebegőpontos rendszerét.) - **1.17.** Legyen x egy adott nem nulla lebegőpontos szám egy normalizált rendszerben, és y a vele szomszédos – szintén nem nulla – lebegőpontos szám. (a) Mi a minimális lehetséges távolság az x és y között? (b) Mi a maximális lehetséges távolság az x és y között? - **1.18.** Hány normalizált gépi szám van egy IEEE egyszeres pontosságú lebegőpontos rendszerben? Hány további gépi számot nyerünk, ha megengedjük a szubnormális számokat? - **1.19.** Egy IEEE egyszeres pontosságú lebegőpontos rendszerben mennyi az OFL legnagyobb gépi szám, és az UFL legkisebb pozitív normalizált gépi szám értéke? Hogyan változnak a válaszaid, ha a szubnormálisak is megengedettek? - **1.20.** Mi a $0{,}1$ decimális tört IEEE egyszeres pontosságú bináris lebegőpontos ábrázolása (a) csonkítással? (b) legközelebbihez való kerekítéssel? - **1.21.** (a) Egy lebegőpontos rendszerben az ϵ_{mach} egységnyi kerekítési hiba szükségképpen gépi szám-e? (b) Létezhet-e olyan lebegőpontos rendszer, amelyben $\epsilon_{\text{mach}} < \text{UFL}$? Ha igen, adj rá példát! - **1.22.** Tegyük fel, hogy az $ax^2 + bx + c = 0$ másodfokú egyenletet oldjuk meg $a = 1{,}22$, $b = 3{,}34$, $c = 2{,}28$ mellett, egy $\beta = 10$, $p = 3$ normalizált lebegőpontos rendszerben. (a) Mi a diszkrimináns, azaz a $b^2 - 4ac$ számított értéke? (b) Mi a diszkrimináns helyes értéke valós (egzakt) aritmetikában? (c) Mi a diszkrimináns számított értékének relatív hibája? - **1.23.** Tekintsünk egy $\beta = 10$, $p = 3$, $L = -98$ normalizált lebegőpontos rendszert! (a) Mennyi az UFL alulcsordulási szint értéke ebben a rendszerben? (b) Ha $x = 6{,}87 \times 10^{-97}$ és $y = 6{,}81 \times 10^{-97}$, mi az $x - y$ eredménye? (c) Mi lenne az $x - y$ eredménye, ha a rendszer megengedné a fokozatos alulcsordulást? - **1.24.** Tekintsük a következő állítást: ha két azonos előjelű lebegőpontos szám, x és y aránya legfeljebb a β alap (azaz $1/\beta \leq x/y \leq \beta$), akkor a különbségük, az $x - y$ pontosan ábrázolható a lebegőpontos rendszerben. Mutasd meg, hogy ez az állítás igaz $\beta = 2$ esetén, de adj ellenpéldát $\beta > 2$ esetére! - **1.25.** Néhány mikroprocesszor rendelkezik egy `mpyadd(a, b, c)` (szorzás-összeadás) utasítással, amely egyszeres pontosságú bemeneteket vár, és hozzáadja a c-t az a és b kétszeres pontosságú szorzatához, mielőtt normalizálná és egyszeres pontosságú eredményt adna vissza. Hogyan lehet egy ilyen utasítást felhasználni dupla pontosságú szorzatok kiszámítására úgy, hogy dupla pontosságú változókat egyáltalán nem használunk (azaz az a és b kétszeres pontosságú szorzata két egyszeres pontosságú változóban – például s-ben és t-ben – tárolódik)? - **1.26.** Igazold, hogy az 1.15. példában adott alternatív megoldóképlet valóban a másodfokú egyenlet helyes gyökeit adja (egzakt aritmetikában)! - **1.27.** Adj részletes magyarázatot a szórás számításához használt egylépéses képlet numerikus gyengeségére az 1.16. példában megadott kétlépéses képlethez képest!

Számítógépes feladatok

- **1.1.** Írj programot, amely kiszámítja a Stirling-formula

$$n! \approx \sqrt{2\pi n}\,(n/e)^n$$

által adott közelítés abszolút és relatív hibáját $n = 1, \ldots, 10$ esetén! Az abszolút hi-

ba nő vagy csökken n növekedésével? És a relatív hiba? - **1.2.** Írj programot, amely közelítő értékeket határoz meg az ϵ_{mach} egységnyi kerekítési hibára és az UFL alulcsordulási szintre, és teszteld egy valós számítógépen! (*Opcionális:* Meg tudod határozni a géped OFL túlcsordulási szintjét is? Ez trükkösebb, mert egy tényleges túlcsordulás végzetes programhibát okozhat.) Írasd ki a kapott értékeket decimális formában, és próbáld megállapítani a használt lebegőpontos rendszer mantissza- és exponensmezőjének bitszámát! - **1.3.** A legtöbb lebegőpontos rendszerben az egységnyi kerekítési hibára gyors közelítés kapható az

$$\epsilon_{\text{mach}} \approx |3 \cdot (4/3 - 1) - 1|$$

kifejezés kiszámításával. (a) Magyarázd meg, miért működik ez a trükk! (b) Próbáld ki különféle számítógépeken (egyszeres és dupla pontossággal egyaránt) és számológépeken, hogy megerősítsd a működését! (c) Működne-e ez a trükk egy $\beta = 3$ alapú lebegőpontos rendszerben? - **1.4.** Írj programot, amely a természetes logaritmus e alapszámát az

$$e = \lim_{n\to\infty} (1 + 1/n)^n$$

definícióból számítja ki! Konkrétan számítsd ki az $(1+1/n)^n$ értékét az $n = 10^k$, $k = 1, 2, \ldots, 20$ helyeken! Ha a használt programnyelvben nincs hatványozási operátor, használhatod a matematikailag ekvivalens

$$(1 + 1/n)^n = \exp\big(n \log(1 + 1/n)\big)$$

képletet, ahol az `exp` és a `log` a beépített függvények. Határozd meg a közelítéseid hibáját az `exp(1)` értékkel való összehasonlítás révén! A hiba mindig csökken n növekedésével? Magyarázd meg az eredményeidet! - **1.5.** (a) Tekintsük az

$$f(x) = (e^x - 1)/x$$

függvényt. A L'Hôpital-szabály alkalmazásával mutasd meg, hogy $\lim_{x\to 0} f(x) = 1$. (b) Ellenőrizd ezt az eredményt kísérletileg úgy, hogy írsz egy programot, amely kiszámítja az $f(x)$ értékét az $x = 10^{-k}$, $k = 1, \ldots, 15$ helyeken! Az eredményeid megegyeznek az elméleti várakozásokkal? Magyarázd meg, miért! (c) Végezd el a (b)-beli kísérletet ismét, ezúttal a matematikailag ekvivalens

$$f(x) = (e^x - 1)/\log(e^x)$$

megfogalmazást használva pontosan a jelzett formában, minden egyszerűsítés nélkül! Ha ez a változat jobban működik, meg tudod magyarázni a jelenség okát? - **1.6.** Tegyük fel, hogy $n + 1$ egyenközű pontot kell generálnunk az $[a, b]$ intervallumon, $h = (b - a)/n$ lépésközzel. (a) Lebegőpontos aritmetikában a következő két módszer közül melyik a jobb, és miért?

$$x_0 = a, \quad x_k = x_{k-1} + h, \quad k = 1, \dots, n$$

vagy

$$x_k = a + kh, \quad k = 0, \dots, n.$$

(b) Írj programot, amely mindkét módszert megvalósítja, és találj egy példát – mondjuk $a = 0$ és $b = 1$ mellett –, amely jól szemlélteti a különbséget közöttük!

- **1.7.** (a) Írj programot, amely egy függvény deriváltjának közelítő értékét a

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

végesdifferencia-képlettel számítja. Teszteld a programot a $\tan(x)$ függvényen $x = 1$-nél. Határozd meg a hibát a beépített $\sec(x)$ függvény négyzetével való összehasonlítással. Ábrázold a hiba nagyságát h függvényében $h = 10^{-k}$-ra, $k = 0, \dots, 16$. Mind h-t, mind a hiba nagyságát logaritmikus skálán érdemes használni. Van a hiba nagyságának minimuma? Hogyan viszonyul a hozzá tartozó h érték az 1.3. példában levezetett $h \approx \sqrt{\epsilon_{\text{mach}}}$ ökölszabályhoz? (b) Ismételd meg a feladatot a

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

középpontos differencia közelítéssel. - **1.8.** Tekintsük a

$$\sum_{n=1}^{\infty} \frac{1}{n}$$

végtelen sort. (a) Bizonyítsd be, hogy a sor divergens. (*Útmutatás:* Csoportosítsd a tagokat olyan halmazokba, amelyek az $1/(2^{k-1} + 1)$-től $1/2^k$-ig tartó tagokat tartalmazzák, $k = 1, 2, \dots$) (b) Magyarázd meg, miért ad véges összeget a sor lebegőpontos aritmetikában való összegzése. (c) Próbáld megjósolni, mikor szűnik meg változni a részletösszeg IEEE egyszeres, illetve dupla pontosságú lebegőpontos aritmetikában. A géped lebegőpontos műveletekre vonatkozó végrehajtási sebességét figyelembe véve próbáld megjósolni, mennyi ideig tartana minden egyes számítás befejezése. (d) Írj két programot a sor összegének kiszámítására: egyet egyszeres, egyet dupla pontossággal. Kövesd az összegzés előrehaladását úgy, hogy az indexet és a részletösszeget időnként kiírod. Milyen megállási kritériumot használnál? Milyen eredményt ad ténylegesen a géped? Hasonlítsd össze az eredményeidet a jóslataiddal, beleértve a szükséges futásidőt is. (*Figyelem:* Az egyszeres pontosságú változat várhatóan viszonylag gyorsan befejeződik, a dupla pontosságú azonban sokkal tovább tarthat, így lehet, hogy nem gyakorlati befejezésig futtatni, még bőkezű számítási kerettel sem.) - **1.9.** (a) Írj programot, amely az e^x exponenciális függvényt az

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$$

végtelen sorral számítja. (b) Természetes sorrendben összegezve milyen megállási kritériumot használnál? (c) Teszteld a programot $x = \pm 1, \pm 5, \pm 10, \pm 15, \pm 20$ értékekre, és hasonlítsd össze az eredményeket a beépített `exp(x)` függvénnyel. (d) Használhatod-e a sort ebben a formában pontos eredmények eléréséhez $x < 0$ esetén? (*Útmutatás:* $e^{-x} = 1/e^x$.) (e) Át tudod-e rendezni a sort, vagy át tudod-e csoportosítani a tagokat oly módon, hogy pontosabb eredményeket kapj $x < 0$ esetén? - **1.10.** Írj programot, amely az $ax^2 + bx + c = 0$ másodfokú egyenletet oldja meg a szabványos

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

képlettel vagy a

$$x = \frac{2c}{-b \mp \sqrt{b^2 - 4ac}}$$

alternatív képlettel. A programnak bemenetként a, b, c együtthatókat kell elfogadnia, kimenetként pedig az egyenlet két gyökét kell előállítania. A programnak észlelnie kell, ha a gyökök nem valósak, de nem szükséges explicit módon komplex aritmetikát használnia (például ebben az esetben visszaadhatja a komplex konjugált gyökpár valós és képzetes részét). Védekezz a felesleges túlcsordulás, alulcsordulás és a számjegykioltás ellen. Mikor kell használnod a két képlet közül melyiket? Tedd robusztussá a programot a szokatlan bemeneti értékekre is, például $a = 0$ vagy $c = 0$, amelyek egyébként az egyik képletet megbuktatnák. Bármely gyökhöz, amely a lebegőpontos rendszer tartományán belül van, pontosan számítsa ki az értékét, még akkor is, ha a másik gyök kívül esik. Teszteld a programot a következő együtthatóértékekkel:

a	b	c
6	5	-4
6×10^{154}	5×10^{154}	-4×10^{154}
0	1	1
1	-10^5	1
1	-4	$3{,}999999$
10^{-155}	-10^{155}	10^{155}

- **1.11.** A nem nulla vezető együtthatóval való osztással bármely harmadfokú egyenlet az

$$x^3 + ax^2 + bx + c = 0$$

alakra hozható. Ha az együtthatók valósak, akkor egy ilyen egyenletnek legalább egy valós gyöke van, amely zárt alakban a következőképpen számítható ki. Alkalmazd az $y = x + a/3$ helyettesítést. Az egyenlet ekkor a

$$y^3 + py + q = 0$$

alakra redukálódik, ahol

$$p = \frac{3b - a^2}{3}, \qquad q = \frac{2a^3}{27} - \frac{ab}{3} + c.$$

Ha a $D = (p/3)^3 + (q/2)^2$ diszkrimináns pozitív, akkor csak egy valós gyök létezik, amelyet az

$$x = -\frac{a}{3} + \sqrt[3]{-\frac{q}{2} + \sqrt{D}} + \sqrt[3]{-\frac{q}{2} - \sqrt{D}}$$

képlet ad meg. Írj valós aritmetikát használó rutint ezzel a módszerrel (ügyelve arra, hogy az alkalmazott köbgyök valós legyen) egy valós együtthatós harmadfokú egyenlet valós gyökének kiszámítására abban az esetben, amikor csak egy gyök valós. Ha különösen ambiciózus vagy, kiszámíthatod a másik két gyököt is erre az esetre, vagy foglalkozhatsz a három valós gyök esetével, de ehhez komplex aritmetikára (vagy bonyolultabb trigonometrikus megoldásra) lesz szükség. Tedd a rutint a lehető legrobusztusabbá, védekezve a felesleges túlcsordulás, alulcsordulás és a számjegykioltás ellen. Mit kell tennie a rutinnak, ha a vezető együttható nulla? Teszteld a rutint különféle együtthatóértékekre, az előző feladathoz hasonlóan. Lásd még az 5.17. számítógépes feladatot. - **1.12.** (a) Írj programot, amely egy véges x_i sorozat $\bar{x}$ átlagát és σ szórását számítja. A program bemenetként egy n dimenziós x vektort fogadjon el, kimenetként pedig a sorozat átlagát és szórását adja meg. A szórásra próbáld ki mind a kétmenetes

$$\sigma = \left[\frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2 \right]^{1/2}$$

képletet, mind az egymenetes

$$\sigma = \left[\frac{1}{n-1} \left(\sum_{i=1}^{n} x_i^2 - n\bar{x}^2 \right) \right]^{1/2}$$

képletet, és hasonlítsd össze az eredményeket egy általad választott bemeneti sorozaton. (b) Ki tudsz találni olyan bemeneti adatsorozatot, amely drámaian szemlélteti a két matematikailag ekvivalens képlet numerikus különbségét? (*Figyelem:* Óvakodj

egy negatív szám négyzetgyökének vételétől.) - **1.13.** Ha egy a összeget r kamatlábbal fektetnek be, és a kamatot évente n-szer tőkésítik, akkor az egy év végén kapott végérték

$$f = a(1 + r/n)^n.$$

Ez a kamatos kamat ismerős képlete. Egyszerű kamat esetén $n = 1$. Jellemzően negyedévente ($n = 4$), vagy akár naponta ($n = 365$) tőkésítenek. Nyilvánvaló, hogy minél gyakoribb a tőkésítés, annál nagyobb a végösszeg, hiszen a korábbi kamatra is kamatot kapunk. De mekkora különbséget okoz valójában ez a gyakoriság? Írj programot, amely a kamatos kamat képletet valósítja meg. Teszteld a programot $a = 100$ kezdeti befektetéssel, $r = 0{,}05$ (5 százalékos) kamatlábbal, és a következő n értékekkel: 1, 4, 12, 365. Kísérletezz azzal is, mi történik, ha n nagyon nagy lesz. Találsz-e olyan értéket, amelyre a végösszeg nem nő a tőkésítés gyakoriságával, ahogy kellene? (Ez könnyű, ha egyszeres pontosságot használsz, de jóval nehezebb, ha duplát.)

Implementáld a kamatos kamat képletet két különböző módon: (a) Ha a használt programnyelvben nincs hatványozási operátor (pl. C), akkor a képletet egy olyan ciklussal implementálhatod, amely a-t ismételten szorozza $(1 + r/n)$-nel, összesen n-szer. Még ha a programnyelved rendelkezik is hatványozási operátorral (pl. Fortran), implementáld a képletet egy ilyen ciklussal, és írd ki az eredményeket a bemeneti értékekre. (b) Az `exp(x)` és `log(x)` függvényekkel a kamatos kamat képlet a

$$f = a \exp\bigl(n \log(1 + r/n)\bigr)$$

alakban is írható. Implementáld ezt a képletet a megfelelő beépített függvények használatával, és hasonlítsd össze az eredményeidet az első – ciklust használó – implementáció eredményeivel ugyanazon bemeneti értékekre. - **1.14.** Az $(x - 1)^6$ polinom $x = 1$-nél nulla, máshol pozitív. A polinom kifejtett alakja, $x^6 - 6x^5 + 15x^4 - 20x^3 + 15x^2 - 6x + 1$, matematikailag ekvivalens, de numerikusan nem feltétlenül adja ugyanazt az eredményt. Számítsd ki és ábrázold e polinom értékeit mindkét forma használatával a $[0{,}995, 1{,}005]$ intervallum 101 egyenletesen elhelyezett pontjában – azaz $0{,}0001$ lépésközzel. A diagramot úgy skálázd, hogy mind az x, mind a polinom értékei a saját tengelyük teljes tartományát kihasználják. Meg tudod magyarázni ezt a viselkedést? - **1.15.** Egy n-dimenziós x vektor euklideszi normáját az

$$\|x\|_2 = \left(\sum_{i=1}^{n} x_i^2 \right)^{1/2}$$

képlet definiálja. Implementálj robusztus rutint e mennyiség kiszámítására egy tetszőleges bemeneti x vektorra. A rutinnak el kell kerülnie a túlcsordulást és a káros alulcsordulást. Hasonlítsd össze a robusztus rutin pontosságát és teljesítményét egy egyszerűbb, naiv implementációval. Ki tudsz találni olyan vektort, amely jelentősen eltérő eredményeket ad a két rutintól? Mennyi teljesítményt áldoz fel a robusztus rutin? - **1.16.** Írj programot, amely n véletlen, egyszeres pontosságú lebegőpontos számot – x_i, a $[0, 1]$ intervallumon egyenletes eloszlású – összegez (lásd a 13.1. táblázatot

egy megfelelő véletlenszám-generátorhoz). A számokat az alábbi módokon összegezd (csak egyszeres pontosságú lebegőpontos változókat használj, hacsak másképp nem jelezzük): (a) Összegezd a számokat a generálás sorrendjében, egy dupla pontosságú változót használva az összeg gyűjtésére. (b) Összegezd a számokat a generálás sorrendjében, ezúttal egy egyszeres pontosságú gyűjtőt használva. (c) Használd a következő (Kahan-féle) kompenzált összegzési algoritmust – szintén csak egyszeres pontosságot használva –, a számokat a generálás sorrendjében összegezve:

$$
\begin{aligned}
& s = x_1 \\
& c = 0 \\
& \textbf{for } i = 2 \text{ to } n \\
& \quad y = x_i - c \\
& \quad t = s + y \\
& \quad c = (t - s) - y \\
& \quad s = t \\
& \textbf{end}
\end{aligned}
$$

(d) Összegezd a számokat növekvő abszolút érték szerinti sorrendben (ehhez a számokat összegzés előtt rendezni kell, amihez használhatsz könyvtári rendezőrutint).

(e) Összegezd a számokat csökkenő abszolút érték szerinti sorrendben (azaz az összegzés sorrendjét fordítsd meg a (d)-beli sorrendhez képest).

Futtasd a programot különböző n értékekre, és hasonlítsd össze a (a)–(e) módszerek eredményeit. Lényeges eltérés megfigyeléséhez elég nagy n értéket érdemes használnod. Hogyan rangsorolhatók a módszerek pontosság szerint, és miért? Hogyan viszonyulnak egymáshoz a módszerek költségben? Meg tudod magyarázni, miért működik a (c)-beli algoritmus? - **1.17.** Írj programot, amely az

$$x_{k+1} = 2{,}25\, x_k - 0{,}5\, x_{k-1}$$

differenciaegyenlet első n tagját állítja elő, kezdőértékek

$$x_1 = \frac{1}{3} \quad \text{és} \quad x_2 = \frac{1}{12}.$$

Használd $n = 25$-öt egyszeres, vagy $n = 60$-at dupla pontosság esetén. Készíts féllogaritmikus ábrát a kapott értékekről k függvényében. A differenciaegyenlet egzakt megoldása

$$x_k = \frac{4^{1-k}}{3},$$

amely k növekedésével szigorúan monoton csökken. Igazolja-e az ábrád ezt az elméletileg várt viselkedést? Meg tudod magyarázni az eredményeidet? (*Útmutatás:* Keresd meg a differenciaegyenlet általános megoldását.) - **1.18.** Írj programot, amely az

$$x_{k+1} = 111 - (1130 - 3000/x_{k-1})/x_k$$

differenciaegyenlet első n tagját állítja elő, kezdőértékek

$$x_1 = \frac{11}{2} \quad \text{és} \quad x_2 = \frac{61}{11}.$$

Használd $n = 10$-et egyszeres, vagy $n = 20$-at dupla pontosság esetén. Az egzakt megoldás egy szigorúan monoton növekvő, 6-hoz konvergáló sorozat. Meg tudod magyarázni az eredményeidet?

Lineáris egyenletrendszerek

2.1 Lineáris rendszerek

A természetben sok összefüggés lineáris, vagyis a hatások az okaikkal arányosak. A mechanikában például Newton második törvénye, $F = ma$, azt mondja ki, hogy az erő arányos a gyorsulással, ahol az arányossági tényező a tárgy tömege. Ha ismerjük az erőt és a tömeget, akkor ezt a lineáris egyenletet a gyorsulásra nézve meg tudjuk oldani. Az elektromosságtanban az Ohm-törvény, $V = iR$, kimondja, hogy egy vezető két vége közötti feszültség (potenciálkülönbség) arányos a rajta átfolyó árammal, ahol az arányossági tényező az ellenállás. Ismét, ha ismerjük a feszültséget és az ellenállást, meg tudjuk oldani az egyenletet az áramra nézve. A rugalmasságtanban a Hooke-törvény szerint a mechanikai feszültség arányos a deformációval, ahol az arányossági tényező a Young-modulus.

Magasabb dimenziókban az ilyen lineáris kapcsolatokat egy $\mathcal{L}$ lineáris transzformáció fejezi ki, amely egy „okokat” tartalmazó u vektort egy „hatásokat” tartalmazó f vektorhoz rendel:

$$\mathcal{L}u = f.$$

Az Ohm-törvény és a Kirchhoff-törvények például lehetővé teszik, hogy felírjunk egy egész egyenletrendszert, amely egy sok vezetőből álló áramkörben a feszültségek és az áramok közötti lineáris kapcsolatot fejezi ki (lásd a 2.1. példát). Ahogy az elemi lineáris algebrából megtanultuk, két végesdimenziós vektortér közötti lineáris transzformációt kényelmesen egy mátrixszal ábrázolhatunk. Mátrix-vektor jelöléssel egy lineáris algebrai egyenletrendszer alakja

$$Ax = b,$$

ahol A egy ismert $m \times n$-es mátrix, b egy m-dimenziós vektor, x pedig egy n-dimenziós vektor. Ha ismerjük x-et, akkor egy ilyen lineáris kapcsolat lehetővé teszi, hogy a mátrix-vektor szorzás elvégzésével meghatározzuk a b hatást az x ok alapján: $b = Ax$. Gyakrabban azonban a lineáris egyenletrendszerek lehetőséget adnak a „visszafejtésre”: ha ismerjük a hatások b vektorát, szeretnénk meghatározni a hozzá tartozó okok

x vektorát. E fejezet tárgyát azok a numerikus módszerek képezik, amelyek ezt lehetővé teszik.

Még ha az összefüggések nemlineárisak is, gyakran jól közelíthetők lokálisan (azaz egy adott rögzített érték környezetében) egy lineáris modellel. Pontosan ezt teszi az analízisben a derivált: egy nemlineáris görbéhez lokális lineáris közelítést – az érintőegyenest – rendel. Ez a megfigyelés számos nemlineáris algebrai feladat numerikus megoldási módszerének is az alapja. Még a nem algebrai feladatokat is – például a differenciál- vagy integrálegyenleteket – végső soron lineáris algebrai egyenletrendszerekkel közelíthetjük, ahogyan azt a könyv későbbi részeiben látni fogjuk. Ezért a lineáris egyenletrendszerek megoldása számos gyakorlati számítási feladat numerikus algoritmusának sarokköve, és ennek megfelelően elengedhetetlen, hogy az ilyen rendszereket pontosan és hatékonyan tudjuk megoldani.

Az $Ax = b$ lineáris egyenletrendszer lényegében azt a kérdést teszi fel: „kifejezhető-e a b vektor az A mátrix oszlopainak lineáris kombinációjaként?" (azaz „benne van-e b a $\operatorname{span}(A) = \{Ax : x \in \mathbb{R}^n\}$ oszloptérben?"). Ha igen, akkor a rendszert *megoldhatónak (konzisztensnek)* mondjuk, és a lineáris kombináció együtthatóit az x megoldásvektor komponensei adják. Előfordulhat, hogy van megoldás, és az is, hogy nincs; ha van megoldás, az lehet egyértelmű, de lehetséges végtelen sok megoldás is. Ebben a fejezetben csak *négyzetes* rendszereket tekintünk, azaz $m = n$ – a mátrix sorainak és oszlopainak száma megegyezik, vagyis az egyenletek száma (A és b sorai) megegyezik az ismeretlenek számával (x komponensei). Az $m \neq n$ esetet a 3. fejezetben tárgyaljuk. Az egyszerűség kedvéért figyelmünket elsősorban valós lineáris rendszerekre fordítjuk; a komplex rendszereket nagyon hasonlóan lehet kezelni.

2.1. Példa. Elektromos áramkör. Tekintsük a 2.1. ábrán látható elektromos áramkört. Adottak a V feszültségek és az R ellenállások, és azt szeretnénk meghatározni, milyen i áramok folynak az áramkör három hurkában. Ehhez az alábbi fizikai törvényeket alkalmazzuk:

- **Ohm-törvény:** egy R ellenálláson egy i áram irányában eső feszültségesés iR.
- **Kirchhoff huroktörvénye:** egy zárt hurokban a feszültségesések (és forrásfeszültségek) algebrai összege nulla.

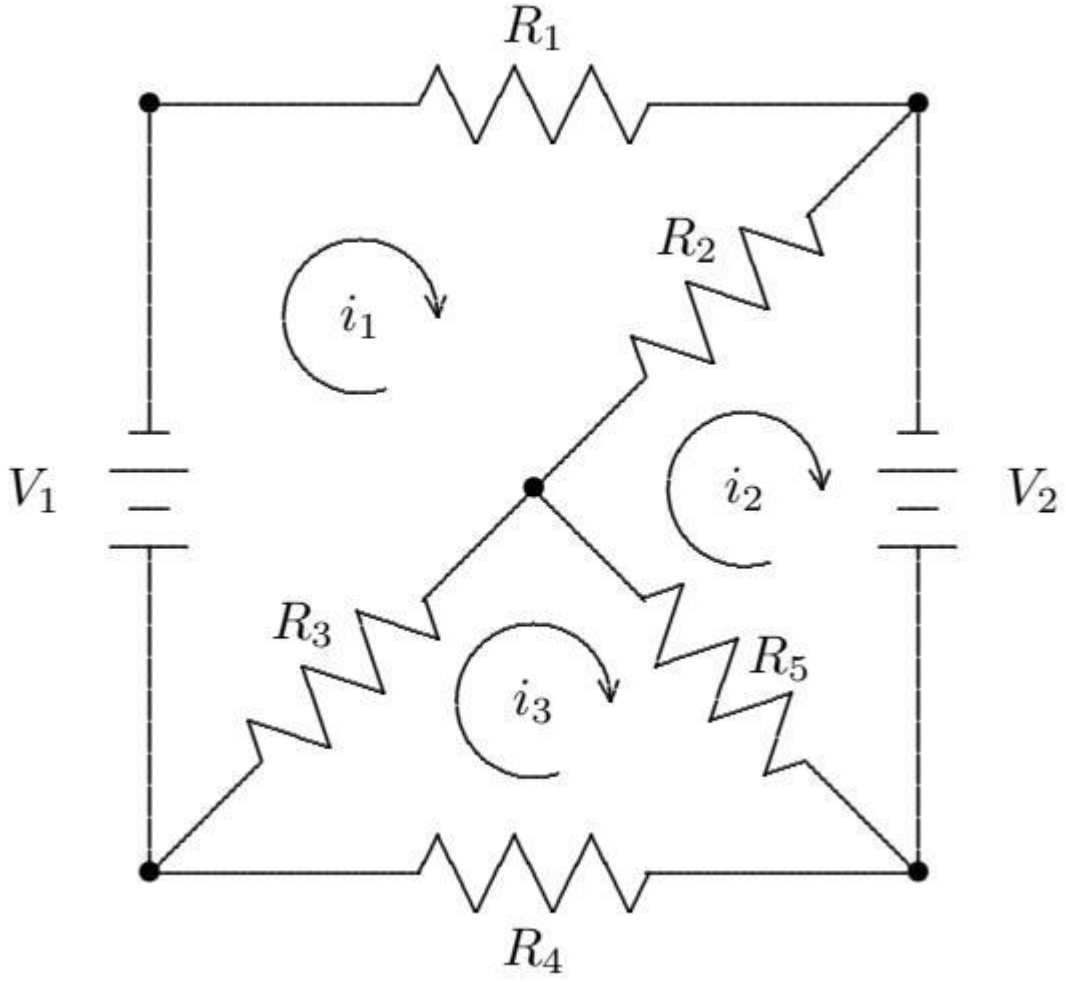
Ezeket a törvényeket az áramkör minden hurkára alkalmazva a következő három lineáris egyenletből álló rendszert kapjuk:

$$\begin{aligned} i_1R_1 + (i_1 - i_2)R_2 + (i_1 - i_3)R_3 + V_1 &= 0, \\ (i_2 - i_1)R_2 + (i_2 - i_3)R_5 - V_2 &= 0, \\ (i_3 - i_1)R_3 + i_3R_4 + (i_3 - i_2)R_5 &= 0, \end{aligned}$$

amely mátrixalakban így írható:

$$\begin{bmatrix} R_1 + R_2 + R_3 & -R_2 & -R_3 \\ -R_2 & R_2 + R_5 & -R_5 \\ -R_3 & -R_5 & R_3 + R_4 + R_5 \end{bmatrix} \begin{bmatrix} i_1 \\ i_2 \\ i_3 \end{bmatrix} = \begin{bmatrix} -V_1 \\ V_2 \\ 0 \end{bmatrix}.$$

Így ez a feladat $Ax = b$ alakú, és megoldható az e fejezetben tanulmányozott módszerekkel.



2.1. ábra: Ellenállásokkal és feszültségforrásokkal felépített elektromos áramkör.

2.2 Létezés és egyértelműség

Egy $n \times n$-es A mátrixot *nemszinguláris*nak nevezünk, ha az alábbi – egymással ekvivalens – feltételek bármelyike teljesül rá:

1. A-nak van inverze (azaz létezik egy olyan, A^{-1}-gyel jelölt $n \times n$-es mátrix, amelyre $AA^{-1} = A^{-1}A = I$, az egységmátrix).
2. $\det(A) \neq 0$ (azaz A determinánsa nem nulla).
3. $\operatorname{rank}(A) = n$ (egy mátrix rangja a benne található lineárisan független sorok vagy oszlopok maximális száma).
4. Bármely $z \neq 0$ vektorra $Az \neq 0$ (azaz A egyetlen nemtriviális vektort sem annullál).

Minden más esetben a mátrix *szinguláris*. Az $Ax = b$ lineáris egyenletrendszer meg-

oldásának létezése és egyértelműsége attól függ, hogy az A mátrix szinguláris vagy nemszinguláris. Ha A nemszinguláris, akkor létezik az inverze (A^{-1}), és az $Ax = b$ rendszernek mindig pontosan egy $x = A^{-1}b$ megoldása van, a b értékétől függetlenül. Ha viszont A szinguláris, akkor a megoldások száma a b jobb oldali vektortól függ: adott b mellett előfordulhat, hogy nincs megoldás, de ha létezik olyan x megoldás, amelyre $Ax = b$, akkor az $A(x + \gamma z) = b$ is teljesül bármely γ skalárra, ahol $z \neq 0$ olyan vektor, amelyre $Az = 0$ (ilyen z-nek léteznie kell, hiszen ellenkező esetben a definíció 4. feltétele szerint a mátrix nemszinguláris volna). Ezért egy négyzetes, konzisztens és szinguláris lineáris rendszer megoldása sosem lehet egyértelmű. Adott négyzetes A mátrixra és b jobb oldali vektorra a lehetőségeket az alábbiak foglalják össze:

- **Egyértelmű megoldás:** A nemszinguláris, b tetszőleges.
- **Végtelen sok megoldás:** A szinguláris, $b \in \operatorname{span}(A)$ (konzisztens).
- **Nincs megoldás:** A szinguláris, $b \notin \operatorname{span}(A)$ (inkonzisztens).

Két dimenzióban a rendszer minden lineáris egyenlete egy egyenest határoz meg a síkban. A rendszer megoldása a két egyenes metszéspontja. Ha a két egyenes nem párhuzamos, akkor pontosan egy metszéspontjuk van (a nemszinguláris eset). Ha a két egyenes párhuzamos, akkor vagy egyáltalán nem metszik egymást (nincs megoldás), vagy a két egyenes egybeesik (az egyenes bármely pontja megoldás). Magasabb dimenziókban minden egyenlet egy hipersíkot határoz meg. A nemszinguláris esetben az egyértelmű megoldás az összes hipersík metszéspontja.

2.2. Példa. Szingularitás és nemszingularitás. A 2×2-es

$$\begin{aligned} 2x_1 + 3x_2 &= b_1, \\ 5x_1 + 4x_2 &= b_2, \end{aligned}$$

rendszer – vagy mátrix-vektor jelöléssel:

$$Ax = \begin{bmatrix} 2 & 3 \\ 5 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = b,$$

– a b értékétől függetlenül egyértelmű megoldással rendelkezik, mert az A nemszinguláris. Például ha $b = [8 \quad 13]^T$, akkor az egyértelmű megoldás $x = [1 \quad 2]^T$.

Viszont a 2×2-es

$$Ax = \begin{bmatrix} 2 & 3 \\ 4 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = b$$

rendszerben az A mátrix szinguláris, ezért a b konkrét értékétől függően előfordulhat, hogy van megoldás, de az is, hogy nincs, egyértelmű megoldása viszont semmiképpen sem lehet. Például ha $b = [4 \quad 7]^T$, akkor nincs megoldás, ha viszont $b = [4 \quad 8]^T$, akkor az

$$x = \begin{bmatrix} \gamma \\ (4 - 2\gamma)/3 \end{bmatrix}$$

megoldás bármely γ valós számra.

2.3 Érzékenység és kondicionáltság

Miután megfogalmaztuk az $Ax = b$ lineáris egyenletrendszer megoldásának létezésére és egyértelműségére vonatkozó feltételeket, most az x megoldás érzékenységét vizsgáljuk a bemeneti adatok perturbációival szemben, amelyek ebben a feladatban az A mátrix és a b jobb oldali vektor. Az ilyen perturbációk méréséhez szükségünk van a vektorok és mátrixok „méretének” valamilyen fogalmára. A nagyság, abszolút érték vagy modulus skaláris fogalma általánosítható a vektorok és mátrixok normáinak fogalmára.

2.3.1 Vektornormák

Bár általánosabb definíció is lehetséges, a továbbiakban használt vektornormák mind a p-normák speciális esetei, amelyeket egy $p > 0$ egész szám és egy n-dimenziós x vektor esetén a következőképpen definiálunk:

$$\|x\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}.$$

Fontos speciális esetek:

- 1-norma:

$$\|x\|_1 = \sum_{i=1}^{n} |x_i|,$$

amelyet néha Manhattan-normának is neveznek, mivel két dimenzióban ez két pont „háztömbökben” mért távolságának felel meg.

- 2-norma:

$$\|x\|_2 = \left(\sum_{i=1}^{n} |x_i|^2 \right)^{1/2},$$

amely az euklideszi térben megszokott távolságfogalomnak felel meg, ezért euklideszi normának is nevezzük.

- ∞-norma:

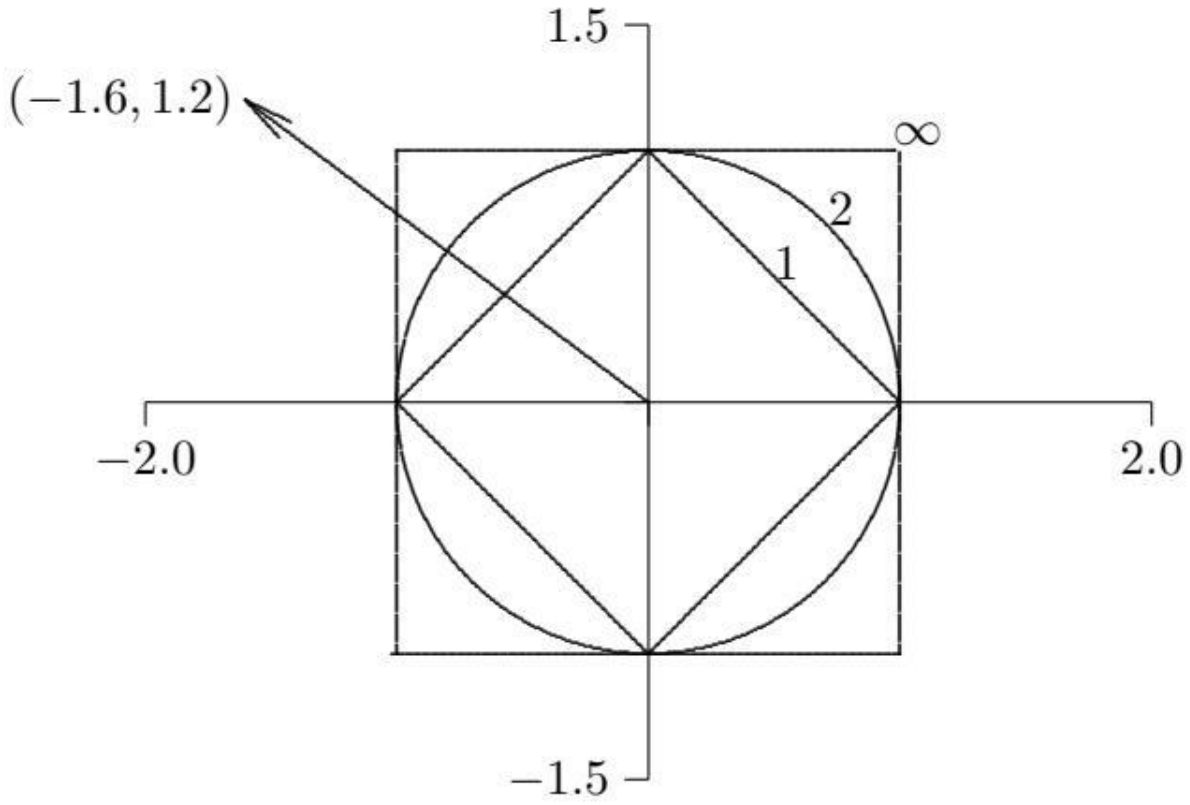
$$\|x\|_\infty = \max_{1\le i\le n} |x_i|,$$

amely $p \to \infty$ határesetként értelmezhető.

Ezek a normák kvalitatívan mind hasonló eredményeket adnak, de bizonyos körülmények között egy adott norma a legkönnyebben kezelhető analitikusan vagy számítástechnikailag. A lineáris egyenletrendszerek megoldásainak érzékenységi vizsgálatához általában az 1-normát vagy az ∞-normát használjuk. A 2-normát később más kontextusokban fogjuk kiterjedten alkalmazni. E normák közötti különbségeket $\mathbb{R}^2$-ben a 2.2. ábra szemlélteti, amely az egységgömböt, azaz az $\{x : \|x\|_p = 1\}$ halmazt mutatja $p = 1, 2, \infty$ esetén. (Megjegyzendő, hogy az egységgömb – amelyet két dimenzióban pontosabb egységkörnek nevezni – csak a 2-normában valóban „kerek”, innen kapta a nevét.) Egy vektor normája egyszerűen az a tényező, amellyel a megfelelő egységgömböt ki kell tágítani vagy össze kell zsugorítani ahhoz, hogy az a vektort magában foglalja.

2.3. Példa. Vektornormák. A 2.2. ábrán látható $x = [-1{,}6, 1{,}2]^T$ vektor esetén

$$\|x\|_1 = 2{,}8, \quad \|x\|_2 = 2{,}0, \quad \|x\|_\infty = 1{,}6.$$



2.2. ábra: Egységgömbök különböző vektornormákban.

Általában tetszőleges n-dimenziós x vektor esetén

$$\|x\|_1 \geq \|x\|_2 \geq \|x\|_\infty.$$

Másrészt fennáll, hogy

$$\|x\|_1 \leq \sqrt{n}\|x\|_2$$

, $\|x\|_2 \leq \sqrt{n}\|x\|_\infty$ és $\|x\|_1 \leq n\|x\|_\infty$.

Így adott n esetén e normák közül bármely kettő legfeljebb egy konstansban tér el egymástól, tehát abban az értelemben mind ekvivalensek, hogy ha az egyik kicsi, akkor valamennyinek arányosan kicsinek kell lennie (valójában minden p-norma ekvivalens ebben az értelemben). Ezért mindig azt a normát választhatjuk, amelyik az adott kontextusban a legkényelmesebb. A könyv hátralévő részében megfelelő alsó index jelöli majd a konkrét normát, amikor ez szükséges, de az alsó indexet elhagyjuk, ha közömbös, hogy melyik normát használjuk.

Minden vektor p-norma rendelkezik a következő fontos tulajdonságokkal, ahol x és y tetszőleges vektorok:

- 1. $\|\mathbf{x}\| > 0$, ha $\mathbf{x} \neq \mathbf{0}$.
- 2. $\|\gamma x\| = |\gamma| \cdot \|x\|$ tetszőleges γ skalár esetén.
- 3. $\|x + y\| \leq \|x\| + \|y\|$ (háromszög-egyenlőtlenség).

Az általánosabb megközelítésben egy vektornorma *definíciójaként* bármely olyan valós értékű függvényt tekinthetünk egy vektoron, amely e három tulajdonságot teljesíti. Megjegyzendő, hogy az első két tulajdonság együtt magában foglalja, hogy $\|x\| = 0$ akkor és csak akkor, ha $x = 0$. A háromszög-egyenlőtlenség hasznos változata

$$|\|x\| - \|y\|| \leq \|x - y\|,$$

amely egyúttal azt is mutatja, hogy a vektornorma folytonos függvény.

2.3.2 Mátrixnormák

A mátrixok méretének vagy nagyságának mérésére is szükségünk van valamilyen módszerre. Itt is lehetséges általánosabb definíció, de a továbbiakban használt mátrixnormákat mind egy alapul szolgáló vektornorma segítségével definiáljuk. Pontosabban: adott vektornorma mellett egy $m \times n$ méretű A mátrixhoz tartozó mátrixnormát a következőképpen értelmezzük:

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}.$$

Az ilyen mátrixnormáról azt mondjuk, hogy a vektornorma által indukált vagy annak szubordinált normája. Szemléletesen: a mátrix normája azt a maximális nyújtást méri, amelyet a mátrix az adott vektornormában mérve bármely vektoron végrehajt.

Egyes mátrixnormák sokkal könnyebben számíthatók ki, mint mások. Például a vektor 1-normához tartozó mátrixnorma egyszerűen a mátrix maximális abszolút oszlopösszege,

$$\|A\|_1 = \max_j \sum_{i=1}^{m} |a_{ij}|,$$

a vektor ∞-normához tartozó mátrixnorma pedig egyszerűen a mátrix maximális abszolút sorösszege,

$$\|\mathbf{A}\|_\infty = \max_i \sum_{j=1}^{n} |a_{ij}|.$$

Ezek megjegyzésének kényelmes módja, hogy ezek a mátrixnormák $n \times 1$ méretű mátrix esetén megegyeznek a megfelelő vektornormákkal. Sajnos a vektor 2-normához tartozó mátrixnormát nem olyan könnyű kiszámítani (lásd 3.6.1. szakasz).

2.4. Példa. Mátrixnormák. Az

$$\mathbf{A} = \begin{bmatrix} 2 & -1 & 1 \\ 1 & 0 & 1 \\ 3 & -1 & 4 \end{bmatrix}$$

mátrix esetén a maximális abszolút oszlop- és sorösszegekből rendre

$$\|A\|_1 = 6$$

és $\|A\|_\infty = 8$ adódik.

E mátrix 2-normáját lásd a 3.17. példában.

A definiált mátrixnormák a következő fontos tulajdonságokkal rendelkeznek, ahol A és B tetszőleges mátrixok:

- 1. $\|A\| > 0$, ha $A \neq O$.
- 2. $\|\gamma A\| = |\gamma| \cdot \|A\|$ tetszőleges γ skalár esetén.
- 3. $\|A + B\| \leq \|A\| + \|B\|$.
- 4. $\|AB\| \leq \|A\| \cdot \|B\|$.
- 5. $\|Ax\| \leq \|A\| \cdot \|x\|$ tetszőleges x vektor esetén.

Az általánosabb megközelítésben egy mátrixnorma definíciójaként bármely olyan valós értékű mátrixfüggvényt tekinthetünk, amely az első három tulajdonságot teljesíti. A fennmaradó két tulajdonság, amelyeket szubmultiplikatív vagy konzisztenciafeltételeknek neveznek, az ilyen általánosabb mátrixnormákra nem feltétlenül teljesül, de a vektor p-normák által indukált mátrixnormákra mindig érvényesek. Ismét megjegyezzük, hogy az első két tulajdonság együtt azt adja, hogy $\|A\| = 0$ akkor és csak akkor, ha $A = O$.

2.3.3 Mátrix kondíciószáma

Egy nemszinguláris A négyzetes mátrix adott mátrixnormára vonatkozó kondíciószámát a következőképpen definiáljuk:

$$\operatorname{cond}(A) = \|A\| \cdot \|A^{-1}\|.$$

Megállapodás szerint $\operatorname{cond}(A) = \infty$, ha A szinguláris. A 2.3.4. szakaszban látni fogjuk, hogy ez a fogalom összhangban van a kondíciószám 1.2.6. szakaszban bevezetett általános fogalmával annyiban, hogy a mátrix kondíciószáma felső korlátot ad a lineáris egyenletrendszer megoldásának relatív változásának és a bemeneti adatok adott relatív változásának arányára.

2.5. Példa. Mátrix kondíciószáma. Mátrixszorzással könnyen ellenőrizhető, hogy a 2.4. példa mátrixának inverze

$$\mathbf{A}^{-1} = \begin{bmatrix} 0{,}5 & 1{,}5 & -0{,}5 \\ -0{,}5 & 2{,}5 & -0{,}5 \\ -0{,}5 & -0{,}5 & 0{,}5 \end{bmatrix},$$

így

$$\|\mathbf{A}^{-1}\|_1 = 4{,}5$$

és $\|\mathbf{A}^{-1}\|_\infty = 3{,}5$.

Ebből tehát

$$\operatorname{cond}_1(\mathbf{A}) = \|\mathbf{A}\|_1 \cdot \|\mathbf{A}^{-1}\|_1 = 6 \cdot 4{,}5 = 27$$

és

$$\operatorname{cond}_\infty(\mathbf{A}) = \|\mathbf{A}\|_\infty \cdot \|\mathbf{A}^{-1}\|_\infty = 8 \cdot 3{,}5 = 28.$$

E mátrix 2-normára vonatkozó kondíciószámát lásd a 3.17. példában.

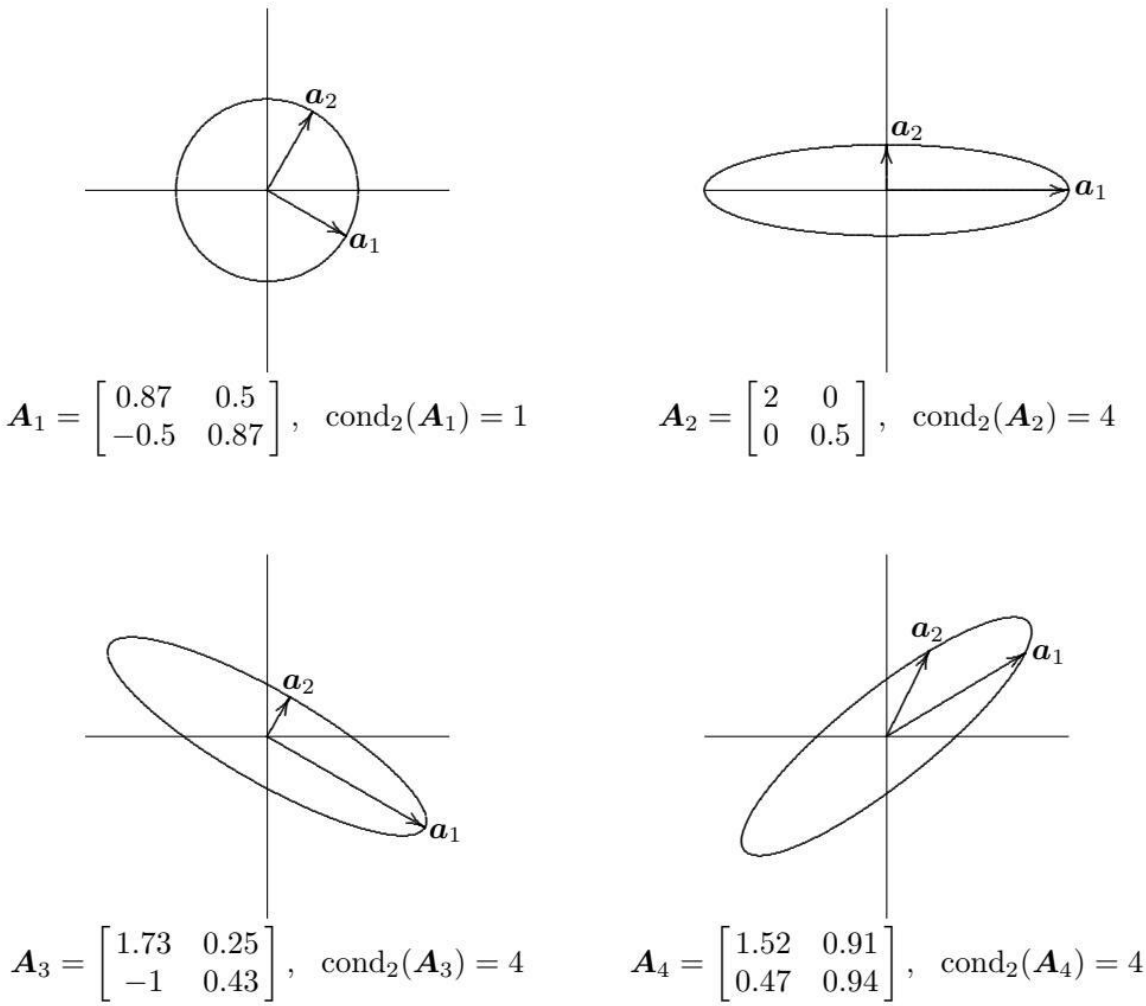
A 2.5. példából látható, hogy egy $n \times n$ méretű mátrix kondíciószámának számértéke függ az alkalmazott normától (amit a megfelelő alsó index jelez), de az alapul szolgáló vektornormák ekvivalenciája miatt ezek az értékek legfeljebb egy rögzített konstansban térhetnek el (amely n-től függ), így a kondicionáltság kvantitatív mértékeként egyaránt hasznosak.

Mivel

$$\|A\| \cdot \|A^{-1}\| = \left(\max_{x \neq 0} \frac{\|Ax\|}{\|x\|} \right) \cdot \left(\min_{x \neq 0} \frac{\|Ax\|}{\|x\|} \right)^{-1},$$

egy mátrix kondíciószáma a maximális relatív nyújtás és a maximális relatív zsugorítás arányát méri, amelyet a mátrix bármely nem nulla vektoron végrehajt. Ezt úgy is megfogalmazhatjuk, hogy a mátrix kondíciószáma az egységgömb torzulásának mértékét méri (a megfelelő vektornormában) a mátrix által való transzformáció során. Minél nagyobb a kondíciószám, annál torzabb (relatíve nyúlt és vékony) lesz az egységgömb a mátrix általi transzformáció után. Két dimenzióban például a 2-normabeli egységkörből egyre inkább szivar alakú ellipszis lesz, az 1-normában vagy az ∞-normában pedig az egységgömb négyzetből egyre ferdébb paralelogrammává torzul, ahogy a kondíciószám nő.

2.6. Példa. Mátrix kondíciószáma. A 2.3. ábra négy különböző mátrix hatását szemlélteti az $\mathbb{R}^2$-beli egységkörön a 2-normát használva. A_1 az egységkört 30 fokkal az óramutató járásával megegyező irányba forgatja el, de egyik vektor euklideszi hosszát sem változtatja meg, így $\mathrm{cond}_2(A_1) = 1$. A_2 az e_1 bázisvektort 2-szeresére nyújtja, az e_2 bázisvektort pedig 0,5-szeresére zsugorítja, és mindkettő maximális, tehát $\mathrm{cond}_2(A_2) = 2/0{,}5 = 4$. A_3 egyszerre forgat és torzít az egységkörön, de a maximális arány itt is ugyanaz, mint A_2 esetén, így $\mathrm{cond}_2(A_3) = 4$. Végül A_4 egy általánosabb transzformáció, amely esetén a maximális arány már nem a bázisvektoroknál jelentkezik, de a maximum értéke továbbra is ugyanaz, tehát $\mathrm{cond}_2(A_4) = 4$.



2.3. ábra: Az egységkör transzformációja 2-normában különböző mátrixokkal.

A kondíciószám következő fontos tulajdonságai a definícióból könnyen levezethetők, és tetszőleges normára érvényesek:

- 1. Tetszőleges A mátrix esetén $\mathrm{cond}(A) \geq 1$.
- 2. Az egységmátrixra $\mathrm{cond}(I) = 1$.
- 3. Tetszőleges A mátrix és nem nulla γ skalár esetén $\mathrm{cond}(\gamma A) = \mathrm{cond}(A)$.
- 4. Tetszőleges $D = \mathrm{diag}(d_i)$ diagonális mátrix esetén $\mathrm{cond}(D) = (\max |d_i|)/(\min |d_i|)$.

A kondíciószám azt méri, hogy a mátrix mennyire van közel a szingularitáshoz: egy nagy kondíciószámú mátrix (amit a 2.3.4. szakaszban fogunk kvantifikálni) közel szinguláris, míg az 1-hez közeli kondíciószámú mátrix messze van a szingularitástól. A definícióból nyilvánvaló, hogy egy nemszinguláris mátrix és inverze kondíciószáma megegyezik. Ez érthető, hiszen ha egy mátrix közel szinguláris, akkor az inverze is ugyanolyan közel van a szingularitáshoz.

Megjegyzendő, hogy a mátrix determinánsa nem jó indikátora a szingularitáshoz való közelségnek: bár egy A mátrix szinguláris, ha $\det(A) = 0$, a nem nulla determináns

nagysága – akár nagy, akár kicsi – semmilyen információt nem ad arról, hogy a mátrix mennyire közelít a szingularitáshoz. Például $\det(\alpha I_n) = \alpha^n$, ami $|\alpha| < 1$ esetén tetszőlegesen kicsi lehet, ennek ellenére a mátrix tetszőleges nem nulla α esetén tökéletesen jól kondicionált, kondíciószáma bármely mátrixnormában 1.

Ahogy nemsokára látni fogjuk, a kondíciószám hasznossága a lineáris egyenletrendszerek megoldásának pontosságának megítélésében rejlik. A kondíciószám definíciója a mátrix inverzét tartalmazza, így értékének kiszámítása nyilvánvalóan nem triviális feladat. Valójában a kondíciószámot közvetlenül a definícióból kiszámítani lényegesen nagyobb munkát igényelne, mint annak a megoldásnak a kiszámítása, amelynek pontosságát a kondíciószám segítségével kívánjuk megítélni. A gyakorlatban ezért a kondíciószámot csupán becsüljük, legfeljebb egy nagyságrend pontossággal, a megoldási folyamat viszonylag olcsó melléktermékeként.

Az $\|A\|$ mátrixnormát könnyen kiszámíthatjuk maximális abszolút oszlopösszegként (vagy sorösszegként, a használt normától függően). Az $\|A^{-1}\|$ alacsony költségű becslése jelenti a kihívást. A normák tulajdonságaiból tudjuk, hogy ha z az $Az = y$ rendszer megoldása, akkor

$$\|z\| = \|A^{-1}y\| \leq \|A^{-1}\| \cdot \|y\|,$$

így

$$\frac{\|z\|}{\|y\|} \leq \|A^{-1}\|,$$

és ez a korlát valamely optimálisan választott y vektor esetén elérhető. Ezért ha y-t úgy tudjuk megválasztani, hogy az $\|z\|/\|y\|$ hányados a lehető legnagyobb legyen, akkor ésszerű becslést kapunk $\|A^{-1}\|$-re.

2.7. Példa. Kondícióbecslés. Tekintsük az

$$\mathbf{A} = \begin{bmatrix} 0{,}913 & 0{,}659 \\ 0{,}457 & 0{,}330 \end{bmatrix}$$

mátrixot. Ha $y = [0, 1{,}5]^T$-t választjuk, akkor $z = [-7780, 10780]^T$, tehát

$$\|A^{-1}\|_1 \approx \frac{\|z\|_1}{\|y\|_1} \approx 1{,}238 \times 10^4,$$

és így

$$\mathrm{cond}_1(\mathbf{A}) = \|\mathbf{A}\|_1 \cdot \|\mathbf{A}^{-1}\|_1 \approx 1{,}370 \times 1{,}238 \times 10^4 = 1{,}696 \times 10^4,$$

ami – mint kiderül – a megjelenített számjegyek pontosságával egzakt. E viszonylag nagy kondíciószám következményeit a 2.8. és a 2.17. példában vizsgáljuk.

A 2.7. példabeli y vektort gondosan úgy választottuk meg, hogy a lehető legnagyobb $\|z\|/\|y\|$ hányadost adja, és így $\|A^{-1}\|$ pontos értékét eredményezze. Egy ilyen optimális y megtalálása általában megengedhetetlenül költséges lenne, de jóval olcsóbban is kaphatunk hasznos közelítést. Az egyik heurisztika az, hogy y-t az $A^T y = c$ rendszer megoldásaként választjuk, ahol c egy olyan vektor, amelynek komponensei ± 1 értékűek; az előjeleket egymás után úgy választjuk meg, hogy az adódó y a lehető legnagyobb legyen. Egy másik stratégia egyszerűen az, hogy néhány véletlen y-t próbálunk; a közöttük legnagyobb hányados megadása gyakorlati célokra általában elegendően pontos becslést ad $\|A^{-1}\|$-re.

A kondícióbecslés alternatív megközelítése, hogy konvex optimalizálási feladatként kezeljük, amely heurisztikus algoritmussal a gyakorlatban nagyon hatékonyan megoldható. További lehetőség – a 2-norma használatakor – a kondíciószámot a szingulárisérték-felbontásból származtatni (lásd 3.6.1. szakasz), ez azonban megengedhetetlenül költséges, hacsak az SVD-t egyéb okból amúgy sem számítjuk ki. Szerencsére a felhasználóknak nem kell aggódniuk e részletek miatt, mivel a lineáris egyenletrendszerek megoldására szolgáló jó modern szoftvercsomagok többsége hatékony és megbízható kondícióbecslőt biztosít, amely az itt vázolt módszerek valamelyikének kifinomult megvalósításán alapul (lásd 2.1. táblázat).

2.3.4 Hibakorlátok

A kondíciószám nemcsak a szingularitáshoz való közelség megbízható indikátora, hanem kvantitatív korlátot is szolgáltat a lineáris egyenletrendszer számított megoldásának hibájára, ahogy azt most megmutatjuk. Legyen x a nemszinguláris $Ax = b$ lineáris egyenletrendszer megoldása, és legyen $\hat{x}$ az $A\hat{x} = b + \Delta b$ perturbált jobb oldalú rendszer megoldása. Ha definiáljuk $\Delta x = \hat{x} - x$-et, akkor

$$A\hat{x} = A(x + \Delta x) = Ax + A\,\Delta x = b + \Delta b.$$

Mivel $Ax = b$, ezért $A\,\Delta x = \Delta b$-nek kell teljesülnie, így $\Delta x = A^{-1}\Delta b$. Normákat véve kapjuk az

$$\|b\| = \|Ax\| \leq \|A\| \cdot \|x\|, \quad \text{vagyis} \quad \|x\| \geq \|b\|/\|A\|,$$

és

$$\|\Delta x\| = \|A^{-1}\Delta b\| \leq \|A^{-1}\| \cdot \|\Delta b\|$$

egyenlőtlenségeket. E két egyenlőtlenséget összekapcsolva adódik, hogy

$$\frac{\|\Delta x\|}{\|x\|} \leq \|A^{-1}\| \cdot \|\Delta b\| \frac{\|A\|}{\|b\|}.$$

Definíció szerint $\|A\| \cdot \|A^{-1}\| = \mathrm{cond}(A)$, így a következő korlátot kapjuk:

$$\frac{\|\Delta x\|}{\|x\|} \leq \text{cond}(A)\frac{\|\Delta b\|}{\|b\|}.$$

Így a mátrix kondíciószáma egy „erősítési tényező”, amely a jobb oldali vektor adott relatív változásából fakadó megoldás maximális relatív változását korlátozza (vö. a kondíciószám 1.2.6. szakaszban megadott általános fogalmával).

Hasonló eredmény érvényes az A mátrix elemeinek relatív változásaira is. Ha $Ax = b$ és $(A + E)\hat{x} = b$, akkor

$$\Delta x = \hat{x} - x = A^{-1}(A\hat{x} - b) = -A^{-1}E\hat{x}.$$

Normákat véve az

$$\|\Delta x\| \leq \|A^{-1}\| \cdot \|E\| \cdot \|\hat{x}\|$$

egyenlőtlenséget kapjuk, amelyből $\text{cond}(A)$ definíciójával adódik a

$$\frac{\|\Delta x\|}{\|\hat{x}\|} \leq \text{cond}(A)\frac{\|E\|}{\|A\|}$$

korlát.

Az imént ismertetett algebrai levezetések alternatívájaként a lineáris egyenletrendszerek érzékenységét a differenciálszámítás segítségével is becsülhetjük. A t valós paraméter bevezetésével definiáljuk az $A(t) = A + tE$ és $b(t) = b + t\Delta b$ mennyiségeket, és tekintjük az $A(t)x(t) = b(t)$ lineáris egyenletrendszer $x(t)$ megoldását. Ezt az egyenletet t szerint differenciálva

$$A^{'}(t)x(t) + A(t)x'(t) = b^{'}(t),$$

vagy ekvivalens módon

$$Ex(t) + A(t)x'(t) = \Delta b$$

adódik. Átrendezve kapjuk, hogy

$$x'(t) = A(t)^{-1}(\Delta b - Ex(t)),$$

amiből

$$x'(0) = A^{-1}(\Delta b - Ex(0)).$$

A Taylor-tétel szerint $x(t) = x(0) + tx'(0) + \mathcal{O}(t^2)$, tehát

$$x(t) - x(0) = tx'(0) + \mathcal{O}(t^2) = tA^{-1}(\Delta b - Ex(0)) + \mathcal{O}(t^2).$$

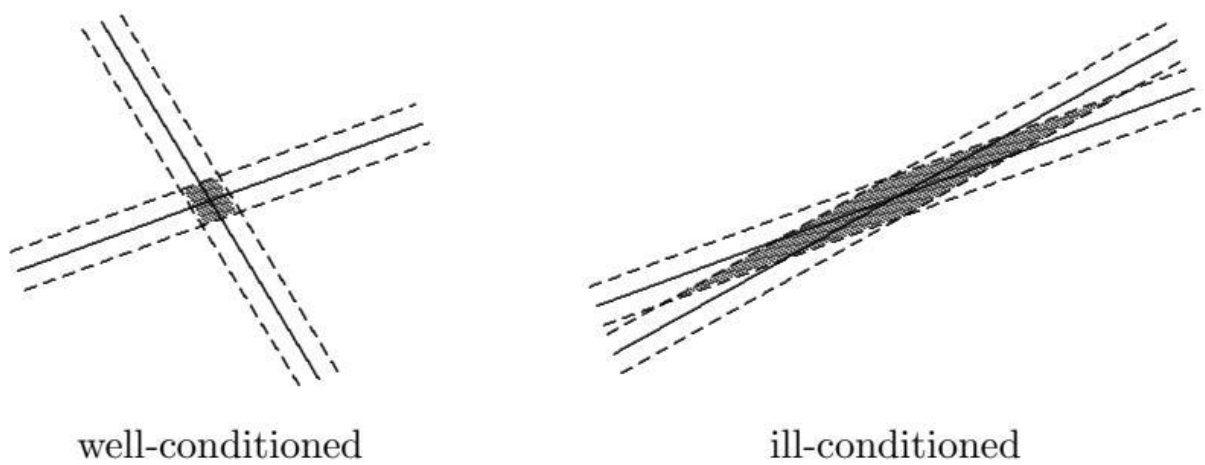
Legyen $x \equiv x(0)$; normákat véve, majd $\|x\|$-szel osztva a

$$\begin{aligned}
\frac{\|x(t) - x\|}{\|x\|} &\leq \|A^{-1}\| \left(\frac{\|\Delta b\|}{\|x\|} + \|E\| \right) |t| + \mathcal{O}(t^2) \\
&\leq \mathrm{cond}(A) \left(\frac{\|\Delta b\|}{\|A\| \cdot \|x\|} + \frac{\|E\|}{\|A\|} \right) |t| + \mathcal{O}(t^2) \\
&\leq \mathrm{cond}(A) \left(\frac{\|\Delta b\|}{\|b\|} + \frac{\|E\|}{\|A\|} \right) |t| + \mathcal{O}(t^2)
\end{aligned}$$

korlátot kapjuk.

Így ismét azt látjuk, hogy a megoldás relatív változását a kondíciószám szorozva a feladat adatainak relatív változásával korlátozza.

Az érzékenységi eredmények geometriai értelmezése két dimenzióban: ha a két egyenlet által meghatározott két egyenes közel párhuzamos, akkor metszéspontjuk nem élesen definiált, amennyiben az egyenesek kerekítési vagy mérési hiba miatt némileg bizonytalanok. Ha viszont az egyenesek távol vannak a párhuzamostól, mondjuk közel merőlegesek, akkor metszéspontjuk viszonylag élesen definiált. E két esetet a 2.4. ábra mutatja be, ahol a szaggatott vonalak jelölik az egyes folytonos vonalakhoz tartozó bizonytalansági tartományt, úgyhogy a metszéspont mindkét esetben a satírozott paralelogramma bármelyik pontja lehet. Így egy nagy kondíciószám a megoldás nagy bizonytalanságával jár.



2.4. ábra: Jól kondicionált és rosszul kondicionált lineáris egyenletrendszerek.

Összefoglalva: ha a bemeneti adatok gépi pontosságig pontosak, akkor egy $Ax = b$ lineáris egyenletrendszer közelítő $\widehat{x}$ megoldásának relatív hibájára azt várhatjuk, hogy

$$\frac{\|\hat{x} - x\|}{\|x\|} \lessapprox \operatorname{cond}(A)\ \epsilon_{\text{mach}}.$$

Ezt az eredményt a hátraható hibaelemzés nyelvén úgy értelmezhetjük, hogy a számított megoldásról azt várhatjuk: a bemenet pontosságához képest körülbelül $\log_{10}(\operatorname{cond}(\mathbf{A}))$ tizedes értékes jegyet veszít pontosságából. A 2.7. példabeli mátrix kondíciószáma például nagyobb 10^4-nél, ezért e mátrixot tartalmazó lineáris egyenletrendszer megoldásában egyetlen helyes számjegyet sem várhatunk, hacsak a bemeneti adatok négy tizedes értékes jegynél pontosabbak, és a munkapontosság is négy tizedes értékes jegynél többet hordoz (lásd a 2.8. és a 2.17. példát).

Az érzékenység kvantitatív mértékeként a mátrix kondíciószáma a lineáris egyenletrendszerek megoldásának feladatában ugyanazt a szerepet tölti be – és ugyanolyan típusú kapcsolatot ad az előreható és a hátraható hiba között –, mint az 1.2.6. szakaszban definiált általános kondíciószám. Fontos különbség azonban, hogy a mátrix kondíciószáma soha nem lehet kisebb 1-nél.

Mielőtt lezárnánk a pontosság kondíciószámok alapján történő megítélésének témáját, vegyük figyelembe e két megjegyzést:

- A fenti normákon alapuló elemzés a megoldásvektor *legnagyobb* komponenseinek relatív hibájára ad korlátot. A kisebb komponensek *relatív* hibája ennél jóval nagyobb is lehet, mivel a vektornormát a vektor legnagyobb komponensei dominálják. Komponensenkénti hibakorlátok is nyerhetők, de ezek kiszámítása valamivel összetettebb, és mi ezzel a témával nem foglalkozunk. A komponensenkénti korlátok különösen érdekesek, amikor a rendszer rosszul skálázott (lásd 2.4.10. szakasz).
- A mátrix kondíciószámát befolyásolja a mátrix skálázása (lásd 2.10. példa). Nagy kondíciószám származhat pusztán rossz skálázásból (lásd 2.20. példa), csakúgy, mint a szingularitáshoz való közelségből. A mátrix átskálázása az előbbin segíthet, az utóbbin nem (lásd 2.4.10. szakasz).

2.3.5 Maradék

Egy egyenlet megoldásának egyik ellenőrzési módja, hogy behelyettesítjük az egyenletbe, és megnézzük, mennyire illeszkedik a bal és a jobb oldal. Az $Ax = b$ lineáris egyenletrendszer közelítő $\hat{x}$ megoldásának maradéka a

$$r = b - A\hat{x}$$

különbség.

Ha A nemszinguláris, akkor elvben a $\|\Delta x\| = \|\hat{x} - x\| = 0$ hiba akkor és csak akkor teljesül, ha $\|r\| = 0$. A gyakorlatban azonban ezek a mennyiségek nem szükségképpen egyszerre kicsik. Először is figyeljük meg, hogy ha az $Ax = b$ egyenletet tetszőleges nem nulla konstanssal megszorozzuk, akkor a megoldás nem változik, de a maradék

ugyanezzel a tényezővel szorzódik. Így a maradék a feladat skálázásától függően tetszőlegesen naggyá vagy kicsivé tehető, ezért a maradék nagysága önmagában értelmetlen, hacsak nem a feladat adatainak és a megoldás méretéhez viszonyítva tekintjük. Ezért a $\hat{x}$ közelítő megoldás relatív maradékát $\|r\|/(\|A\| \cdot \|\hat{x}\|)$-ként definiáljuk.

A hiba és a maradék kapcsolatának megállapítása érdekében megfigyeljük, hogy

$$\|\Delta x\| = \|\hat{x} - x\| = \|A^{-1}(A\hat{x} - b)\| = \| - A^{-1}r\| \leq \|A^{-1}\| \cdot \|r\|.$$

Mindkét oldalt $\|\hat{x}\|$-szel osztva és $\mathrm{cond}(A)$ definícióját felhasználva kapjuk, hogy

$$\frac{\|\Delta x\|}{\|\hat{x}\|} \leq \mathrm{cond}(A)\frac{\|r\|}{\|A\| \cdot \|\hat{x}\|}.$$

Így kis relatív maradék akkor és csak akkor jelent kis relatív hibát a megoldásban, ha A jól kondicionált.

Ha viszont a nagy maradék következményeit akarjuk látni, tegyük fel, hogy a számított $\hat{x}$ megoldás egzaktul kielégíti az

$$(A + E)\hat{x} = b$$

egyenletet. Ekkor

$$\|r\| = \|b - A\hat{x}\| = \|E\hat{x}\| \leq \|E\| \cdot \|\hat{x}\|,$$

tehát a

$$\frac{\|r\|}{\|A\| \cdot \|\hat{x}\|} \leq \frac{\|E\|}{\|A\|}$$

egyenlőtlenséget kapjuk.

Így a nagy relatív maradék nagy hátraható hibát jelent a mátrixban, azaz a megoldás kiszámítására használt algoritmus instabil. Másként fogalmazva: egy stabil algoritmus a feladat kondicionáltságától függetlenül mindig kis relatív maradékú megoldást ad, ezért egy kis maradék önmagában keveset mond a közelítő megoldás minőségéről. Erre a kérdésre a 2.4.5. szakaszban még visszatérünk.

2.8. Példa. Kis maradék. Tekintsük az

$$Ax = \begin{bmatrix} 0{,}913 & 0{,}659 \\ 0{,}457 & 0{,}330 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0{,}254 \\ 0{,}127 \end{bmatrix} = b$$

lineáris egyenletrendszert, amelynek mátrixát a 2.7. példában láttuk. Tekintsük a két közelítő megoldást:

$$\hat{x}_1 = \begin{bmatrix} 0{,}6391 \\ -0{,}5 \end{bmatrix}$$

és $\hat{x}_2 = \begin{bmatrix} 0{,}999 \\ -1{,}001 \end{bmatrix}$.

A megfelelő maradékok normái

$$\|r_1\|_1 = 7{,}0 \times 10^{-5}$$

és $\|r_2\|_1 = 2{,}4 \times 10^{-2}$.

Melyik tehát a jobb megoldás? Hajlamosak lennénk $\hat{x}_1$-et választani a sokkal kisebb maradéka miatt. Azonban e rendszer egzakt megoldása – ahogy könnyen ellenőrizhető – $x = [1, -1]^T$, tehát $\hat{x}_2$ valójában sokkal pontosabb, mint $\hat{x}_1$. E meglepő viselkedés oka, hogy az A mátrix rosszul kondicionált, ahogy a 2.7. példában láttuk, és nagy kondíciószáma miatt a kis maradék nem jelent kis hibát a megoldásban. Arra, hogy $\hat{x}_1$-et hogyan kaptuk, lásd a 2.17. példát.

2.4 Lineáris egyenletrendszerek megoldása

2.4.1 Feladatátalakítások

Az $Ax = b$ lineáris egyenletrendszer megoldásához az 1.1.2. szakaszban felvázolt általános stratégia azt sugallja, hogy a rendszert olyan rendszerré kell átalakítanunk, amelynek megoldása megegyezik az eredetiével, de könnyebben kiszámítható. Milyen típusú átalakítás hagyja változatlanul egy lineáris rendszer megoldását? A válasz az, hogy az $Ax = b$ lineáris rendszer mindkét oldalát balról bármely nemszinguláris M mátrixszal megszorozhatjuk anélkül, hogy a megoldást befolyásolnánk. Hogy lássuk, miért, vegyük észre, hogy az $MAz = Mb$ lineáris rendszer megoldása

$$z = (MA)^{-1}Mb = A^{-1}M^{-1}Mb = A^{-1}b = x.$$

2.9. Példa. Permutációk. Ilyen átalakítás fontos példája az a tény, hogy A sorai és b megfelelő elemei átrendezhetők az x megoldás megváltoztatása nélkül. Ez intuitíve nyilvánvaló: a rendszer összes egyenletének egyidejűleg teljesülnie kell, ezért az a sorrend, amelyben leírjuk őket, közömbös; akár véletlenszerűen is húzhatnánk őket egy kalapból. Formálisan a sorok ilyen átrendezését úgy valósítjuk meg, hogy az egyenlet mindkét oldalát balról megszorozzuk egy P permutációs mátrixszal, amely olyan négyzetes mátrix, amelynek minden sorában és oszlopában pontosan egy 1-es szerepel, a többi elem pedig nulla (azaz egy egységmátrix, amelynek sorai és oszlopai permutálva vannak). Például

$$\begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} v_3 \\ v_1 \\ v_2 \end{bmatrix}.$$

Egy permutációs mátrix mindig nemszinguláris; valójában az inverze egyszerűen a transzponáltja, $P^{-1} = P^T$ (egy M mátrix transzponáltja, amelyet M^T jelöl, olyan

mátrix, amelynek oszlopai M sorai, azaz ha $N = M^T$, akkor $n_{ij} = m_{ji}$). Így az átrendezett rendszer $PAx = Pb$ alakban írható, és az x megoldás változatlan marad.

Ha jobbról szorzunk meg egy permutációs mátrixszal, akkor az a mátrix oszlopait rendezi át a sorok helyett. Az ilyen átalakítás megváltoztatja a megoldást, de csak úgy, hogy a megoldás komponensei permutálódnak. Ennek belátásához figyeljük meg, hogy az $APz = b$ rendszer megoldása

$$z = (AP)^{-1}b = P^{-1}A^{-1}b = P^T A^{-1}b = P^T x,$$

és ennélfogva az eredeti $Ax = b$ rendszer megoldása $x = Pz$.

2.10. Példa. Diagonális skálázás. Egy másik egyszerű, de fontos átalakítási típus a diagonális skálázás. Emlékezzünk vissza, hogy egy D mátrix diagonális, ha $d_{ij} = 0$ minden $i \neq j$ esetén, azaz az egyetlen nemnulla elemei a főátlón lévő d_{ii}, $i = 1, \dots, n$ elemek. Ha az $Ax = b$ lineáris rendszer mindkét oldalát balról megszorozzuk egy nemszinguláris D diagonális mátrixszal, akkor a mátrix minden sorát és a jobb oldalt D megfelelő átlóbeli elemével szorozzuk meg, és ezt ezért sorkénti skálázásnak nevezzük. Elvben a sorkénti skálázás nem változtatja meg a lineáris rendszer megoldását, a gyakorlatban azonban befolyásolhatja a numerikus megoldási folyamatot és az adott feladatnál elérhető pontosságot, mint azt látni fogjuk.

Az oszloponkénti skálázás – egy lineáris rendszer mátrixának jobbról való megszorzása egy nemszinguláris D diagonális mátrixszal – a mátrix minden oszlopát D megfelelő átlóbeli elemével szorozza meg. Az ilyen átalakítás valóban megváltoztatja a megoldást, gyakorlatilag azokat a mértékegységeket változtatja meg, amelyekben a megoldás komponenseit mérjük. A skálázott $ADz = b$ rendszer megoldása

$$z = (AD)^{-1}b = D^{-1}A^{-1}b = D^{-1}x,$$

és ennélfogva az eredeti $Ax = b$ rendszer megoldása $x = Dz$.

2.4.2 Háromszögű lineáris egyenletrendszerek

A következő kérdés az, hogy milyen típusú lineáris rendszer könnyen megoldható. Tegyük fel, hogy az $Ax = b$ rendszerben van egy olyan egyenlet, amely csak egy ismeretlen megoldáskomponenst tartalmaz (azaz A ezen sorában csak egy elem nemnulla). Akkor ez az egyenlet könnyen megoldható (osztással) erre az ismeretlenre. Tegyük fel most, hogy a rendszerben van egy másik egyenlet, amely csak két ismeretlent tartalmaz, és ezek egyike a már meghatározott. Ha a már meghatározott megoldáskomponenst behelyettesítjük ebbe a második egyenletbe, akkor könnyen megoldhatjuk a másik ismeretlenre. Ha ez a minta folytatódik – egyenletenként csak egy új ismeretlen komponens lép fel –, akkor a megoldás összes komponense egymás után kiszámítható. Az ilyen speciális tulajdonságú mátrixot *háromszögnek* nevezzük – a név okát hamar látni fogjuk. Mivel a háromszögű lineáris rendszerek ezzel az egymás utáni helyettesítéssel könnyen megoldhatók, alkalmas cél, amelyre egy általános lineáris rendszert átalakíthatunk.

Bár a fentebb leírt általános háromszögalak elegendő ahhoz, hogy a rendszer egymás utáni helyettesítéssel megoldható legyen, számítási célokra kényelmes két speciális háromszögalakot definiálnunk. Egy L mátrix *alsó háromszögmátrix*, ha a főátló feletti összes eleme nulla (azaz $\ell_{ij} = 0$ $i < j$ esetén). Hasonlóan, egy U mátrix *felső háromszögmátrix*, ha a főátló alatti összes eleme nulla (azaz $u_{ij} = 0$ $i > j$ esetén). Egy a fent definiált általánosabb értelemben háromszögalakú mátrix sorainak vagy oszlopainak alkalmas permutációjával felső vagy alsó háromszögalakba hozható.

Egy $Lx = b$ alsó háromszögű rendszer esetén az egymás utáni helyettesítést *előrehelyettesítésnek* nevezzük, és matematikailag így fejezhetjük ki:

$$x_1 = b_1/\ell_{11}, \quad x_i = \left(b_i - \sum_{j=1}^{i-1} \ell_{ij} x_j \right) / \ell_{ii}, \quad i = 2, \dots, n.$$

Megvalósításának egyik módját a 2.1. Algoritmus mutatja.

2.1. Algoritmus. Előrehelyettesítés alsó háromszögű rendszerre.

```
for j = 1 to n                          { ciklus az oszlopokra }
    if □jj = 0 then stop                { leállás, ha a mátrix szinguláris }
    xj = bj/□jj                         { megoldáskomponens kiszámítása }
    for i = j + 1 to n
        bi = bi - □ij xj                { jobb oldal frissítése }
    end
end
```

Hasonlóan, egy $Ux = b$ felső háromszögű rendszer esetén az egymás utáni helyettesítést *visszahelyettesítésnek* nevezzük, és matematikailag így fejezhetjük ki:

$$x_n = b_n/u_{nn}, \quad x_i = \left(b_i - \sum_{j=i+1}^{n} u_{ij} x_j \right) / u_{ii}, \quad i = n-1, \dots, 1.$$

Megvalósításának egyik módját a 2.2. Algoritmus mutatja.

Mindkét algoritmusban úgy választottuk meg a ciklusváltozók sorrendjét, hogy a mátrixhoz oszloponként (és nem soronként) férünk hozzá, és a belső ciklus művelete egy skalár szorozva egy vektorral, majd ehhez hozzáadva egy vektor – azaz egy „saxpy” művelet (lásd a 2.7.2. szakaszt).

2.2. Algoritmus. Visszahelyettesítés felső háromszögű rendszerre.

```
for j = n to 1                          { ciklus az oszlopokra visszafelé }
    if ujj = 0 then stop                { leállás, ha a mátrix szinguláris }
    xj = bj/ujj                         { megoldáskomponens kiszámítása }
    for i = 1 to j - 1
        bi = bi - uij xj                { jobb oldal frissítése }
    end
end
```

Választhattuk volna a ciklusváltozók fordított sorrendjét is, ekkor a mátrixhoz soronként férnénk hozzá, és a belső ciklus művelete két vektor belső szorzata lenne, azaz „sdot”. Ezek a megvalósítási választások jelentős hatással lehetnek a teljesítményre, az alkalmazott programozási nyelvtől és számítógépes rendszertől függően (lásd a 2.16. számítógépes feladatot). Azt is érdemes megjegyezni, hogy egy nulla átlóbeli elem mindkét algoritmust elbukatja, de ez így is várható, mivel egy nulla átlóbeli elemű háromszögmátrix szükségképpen szinguláris.

2.11. Példa. Háromszögű lineáris egyenletrendszer. Tekintsük a következő felső háromszögű lineáris rendszert:

$$\begin{bmatrix} 1 & 2 & 2 \\ 0 & -4 & -6 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ -6 \\ 1 \end{bmatrix}.$$

Az utolsó egyenlet, $-x_3 = 1$, közvetlenül megoldható $x_3 = -1$-re. Ezt az értéket ezután behelyettesíthetjük a második egyenletbe, hogy $x_2 = 3$-at kapjunk, végül pedig x_3-at és x_2-t egyaránt behelyettesítve az első egyenletbe, $x_1 = -1$ adódik.

2.4.3 Elemi eliminációs mátrixok

Stratégiánk tehát az, hogy olyan nemszinguláris lineáris transzformációt tervezzünk, amely egy adott általános lineáris rendszert olyan háromszögű lineáris rendszerré alakít, amely utána az egymás utáni helyettesítéssel könnyen megoldható. Ehhez tehát olyan transzformációra van szükségünk, amely az adott mátrix kiválasztott nemnulla elemeit nullákkal helyettesíti. Ez a mátrix sorainak alkalmas lineáris kombinációival érhető el, ahogyan azt most megmutatjuk.

Tekintsük az $a = [a_1 \quad a_2]^T$ 2-vektort. Ha $a_1 \neq 0$, akkor

$$\begin{bmatrix} 1 & 0 \\ -a_2/a_1 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} a_1 \\ 0 \end{bmatrix}.$$

Általánosabban: egy n-dimenziós a vektor k-adik pozíciója alatti minden elemet annullálhatunk – feltéve, hogy $a_k \neq 0$ – a következő transzformációval:

$$M_k\, a = \begin{bmatrix} 1 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 1 & 0 & \cdots & 0 \\ 0 & \cdots & -m_{k+1} & 1 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & -m_n & 0 & \cdots & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ \vdots \\ a_k \\ a_{k+1} \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} a_1 \\ \vdots \\ a_k \\ 0 \\ \vdots \\ 0 \end{bmatrix},$$

ahol $m_i = a_i/a_k$, $i = k+1, \dots, n$. Az a_k osztót *főelemnek* (pivotnak) nevezzük. Az ilyen alakú mátrixot néha *elemi eliminációs mátrixnak* vagy *Gauss-transzformációnak* nevezzük, és egy vektorra gyakorolt hatása az, hogy a k-adik sor többszörösét hozzáadja minden további sorhoz, úgy megválasztva az m_i szorzótényezőket, hogy az

eredmény minden esetben nulla legyen. Vegyük észre az elemi eliminációs mátrixok alábbi tulajdonságait:

1. M_k alsó háromszögmátrix, amelynek főátlójában csupa egyes áll, ezért szükségképpen nemszinguláris.
2. $M_k = I - m_k e_k^T$, ahol $m_k = [0, \dots, 0, m_{k+1}, \dots, m_n]^T$, és e_k az egységmátrix k-adik oszlopa.
3. $M_k^{-1} = I + m_k e_k^T$, ami azt jelenti, hogy M_k^{-1} – amelyet L_k-val fogunk jelölni – ugyanaz, mint M_k, csak a szorzótényezők előjele megváltozik.
4. Ha M_j, $j > k$, egy másik elemi eliminációs mátrix m_j szorzótényező-vektorral, akkor

$$M_k M_j = I - m_k e_k^T - m_j e_j^T + m_k e_k^T m_j e_j^T = I - m_k e_k^T - m_j e_j^T,$$

mivel $e_k^T m_j = 0$. Így szorzatuk lényegében „uniójuk”. Mivel azonos alakúak, hasonló eredmény érvényes inverzeik $L_k L_j$ szorzatára is. Figyeljük meg, hogy a szorzás sorrendje lényeges; ezek az eredmények a fordított szorzatra nem teljesülnek.

2.12. Példa. Elemi eliminációs mátrixok. Ha $a = [2 \quad 4 \quad -2]^T$, akkor

$$M_1 a = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 4 \\ -2 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}, \quad M_2 a = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0{,}5 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 4 \\ -2 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \\ 0 \end{bmatrix}.$$

Megjegyezzük továbbá, hogy

$$L_1 = M_1^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}, \quad L_2 = M_2^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -0{,}5 & 1 \end{bmatrix},$$

és

$$M_1 M_2 = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 1 & 0{,}5 & 1 \end{bmatrix}, \quad L_1 L_2 = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -0{,}5 & 1 \end{bmatrix}.$$

2.4.4 Gauss-kiküszöbölés és LU-felbontás

Elemi eliminációs mátrixokat használva viszonylag egyszerűen redukálhatunk egy általános $Ax = b$ lineáris rendszert felső háromszögalakra. Először a 2.4.3. szakaszban megadott recept szerint választunk egy M_1 elemi eliminációs mátrixot, úgy, hogy a főelem az első átlóbeli elem, a_{11} legyen, hogy A első oszlopa nullává váljon az első sor alatt, amikor M_1-gyel balról szorozzuk. Természetesen A többi oszlopát, valamint a b jobb oldali vektort is meg kell szorozni M_1-gyel, így az új rendszer $M_1 A x = M_1 b$ alakú lesz, de korábbi tárgyalásunk szerint a megoldás változatlan.

Ezután a második átlóbeli elemet használjuk főelemként egy olyan M_2 elemi eliminációs mátrix meghatározásához, amely az új $M_1 A$ mátrix második oszlopának minden

elemét a második sor alatt annullálja. M_2-t ismét az egész mátrixra és a jobb oldali vektorra kell alkalmazni, így a $M_2M_1Ax = M_2M_1b$ módosított lineáris rendszert kapjuk. Megjegyzendő, hogy M_1A első oszlopát M_2 nem befolyásolja, mivel az érintett sorokban minden eleme nulla. Ez a folyamat minden egyes egymás utáni oszlopra folytatódik, amíg a mátrix összes főátló alatti eleme nem annullálódik. Ha bevezetjük az $M = M_{n-1} \cdots M_1$ mátrixot, akkor a transzformált lineáris rendszer

$$MAx = M_{n-1} \cdots M_1 Ax = M_{n-1} \cdots M_1 b = Mb$$

felső háromszögű, és visszahelyettesítéssel megoldható, hogy megkapjuk az eredeti $Ax = b$ lineáris rendszer megoldását.

Az imént leírt eljárást *Gauss-kiküszöbölésnek* nevezzük. *LU-felbontásnak* is nevezik, mert az A mátrixot egy egységnyi átlójú alsó háromszögmátrix, L, és egy felső háromszögmátrix, U, szorzatára bontja. Ennek belátásához emlékezzünk vissza, hogy az L_kL_j szorzat egységnyi átlójú alsó háromszögű, ha $k < j$, így

$$L = M^{-1} = (M_{n-1} \cdots M_1)^{-1} = M_1^{-1} \cdots M_{n-1}^{-1} = L_1 \cdots L_{n-1}$$

egységnyi átlójú alsó háromszögmátrix. Már láttuk, hogy konstrukció szerint az $U = MA$ mátrix felső háromszögű. Ennek megfelelően A-t

$$A = LU$$

szorzatként állítottuk elő, ahol L egységnyi átlójú alsó háromszögmátrix és U felső háromszögmátrix. Egy ilyen felbontás birtokában az $Ax = b$ lineáris rendszer $LUx = b$ alakban írható, és így előbb az $Ly = b$ alsó háromszögű rendszert előrehelyettesítéssel, majd az $Ux = y$ felső háromszögű rendszert visszahelyettesítéssel megoldva megoldható. Megjegyzendő, hogy az y köztes megoldás megegyezik a korábbi megfogalmazásban szereplő transzformált jobb oldali Mb vektorral. Így a Gauss-kiküszöbölés és az LU-felbontás ugyanannak a megoldási folyamatnak két kifejezésmódja. Azzal azonban, hogy az előre- és visszahelyettesítő háromszöges megoldási fázisokat a felbontástól elkülönítve hangsúlyozza, az LU-alak talán világosabbá teszi, hogy a felbontási fázist nem kell megismételni, amikor ugyanazzal az A mátrixszal, de más jobb oldali vektorokkal további rendszereket oldunk meg, mivel az L és U tényezők újra felhasználhatók.

A Gauss-kiküszöbölési eljárást formálisan a 2.3. Algoritmus foglalja össze. Ez az algoritmus egyúttal A LU-felbontását is kiszámítja: L főátló alatti elemeit $\ell_{ik} = m_{ik}$ adja meg, U átlóbeli és főátló feletti elemei pedig A megfelelő elemeit írják felül. A gyakorlatban nem szükséges A főátló alatti elemeit explicit módon nullára állítani, mivel csak a U és L eredményül kapott háromszögmátrixok érdekelnek bennünket. Sőt: A főátló alatti elemei, amelyeket kinulláztunk, tökéletes helyet nyújtanak L főátló alatti elemeinek tárolására; a 2.3. Algoritmus lényegében „helyben” végzi el a felbontást, ha m_{ik} helyett a_{ik}-t írunk minden előforduláskor (lásd a 2.4.6. szakaszt ezekről és más megvalósítási részletekről). Egy $Ax = b$ lineáris rendszer megoldása során a b jobb oldali vektor szükséges transzformációját beépíthetjük az LU-felbontási folyamatba,

vagy különálló lépésként végezhetjük el, a 2.1. Algoritmust használva az $Ly = b$ alsó háromszögű rendszer megoldására, miután L-et a 2.3. Algoritmussal meghatároztuk. Bármelyik esetben a 2.2. Algoritmust használjuk ezután az $Ux = y$ felső háromszögű rendszer megoldására, hogy megkapjuk az x megoldást.

2.3. Algoritmus. LU-felbontás Gauss-kiküszöböléssel.

```
for k = 1 to n - 1                      { ciklus az oszlopokra }
    if akk = 0 then stop                { leállás, ha a főelem nulla }
    for i = k + 1 to n                  { szorzótényezők kiszámítása
        mik = aik/akk                       az aktuális oszlopra }
    end
    for j = k + 1 to n                  { transzformáció alkalmazása
        for i = k + 1 to n                  a maradék részmátrixra }
            aij = aij - mik akj
        end
    end
end
```

2.13. Példa. Gauss-kiküszöbölés. A Gauss-kiküszöbölést a következő lineáris egyenletrendszer megoldásán illusztráljuk:

$$\begin{aligned} x_1 + 2x_2 + 2x_3 &= 3, \\ 4x_1 + 4x_2 + 2x_3 &= 6, \\ 4x_1 + 6x_2 + 4x_3 &= 10, \end{aligned}$$

vagy mátrix jelöléssel

$$Ax = \begin{bmatrix} 1 & 2 & 2 \\ 4 & 4 & 2 \\ 4 & 6 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = b.$$

A első oszlopának főátló alatti elemeinek annullálásához az első sor négyszeresét vonjuk ki a második és a harmadik sorból:

$$M_1 A = \begin{bmatrix} 1 & 0 & 0 \\ -4 & 1 & 0 \\ -4 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 2 \\ 4 & 4 & 2 \\ 4 & 6 & 4 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 2 \\ 0 & -4 & -6 \\ 0 & -2 & -4 \end{bmatrix},$$

$$M_1 b = \begin{bmatrix} 1 & 0 & 0 \\ -4 & 1 & 0 \\ -4 & 0 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 6 \\ 10 \end{bmatrix} = \begin{bmatrix} 3 \\ -6 \\ -2 \end{bmatrix}.$$

Most az $M_1 A$ második oszlopa főátló alatti elemének annullálásához a második sor $0{,}5$-szörösét vonjuk ki a harmadik sorból:

$$\mathbf{M}_2\mathbf{M}_1\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -0{,}5 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 2 \\ 0 & -4 & -6 \\ 0 & -2 & -4 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 2 \\ 0 & -4 & -6 \\ 0 & 0 & -1 \end{bmatrix},$$

$$\mathbf{M}_2\mathbf{M}_1\mathbf{b} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -0{,}5 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ -6 \\ -2 \end{bmatrix} = \begin{bmatrix} 3 \\ -6 \\ 1 \end{bmatrix}.$$

Ezzel az eredeti rendszert a következő ekvivalens felső háromszögű rendszerre redukáltuk:

$$Ux = \begin{bmatrix} 1 & 2 & 2 \\ 0 & -4 & -6 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ -6 \\ 1 \end{bmatrix} = Mb = y,$$

amelyet most visszahelyettesítéssel oldhatunk meg (mint a 2.11. Példában), és így $x = [-1 \quad 3 \quad -1]^T$ adódik. Az LU-felbontás explicit felírásához

$$L_1L_2 = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 4 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0{,}5 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 4 & 0{,}5 & 1 \end{bmatrix} = L,$$

így

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 2 \\ 4 & 4 & 2 \\ 4 & 6 & 4 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 4 & 0{,}5 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 2 \\ 0 & -4 & -6 \\ 0 & 0 & -1 \end{bmatrix} = \mathbf{LU}.$$

2.4.5 Főelemkiválasztás

A Gauss-kiküszöbölési eljárással, ahogy eddig leírtuk, van egy kézenfekvő probléma, és van egy másik, némileg finomabb is. A kézenfekvő potenciális nehézség az, hogy az eljárás összeomlik, ha a mátrix még redukálatlan részének vezető átlóbeli eleme valamelyik lépésben nulla, mivel egy adott oszlop m_i szorzótényezőinek kiszámításához az oszlop átlóbeli elemével való osztásra van szükség. A probléma megoldása majdnem ugyanilyen kézenfekvő: ha a k-adik lépésben az átlóbeli elem nulla, akkor cseréljük ki a rendszer k-adik sorát (mind a mátrixét, mind a jobb oldali vektorét) egy olyan későbbi sorral, amelynek k-adik oszlopbeli eleme nemnulla. A 2.9. Példából tudjuk, hogy egy ilyen csere nem változtatja meg a rendszer megoldását. Nemnulla átlóbeli elemet főelemként véve az eljárás a szokásos módon folytatódhat. A sorok ilyen cseréjét *főelemkiválasztásnak* nevezzük.

2.14. Példa. Főelemkiválasztás és szingularitás. A főelemkiválasztás esetleges szüksége semmilyen kapcsolatban nem áll azzal, hogy a mátrix szinguláris-e. Például az

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

mátrix nemszinguláris, mégsincs LU-felbontása, hacsak nem cseréljük fel a sorokat, míg a szinguláris

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

mátrixnak van LU-felbontása:

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} = LU.$$

De mi van akkor, ha a k-adik oszlop átlóján vagy az alatt nincs nemnulla elem? Akkor ebben a lépésben nincs mit tenni, mivel az annullálandó elemek már eleve nullák, így egyszerűen továbbléphetünk a következő oszlopra (azaz $M_k = I$). Jegyezzük meg, hogy ez a lépés nullát hagy az átlón, és így az eredményül kapott U felső háromszögmátrix szinguláris lesz, de az LU-felbontás még így is befejezhető. Ez viszont azt jelenti, hogy az ezt követő visszahelyettesítés meghiúsul, mert U minden átlóbeli elemével osztani kellene, de ez nem meglepő, hiszen az eredeti mátrixnak mindenképpen szingulárisnak kellett lennie. Sokkal alattomosabb probléma az, hogy lebegőpontos aritmetikában előfordulhat, hogy nem pontosan nullát kapunk, hanem csak egy nagyon kicsi átlóbeli elemet – és ezzel érkezünk meg a finomabb ponthoz.

Elvben bármely nemnulla érték megfelelhet főelemnek a szorzótényezők kiszámítására, de véges pontosságú aritmetikában a választást körültekintően kell meghozni, hogy minimalizáljuk a numerikus hiba terjedését. Konkrétan korlátozni szeretnénk a szorzótényezők nagyságát, hogy a korábbi kerekítési hibák ne erősödjenek fel, amikor a mátrix és a jobb oldal maradék részét megszorozzuk az egyes elemi eliminációs mátrixokkal. A szorzótényezők nagysága soha nem fogja meghaladni az 1-et, ha minden oszlopban az átlón vagy az alatt található legnagyobb abszolút értékű elemet választjuk főelemnek. Az ilyen eljárást *részleges főelemkiválasztásnak* nevezzük, és a gyakorlatban elengedhetetlen az általános lineáris rendszereken végzett Gauss-kiküszöbölés numerikusan stabil megvalósításához.

2.15. Példa. Kis főelemek. Véges pontosságú aritmetikában nemcsak a nulla főelemeket, hanem a *kis* főelemeket is kerülnünk kell, hogy megelőzzük az elfogadhatatlan hibanövekedést, mint az alábbi példa mutatja. Legyen

$$A = \begin{bmatrix} \epsilon & 1 \\ 1 & 1 \end{bmatrix},$$

ahol ϵ egy olyan pozitív szám, amely az adott lebegőpontos rendszer ϵ_{mach} egységnyi kerekítési hibájánál kisebb. Ha nem cseréljük fel a sorokat, akkor a főelem ϵ, és a kapott szorzótényező $-1/\epsilon$, így az eliminációs mátrix

$$M = \begin{bmatrix} 1 & 0 \\ -1/\epsilon & 1 \end{bmatrix},$$

és ennélfogva

$$L = \begin{bmatrix} 1 & 0 \\ 1/\epsilon & 1 \end{bmatrix} \quad \text{és} \quad U = \begin{bmatrix} \epsilon & 1 \\ 0 & 1 - 1/\epsilon \end{bmatrix} = \begin{bmatrix} \epsilon & 1 \\ 0 & -1/\epsilon \end{bmatrix}$$

lebegőpontos aritmetikában. De akkor

$$LU = \begin{bmatrix} 1 & 0 \\ 1/\epsilon & 1 \end{bmatrix} \begin{bmatrix} \epsilon & 1 \\ 0 & -1/\epsilon \end{bmatrix} = \begin{bmatrix} \epsilon & 1 \\ 1 & 0 \end{bmatrix} \neq A.$$

Egy kis főelem és az ennek megfelelő nagy szorzótényező használata a transzformált mátrixban helyrehozhatatlan információvesztést okozott. Ha viszont felcseréljük a sorokat, akkor a főelem 1, és a kapott szorzótényező $-\epsilon$, így az eliminációs mátrix

$$M = \begin{bmatrix} 1 & 0 \\ -\epsilon & 1 \end{bmatrix},$$

és ennélfogva

$$L = \begin{bmatrix} 1 & 0 \\ \epsilon & 1 \end{bmatrix} \quad \text{és} \quad U = \begin{bmatrix} 1 & 1 \\ 0 & 1 - \epsilon \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

lebegőpontos aritmetikában. Így

$$LU = \begin{bmatrix} 1 & 0 \\ \epsilon & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ \epsilon & 1 \end{bmatrix},$$

ami a permutálás utáni helyes eredmény.

Bár az előbbi példa meglehetősen szélsőséges, az elv általánosan érvényes: a nagyobb főelemek kisebb szorzótényezőkhöz, és így kisebb hibákhoz vezetnek. Különösen, ha minden oszlopban az átlón vagy az alatt található legnagyobb elemet használjuk főelemnek – mint a 2.4. Algoritmusban –, akkor a szorzótényezők nagysága 1-gyel felülről korlátozott. A részleges főelemkiválasztás által megkövetelt sorcserék némileg bonyolultabbá teszik az LU-felbontás korábban megadott formális leírását. Konkrétan minden M_k elemi eliminációs mátrix előtt egy P_k permutációs mátrix áll, amely sorcserékkel a k-adik oszlop átlón vagy alatt található legnagyobb abszolút értékű elemét az átlós főelempozícióba hozza. Továbbra is $MA = U$, ahol U felső háromszögű, de most

$$M = M_{n-1}P_{n-1} \cdots M_1 P_1.$$

M^{-1} továbbra is háromszögalakú a korábban definiált általános értelemben, de a permutációk miatt M^{-1} nem szükségképpen alsó háromszögű, bár továbbra is L-lel jelöljük. Így az „LU”-felbontás szó szerint már nem „alsó szorozva felső” háromszögmátrixot jelent, de a lineáris rendszerek egymás utáni helyettesítéssel való megoldására ugyanolyan hasznos.

2.4. Algoritmus. LU-felbontás Gauss-kiküszöböléssel részleges főelemkiválasztással.

```
for k = 1 to n - 1                          { ciklus az oszlopokra }
    Find index p such that                  { főelem keresése az
        |apk| ≥ |aik| for k ≤ i ≤ n             aktuális oszlopban }
    if p ≠ k then                           { sorok cseréje,
        interchange rows k and p                ha szükséges }
    if akk = 0 then                         { az aktuális oszlop átugrása,
        continue with next k                    ha már eleve nulla }
    for i = k + 1 to n                      { szorzótényezők kiszámítása
        mik = aik/akk                           az aktuális oszlopra }
    end
    for j = k + 1 to n                      { transzformáció alkalmazása
        for i = k + 1 to n                      a maradék részmátrixra }
            aij = aij - mik akj
        end
    end
end
```

Megjegyezzük, hogy a

$$P = P_{n-1} \cdots P_1$$

permutációs mátrix a részleges főelemkiválasztás által meghatározott sorrendbe rendezi A sorait. Egy másik értelmezés szerint tehát a részleges főelemkiválasztásra úgy is tekinthetünk, mint egy olyan sorrend meghatározásának módjára, amelyben a numerikus stabilitás érdekében nem volna szükség semmilyen cserére (persze egy ilyen sorrendet előre meghatározni nem lehet). Így a

$$PA = LU$$

felbontást kapjuk, ahol most L valóban alsó háromszögmátrix. Az $Ax = b$ lineáris rendszer megoldásához először az $Ly = Pb$ alsó háromszögű rendszert előrehelyettesítéssel, majd az $Ux = y$ felső háromszögű rendszert visszahelyettesítéssel oldjuk meg.

2.16. Példa. Gauss-kiküszöbölés részleges főelemkiválasztással. A 2.13. Példában nem használtunk sorcseréket, és néhány szorzótényező nagysága meghaladta az 1-et. Most illusztráció kedvéért megismételjük ezt a példát, ezúttal részleges főelemkiválasztást alkalmazva. A 2.13. Példa rendszere

$$Ax = \begin{bmatrix} 1 & 2 & 2 \\ 4 & 4 & 2 \\ 4 & 6 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \\ 10 \end{bmatrix} = b.$$

Az első oszlop legnagyobb eleme 4, ezért felcseréljük az első két sort a

$$\mathbf{P}_1 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

permutációs mátrixszal, így a permutált rendszer

$$P_1 Ax = \begin{bmatrix} 4 & 4 & 2 \\ 1 & 2 & 2 \\ 4 & 6 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6 \\ 3 \\ 10 \end{bmatrix} = P_1 b.$$

Az első oszlop főátló alatti elemeinek annullálásához a

$$\mathbf{M}_1 = \begin{bmatrix} 1 & 0 & 0 \\ -0{,}25 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

eliminációs mátrixot használjuk, így a transzformált rendszer

$$M_1 P_1 Ax = \begin{bmatrix} 4 & 4 & 2 \\ 0 & 1 & 1{,}5 \\ 0 & 2 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6 \\ 1{,}5 \\ 4 \end{bmatrix} = M_1 P_1 b.$$

A második oszlopban az átlón vagy az alatt található legnagyobb elem 2, ezért felcseréljük az utolsó két sort a

$$P_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

permutációs mátrixszal, így a permutált rendszer

$$P_2 M_1 P_1 Ax = \begin{bmatrix} 4 & 4 & 2 \\ 0 & 2 & 2 \\ 0 & 1 & 1{,}5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6 \\ 4 \\ 1{,}5 \end{bmatrix} = P_2 M_1 P_1 b.$$

A második oszlop főátló alatti elemének annullálásához a

$$\mathbf{M}_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -0{,}5 & 1 \end{bmatrix}$$

eliminációs mátrixot használjuk, így a transzformált rendszer

$$M_2P_2M_1P_1Ax = \begin{bmatrix} 4 & 4 & 2 \\ 0 & 2 & 2 \\ 0 & 0 & 0{,}5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6 \\ 4 \\ -0{,}5 \end{bmatrix} = M_2P_2M_1P_1b.$$

Ezzel az eredeti rendszert egy ekvivalens felső háromszögű rendszerre redukáltuk, amelyet most visszahelyettesítéssel oldhatunk meg, és ugyanazt a megoldást kapjuk, mint korábban: $x = [-1 \quad 3 \quad -1]^T$.

Az LU-felbontás explicit felírásához

$$L = M^{-1} = (M_2P_2M_1P_1)^{-1} = P_1^TL_1P_2^TL_2 =$$

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0{,}25 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0{,}5 & 1 \end{bmatrix} = \begin{bmatrix} 0{,}25 & 0{,}5 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix},$$

és ennélfogva

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 2 \\ 4 & 4 & 2 \\ 4 & 6 & 4 \end{bmatrix} = \begin{bmatrix} 0{,}25 & 0{,}5 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 4 & 4 & 2 \\ 0 & 2 & 2 \\ 0 & 0 & 0{,}5 \end{bmatrix} = \mathbf{LU}.$$

Vegyük észre, hogy L nem alsó háromszögű, de az általánosabb értelemben háromszögű (egy alsó háromszögmátrix permutációja). Alternatívaként választhatjuk a

$$P = P_2P_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix},$$

és

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0{,}25 & 0{,}5 & 1 \end{bmatrix}$$

megválasztást, így

$$PA = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 & 2 \\ 4 & 4 & 2 \\ 4 & 6 & 4 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0{,}25 & 0{,}5 & 1 \end{bmatrix} \begin{bmatrix} 4 & 4 & 2 \\ 0 & 2 & 2 \\ 0 & 0 & 0{,}5 \end{bmatrix} = LU,$$

ahol most L valóban alsó háromszögmátrix, de A permutálva van.

A „részleges” főelemkiválasztás elnevezés abból a tényből ered, hogy csak az aktuális oszlopban keresünk alkalmas főelemet. Egy kimerítőbb főelemkiválasztási stratégia a *teljes főelemkiválasztás*, amelyben a teljes maradék, redukálatlan részmátrixban keressük a legnagyobb elemet, majd azt permutáljuk az átlós főelempozícióba. Vegyük észre, hogy ez a sorok cseréje mellett az oszlopok cseréjét is igényli, és ezért a

$$PAQ = LU$$

alakú felbontáshoz vezet, ahol L egységnyi átlójú alsó háromszögmátrix, U felső háromszögmátrix, P és Q pedig permutációs mátrixok, amelyek rendre A sorait és oszlopait rendezik át. Az $Ax = b$ lineáris rendszer megoldásához először az $Ly = Pb$ alsó háromszögű rendszert előrehelyettesítéssel, majd az $Uz = y$ felső háromszögű rendszert visszahelyettesítéssel oldjuk meg, végül pedig a megoldás komponenseit permutáljuk, hogy megkapjuk $x = Qz$-t. Bár a teljes főelemkiválasztás numerikus stabilitása elméletben jobb, sokkal költségesebb főelem-keresést igényel, mint a részleges főelemkiválasztás. Mivel a részleges főelemkiválasztás numerikus stabilitása a gyakorlatban bőven elegendő, általános lineáris rendszerek Gauss-kiküszöböléssel történő megoldásában szinte univerzálisan ezt használják.

A főelem kiválasztása az egyes mátrixelemek nagyságától függ, így a konkrét választás nyilvánvalóan függ a mátrix skálázásától. A mátrix diagonális skálázása (emlékezzünk a 2.10. Példára) eltérő főelem-sorrendhez vezethet. Például egy adott oszlop bármelyik nemnulla eleme a legnagyobbá tehető abszolút értékben, ha az adott sornak kellően nagy súlyt adunk. Ez azonban nem jelenti azt, hogy tetszőleges főelem-sorrend elfogadható: egy rosszul eltorzított skálázás rosszul kondicionált rendszerhez és ennek megfelelően pontatlan megoldáshoz vezethet (lásd a 2.20. Példát). Egy jól megfogalmazott feladatban az ismeretlen változók mérésére megfelelően arányos mértékegységeket kell használni (oszloponkénti skálázás), és az egyes egyenleteknek a viszonylagos fontosságukat helyesen tükröző súlyozást kell kapniuk (sorkénti skálázás). Figyelembe kell venni a bemeneti adatok viszonylagos pontosságát is. Ezek között a körülmények között a főelemkiválasztási eljárás általában olyan megoldást produkál, amely annyira pontos, amennyire a feladat indokolja (lásd a 2.3.4. szakaszt).

A 2.3.5. szakaszban láttuk, hogy egy kiszámított megoldás relatív maradéka kielégíti a

$$\frac{\|r\|}{\|A\| \cdot \|\hat{x}\|} \leq \frac{\|E\|}{\|A\|}$$

egyenlőtlenséget, ahol E az A mátrixhoz tartozó hátraható hiba. De mekkora is $\|E\|$ valójában a gyakorlatban? Wilkinson [499] megmutatta, hogy a Gauss-kiküszöböléssel végzett LU-felbontás esetén

$$\frac{\|E\|}{\|A\|} \leq \rho\, n^2\, \epsilon_{\text{mach}}$$

alakú korlát érvényes, ahol ρ – amelyet *növekedési tényezőnek* nevezünk – U legnagyobb abszolút értékű elemének és A legnagyobb abszolút értékű elemének há-

nyadosa (technikailag a növekedési tényező a felbontási folyamat *bármely* lépésében előállított legnagyobb elemtől függ, de ez rendszerint az utolsó, azaz U). Főelemkiválasztás nélkül ρ tetszőlegesen nagy lehet, és ezért a főelemkiválasztás nélküli Gauss-kiküszöbölés instabil, amint azt már láttuk. Részleges főelemkiválasztás mellett a növekedési tényező legrosszabb esetben elérheti a 2^{n-1}-t (mivel legrosszabb esetben az elemek mérete az elimináció minden lépésében megduplázódhat), de az ilyen viselkedés rendkívül ritka. A gyakorlatban kicsi vagy egyáltalán nincs növekedés, és egy realisztikus korlátot a

$$\frac{\|E\|}{\|A\|} \lessapprox n\, \epsilon_{\text{mach}}$$

reláció ad. Ez a reláció azt jelenti, hogy egy lineáris rendszernek részleges főelemkiválasztásos Gauss-kiküszöböléssel, majd visszahelyettesítéssel való megoldása szinte mindig nagyon kicsi relatív maradékot eredményez, függetlenül attól, mennyire rosszul kondicionált a rendszer. Így egy kicsi relatív maradék önmagában nem feltétlenül jelenti, hogy a kiszámított megoldás pontos, hacsak a rendszer nem jól kondicionált. A teljes főelemkiválasztás még kisebb növekedési tényezőt eredményez, mind elméletben, mind a gyakorlatban, de az általa nyújtott további stabilitási tartalék rendszerint nem éri meg az extra költséget.

2.17. Példa. Kis maradék. Tekintsük a

$$Ax = \begin{bmatrix} 0{,}913 & 0{,}659 \\ 0{,}457 & 0{,}330 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0{,}254 \\ 0{,}127 \end{bmatrix} = b$$

lineáris rendszert a 2.8. Példából. Négyjegyű decimális aritmetika használatával a Gauss-kiküszöbölés a

$$\begin{bmatrix} 0{,}9130 & 0{,}6590 \\ 0 & 0{,}0002 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0{,}2540 \\ -0{,}0001 \end{bmatrix}$$

háromszögű rendszert adja, majd a visszahelyettesítés az

$$\hat{x} = \begin{bmatrix} 0{,}6391 \\ -0{,}5 \end{bmatrix}$$

kiszámított megoldást szolgáltatja. Amint a 2.8. Példában láttuk, ennek a megoldásnak a maradéknormája $\|r\|_1 = 7{,}04 \times 10^{-5}$, ami olyan kicsi, amilyenre négyjegyű aritmetikával számíthatunk. Mégis a rendszer pontos megoldása könnyen ellenőrizhetően $x = [1, -1]^T$, így a hiba ugyanakkora, mint a megoldás. Ennek a jelenségnek az oka az, hogy az A mátrix közel szinguláris: amint a 2.7. Példában láttuk, kondíciószáma nagyobb, mint 10^4. Az x_2-t meghatározó osztás két olyan mennyiség között történik, amelyek mindegyike a kerekítési hiba nagyságrendjébe esik (négyjegyű aritmetikában), és ezért az eredmény lényegében tetszőleges. Mégis, konstrukció szerint, amikor ezt az x_2-re vonatkozó tetszőleges értéket ezután behelyettesítjük az első

egyenletbe, egy olyan x_1 érték adódik, amelyre az első egyenlet teljesül. Így kicsi maradékot, de rossz megoldást kapunk.

Amint az imént láttuk, a Gauss-kiküszöbölés stabilitásához általában szükség van a főelemkiválasztásra. Vannak azonban olyan mátrixosztályok, amelyekre a Gauss-kiküszöbölés főelemkiválasztás nélkül is stabil. Például ha az A mátrix *oszloponként diagonálisan domináns*, ami azt jelenti, hogy minden átlóbeli elem abszolút értéke nagyobb, mint az oszlopa többi eleme abszolút értékeinek összege,

$$\sum_{i=1, i\neq j}^{n} |a_{ij}| < |a_{jj}|, \quad j = 1, \dots, n,$$

akkor az LU-felbontás Gauss-kiküszöböléssel történő kiszámításához nincs szükség főelemkiválasztásra. Ha ilyen mátrixra részleges főelemkiválasztást alkalmazunk, akkor ténylegesen nem történik sorcsere. Egy másik fontos osztály, amelyre nincs szükség főelemkiválasztásra, a szimmetrikus és pozitív definit mátrixok, amelyeket a 2.5. szakaszban fogunk definiálni. A felesleges főelem-keresések elkerülése jelentős időmegtakarítást jelenthet a felbontás kiszámításában.

2.4.6 A Gauss-kiküszöbölés megvalósítása

A Gauss-kiküszöbölés, azaz az LU-felbontás, általános alakja egy háromszorosan egymásba ágyazott ciklus, amint azt a 2.5. Algoritmus sematikusan mutatja. A `for` ciklusok i, j és k indexeit tetszőleges sorrendben használhatjuk, így a ciklusok összesen $3! = 6$-féleképpen rendezhetők el. Néhány jelzett aritmetikai művelet nagyobb hatékonyság érdekében kivihető a legbelső ciklusból (mint például a 2.3. Algoritmusban), és a műveletek olyan további átrendezései is lehetségesek, amelyek esetleg nem szigorúan egymásba ágyazott ciklusokat eredményeznek. Az alapalgoritmus ezen változatainak eltérő hozzáférési mintáik (például sorkénti vagy oszloponkénti) vannak, és eltérőek abban, mennyire képesek kihasználni egy adott számítógép architekturális jellemzőit (például gyorsítótár, lapozás, vektorizáció, több processzor). Így teljesítményük adott számítógépen vagy különböző számítógépeken nagyon eltérő lehet, és nincs olyan egyetlen elrendezés, amely egyöntetűen felsőbbrendű volna.

2.5. Algoritmus. Általános Gauss-kiküszöbölés.

```
for ______

    for ______

        for ______

            aij = aij - (aik/akk) akj
        end
    end
end
```

Számos megvalósítási részlet ehhez hasonló módon változó lehet. Például az általunk leírt részleges főelemkiválasztási eljárás oszlopok mentén keres és sorokat cserél, de

alternatívaként keresni lehetne sorok mentén és oszlopokat cserélni. Úgy is vettük, hogy L-nek egységnyi átlója van, de intézhetnénk úgy is, hogy U-nak legyen egységnyi átlója. A Gauss-kiküszöbölés ezen változatai közül némelyiket olyan fontosnak tartják, hogy nevet is kaptak, mint például a Crout- és Doolittle-módszer.

Bár a Gauss-kiküszöbölés sokféle lehetséges változata drámai hatással lehet a teljesítményre, nemszinguláris A mátrix esetén mind lényegében ugyanazt a felbontást produkálják. Ha a sorok főelem-kiválasztási sorrendje ugyanaz, akkor két LU-felbontás, $PA = LU = \hat{L}\hat{U}$, esetén ez a kifejezés azt jelenti, hogy $\hat{L}^{-1}L = \hat{U}U^{-1} = D$ egyszerre alsó és felső háromszögű, és ennélfogva diagonális. Ha mind L-ről, mind $\hat{L}$-ről feltesszük, hogy egységnyi átlójú alsó háromszögmátrix, akkor D valójában az I egységmátrix kell legyen, és így $L = \hat{L}$ és $U = \hat{U}$, vagyis a felbontás egyértelmű. E feltevés nélkül azonban még mindig következtethetünk arra, hogy az LU-felbontás a tényezők diagonális skálázása erejéig egyértelmű. Ezt az egyértelműséget az LDU-felbontás, $PA = LDU$, teszi explicitté, ahol L egységnyi átlójú alsó háromszögmátrix, U egységnyi átlójú felső háromszögmátrix, D pedig diagonális.

A tárkezelés egy másik fontos megvalósítási kérdés. A tárgyalt számos mátrix – az M_k elemi eliminációs mátrixok, azok L_k inverzei és a P_k permutációs mátrixok – csupán formálisan írja le a felbontási folyamatot. A gyakorlati megvalósításban nem képezzük őket explicit módon. A tárhely megtakarítása érdekében az L és U tényezők felülírják a bemeneti A mátrix kezdeti tárterületét: a transzformált U mátrix A felső háromszögét (az átlót is beleértve) foglalja el, a szigorú alsó háromszögű L-t alkotó szorzótényezők pedig A (most már nulla) szigorú alsó háromszögét. L egységnyi átlóját nem kell tárolni.

Az adatmozgatás minimalizálása érdekében a főelemkiválasztás által megkövetelt sorcseréket rendszerint nem végezzük el ténylegesen. Ehelyett a sorok eredeti helyükön maradnak, és egy segéd egészértékű vektor nyilvántartja az új sorsorrendet. Megjegyzendő, hogy egyetlen ilyen vektor elegendő, mivel az összes csere nettó hatása még mindig csak az $1, \ldots, n$ egészek egy permutációja.

2.4.7 A lineáris egyenletrendszerek megoldásának komplexitása

Egy $n \times n$-es mátrix LU-felbontásának Gauss-kiküszöböléssel való kiszámítása körülbelül $n^3/3$ lebegőpontos szorzást és hasonló számú összeadást igényel. Az eredményül kapott háromszögű rendszer egyetlen jobb oldali vektorra való megoldása előre- és visszahelyettesítéssel körülbelül n^2 szorzást és hasonló számú összeadást igényel. Így ahogy a mátrix n mérete nő, a lineáris rendszerek megoldásának költségében az LU-felbontás fázisa egyre dominánsabbá válik.

Egy lineáris rendszert a mátrix explicit invertálásával is megoldhatunk, úgy, hogy a megoldást $x = A^{-1}b$ adja. De A^{-1} kiszámítása egyenértékű n darab lineáris rendszer megoldásával: A egy LU-felbontását, majd n darab előre- és visszahelyettesítést igényel, egyet az egységmátrix minden oszlopára. A teljes műveletszám körülbelül n^3 szorzás és hasonló számú összeadás (kihasználva a jobb oldali vektorok nullait az előrehelyettesítésnél). Az explicit invertálás tehát háromszor költségesebb, mint az LU-felbontás.

Az azt követő $x = A^{-1}b$ mátrix-vektor szorzás, amellyel a lineáris rendszert meg-

oldjuk, körülbelül n^2 szorzást és hasonló számú összeadást igényel, ami hasonló az előre- és visszahelyettesítés összköltségéhez. Így, még több jobb oldali vektor esetén is, a mátrixinverzió költségesebb, mint az LU-felbontás, ha lineáris rendszereket szeretnénk megoldani. Ezen felül az explicit invertálás pontatlanabb választ ad. Egyszerű példaként, ha a $3x = 18$ 1×1-es lineáris rendszert osztással oldjuk meg, akkor $x = 18/3 = 6$-ot kapunk, de az explicit invertálás $x = 3^{-1} \times 18 = 0{,}333 \times 18 = 5{,}99$-et adna háromjegyű aritmetikát használva. Ebben a kis példában az invertáláshoz egy plusz aritmetikai műveletre van szükség, és kevésbé pontos eredményt ad. Az invertálás ezen hátrányai csak súlyosbodnak, ahogy a rendszer mérete nő.

Az explicit mátrixinverzek kényelmes jelölésként gyakran előfordulnak különféle képletekben, de ez a gyakorlat nem jelenti azt, hogy egy ilyen képlet megvalósításához explicit inverzre volna szükség. Csupán megfelelő jobb oldallal – amely maga is lehet mátrix – egy lineáris rendszert kell megoldani. Így például egy $A^{-1}B$ alakú szorzatot A LU-felbontásával, majd B minden oszlopát használva elvégzett előre- és visszahelyettesítésekkel kell kiszámítani. A gyakorlatban rendkívül ritka, hogy tényleg explicit mátrixinverzre volna szükség, így amikor csak egy mátrixinverzet látsz egy képletben, mindig arra gondolj: „oldj meg egy rendszert”, ne arra, hogy „invertálj egy mátrixot”.

Egy másik lineáris rendszer megoldási módszer, amelyet kerülni kell, a Cramer-szabály, amelyben a megoldás minden komponensét determinánsok hányadosaként számítjuk ki. Bár elemi lineáris algebrai kurzusokon gyakran tanítják, ez a módszer nem triviális méretű teli mátrixok esetén csillagászatilag drága. A Cramer-szabály főként elméleti eszközként hasznos.

2.4.8 Gauss–Jordan-kiküszöbölés

A Gauss-kiküszöbölés mögötti motiváció az, hogy egy általános mátrixot háromszögalakra redukáljunk, mivel az így kapott lineáris rendszer könnyen megoldható. A diagonális lineáris rendszerek azonban még könnyebben megoldhatók, így a diagonális alak még kívánatosabb célnak tűnik. A *Gauss–Jordan-kiküszöbölés* a standard Gauss-kiküszöbölés olyan változata, amelyben a mátrixot nemcsak háromszög-, hanem diagonális alakra redukáljuk. A mátrixelemek kiküszöbölésére ugyanolyan jellegű sorkombinációkat használunk, mint a standard Gauss-kiküszöbölésben, de ezeket mind az átló alatti, mind az átló feletti elemek annullálására alkalmazzuk. Így egy adott a oszlopvektorra alkalmazott eliminációs mátrix alakja

$$\begin{bmatrix} 1 & \cdots & 0 & -m_1 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 1 & -m_{k-1} & 0 & \cdots & 0 \\ 0 & \cdots & 0 & 1 & 0 & \cdots & 0 \\ 0 & \cdots & 0 & -m_{k+1} & 1 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & -m_n & 0 & \cdots & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ \vdots \\ a_{k-1} \\ a_k \\ a_{k+1} \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ a_k \\ 0 \\ \vdots \\ 0 \end{bmatrix},$$

ahol $m_i = a_i/a_k$, $i = 1, \ldots, n$. Ez az eljárás körülbelül $n^3/2$ szorzást és hasonló számú összeadást igényel, ami 50 százalékkal drágább, mint a standard Gauss-kiküszöbölés.

A kiküszöbölési fázis során ugyanezeket a sorműveleteket alkalmazzuk a lineáris egyenletrendszer jobb oldali vektorára (vagy vektoraira) is. Ha egyszer a kiküszöbölési fázis befejeződött, és a mátrix diagonális alakba került, akkor a lineáris rendszer megoldásának komponensei egyszerűen úgy számíthatók ki, hogy a transzformált jobb oldal minden elemét a mátrix megfelelő átlóbeli elemével elosztjuk. Ez a számítás összesen csak n osztást igényel, ami jelentősen olcsóbb, mint egy háromszögű rendszer megoldása, de nem elég a költségesebb kiküszöbölési fázis kompenzálására. A Gauss–Jordan-kiküszöbölésnek az a numerikus hátránya is megvan, hogy a szorzótényezők nagysága még főelemkiválasztás esetén is meghaladhatja az 1-et.

Nagyobb összköltsége ellenére bizonyos helyzetekben a Gauss–Jordan-kiküszöbölés előnyben részesíthető, mert végső megoldási fázisa rendkívül egyszerű. Például néha párhuzamos számítógépekre való megvalósításhoz ajánlják, mivel a felbontási fázis alatt egyenletes a munkaterhelés, és a megoldás minden komponense egyszerre számítható ki – nem pedig egyesével, ahogyan a közönséges visszahelyettesítésnél.

A Gauss–Jordan-kiküszöbölést néha egy mátrix inverzének explicit kiszámítására is használják, ha arra szükség van. Ha a jobb oldali mátrixot az I egységmátrixra inicializáljuk, és az adott A mátrixot Gauss–Jordan-kiküszöböléssel az egységmátrixra redukáljuk, akkor a transzformált jobb oldali mátrix A inverze lesz. Az inverz kiszámításához a Gauss–Jordan-kiküszöbölés körülbelül ugyanannyi műveletet igényel, mint a Gauss-kiküszöböléssel végzett explicit invertálás, amelyet előre- és visszahelyettesítés követ.

2.18. Példa. Gauss–Jordan-kiküszöbölés. A Gauss–Jordan-kiküszöbölést annak használatával illusztráljuk, hogy kiszámítjuk a 2.13. Példa mátrixának inverzét. Az egyszerűség kedvéért a főelemkiválasztást kihagyjuk. Az A mátrixszal kezdünk, jobb oldalon az I egységmátrixszal bővítve, és ismételten eliminációs mátrixokat alkalmazunk A átlón kívüli elemeinek annullálására, amíg el nem érjük a diagonális alakot; majd a megmaradt átlóbeli elemekkel skálázva a bal oldalon az egységmátrixot, így a jobb oldalon az A^{-1} inverz mátrixot állítjuk elő.

$$\begin{bmatrix} 1 & 0 & 0 \\ -4 & 1 & 0 \\ -4 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 2 & 1 & 0 & 0 \\ 4 & 4 & 2 & 0 & 1 & 0 \\ 4 & 6 & 4 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 2 & 1 & 0 & 0 \\ 0 & -4 & -6 & -4 & 1 & 0 \\ 0 & -2 & -4 & -4 & 0 & 1 \end{bmatrix},$$

$$\begin{bmatrix} 1 & 0{,}5 & 0 \\ 0 & 1 & 0 \\ 0 & -0{,}5 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 2 & 1 & 0 & 0 \\ 0 & -4 & -6 & -4 & 1 & 0 \\ 0 & -2 & -4 & -4 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & -1 & -1 & 0{,}5 & 0 \\ 0 & -4 & -6 & -4 & 1 & 0 \\ 0 & 0 & -1 & -2 & -0{,}5 & 1 \end{bmatrix},$$

$$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & -6 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 & -1 & 0{,}5 & 0 \\ 0 & -4 & -6 & -4 & 1 & 0 \\ 0 & 0 & -1 & -2 & -0{,}5 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & -1 \\ 0 & -4 & 0 & 8 & 4 & -6 \\ 0 & 0 & -1 & -2 & -0{,}5 & 1 \end{bmatrix},$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -0{,}25 & 0 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & -1 \\ 0 & -4 & 0 & 8 & 4 & -6 \\ 0 & 0 & -1 & -2 & -0{,}5 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & -1 \\ 0 & 1 & 0 & -2 & -1 & 1{,}5 \\ 0 & 0 & 1 & 2 & 0{,}5 & -1 \end{bmatrix},$$

így

$$A^{-1} = \begin{bmatrix} 1 & 1 & -1 \\ -2 & -1 & 1{,}5 \\ 2 & 0{,}5 & -1 \end{bmatrix}.$$

2.4.9 Módosított feladatok megoldása

Számos gyakorlati helyzetben a lineáris rendszerek nem elszigetelten fordulnak elő, hanem rokon feladatok olyan sorozatának részeként, amelyek valamilyen szisztematikus módon változnak. Előfordulhat például, hogy olyan $Ax = b$ lineáris rendszerek sorozatát kell megoldani, amelyeknek ugyanaz az A mátrixuk, de eltérőek a b jobb oldalaik. Az első rendszer Gauss-kiküszöböléssel való megoldása után a már kiszámított L és U tényezőket fel lehet használni a további rendszereknek előre- és visszahelyettesítéssel való megoldására. A felbontási fázist a későbbi lineáris rendszerek megoldása során nem kell megismételni, hacsak a mátrix nem változik. Ez az eljárás jelentős munkamegtakarítást jelent, mivel a további háromszögű megoldások költsége csupán $\mathcal{O}(n^2)$, ellentétben egy felbontás $\mathcal{O}(n^3)$ költségével.

Sőt: bizonyos fontos különleges esetekben még akkor is elkerülhető egy új felbontás, ha a mátrix változik. Ilyen, gyakran előforduló eset egy olyan $n \times n$-es mátrix hozzáadása vagy kivonása, amely két nemnulla n-dimenziós vektor, u és v, uv^T külső szorzata. Ezt *egyrangú módosításnak* nevezzük, mert az uv^T külső szorzatmátrix rangja egy (azaz csak egy lineárisan független sora vagy oszlopa van), és bármely egyrangú mátrix kifejezhető két vektor ilyen külső szorzataként (lásd a 2.25. Feladatot). Például ha az A mátrix egyetlen eleme változik meg – például a (j, k) elem a_{jk}-ról $\tilde{a}_{jk}$-ra –, akkor az új mátrix $A - \alpha e_j e_k^T$, ahol e_j és e_k az egységmátrix megfelelő oszlopai, és $\alpha = a_{jk} - \tilde{a}_{jk}$.

A Sherman–Morrison-formula,

$$(A - uv^T)^{-1} = A^{-1} + A^{-1}u(1 - v^T A^{-1} u)^{-1} v^T A^{-1},$$

ahol u és v n-dimenziós vektorok, megadja egy olyan mátrix inverzét, amely egy olyan mátrix egyrangú módosításából származik, amelynek inverze már ismert; ez közvetlen szorzással könnyen igazolható (lásd a 2.27. Feladatot). Ennek a képletnek a kiértékelése csak $\mathcal{O}(n^2)$ munkát (mátrix-vektor szorzásokat) igényel, szemben a módosított mátrix nulláról való invertálásához szükséges $\mathcal{O}(n^3)$ munkával.

Az $(A - uv^T)x = b$ új mátrixú lineáris rendszerre a Sherman–Morrison-formula az

$$x = (A - uv^T)^{-1} b = A^{-1}b + A^{-1}u(1 - v^T A^{-1} u)^{-1} v^T A^{-1} b$$

megoldást adja, de kerüljük az explicit inverzeket. Ha van LU-felbontásunk az eredeti A mátrixra, akkor a módosított rendszer megoldása a 2.6. Algoritmussal számítható, amely háromszögű rendszerek megoldását és vektorok belső szorzatának kiszámítását tartalmazza, így nem igényel explicit inverzet, és csak $\mathcal{O}(n^2)$ munkát igényel. Megjegyzendő, hogy az első lépés b-től független, és ezért nem kell megismételni, ha több b jobb oldali vektor van.

2.6. Algoritmus. Megoldás egyrangú frissítése.

```
Oldjuk meg Az = u-t z-re, így z = A^{-1} u
Oldjuk meg Ay = b-t y-ra, így y = A^{-1} b
x = y + ((v^T y) / (1 - v^T z)) z
```

Hasonló technikákkal az inverz vagy a megoldás helyett a felbontást is frissíthetjük. Ezeknek a frissítési képleteknek a használatakor azonban óvatosnak kell lennünk, mert a mátrix változása során az egymást követő frissítésekben általánosságban nincs garancia a numerikus stabilitásra. A Woodbury-formula,

$$(A - UV^T)^{-1} = A^{-1} + A^{-1}U(I - V^T A^{-1} U)^{-1} V^T A^{-1},$$

ahol U és V $n \times k$-as mátrixok, általánosítja a Sherman–Morrison-formulát a mátrix egy k-rangú módosítására (lásd a 2.28. Feladatot).

2.19. Példa. Megoldás egyrangú frissítése. Az egyrangú frissítést a

$$\begin{bmatrix} 1 & 2 & 2 \\ 4 & 4 & 2 \\ 4 & 4 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \\ 10 \end{bmatrix}$$

lineáris rendszer megoldásán illusztráljuk, amely a 2.13. Példa rendszerének egyrangú módosítása, mivel csak az A mátrix $(3, 2)$ eleme változott 6-ról 4-re. A frissítővektorok egyik választási lehetősége

$$u = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad \text{és} \quad v = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix},$$

így a módosított rendszer mátrixa $A - uv^T =$

$$\begin{bmatrix} 1 & 2 & 2 \\ 4 & 4 & 2 \\ 4 & 6 & 4 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 2 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 2 \\ 4 & 4 & 2 \\ 4 & 6 & 4 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 2 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 2 \\ 4 & 4 & 2 \\ 4 & 4 & 4 \end{bmatrix},$$

a b jobb oldali vektor pedig nem változott.

A 2.13. Példában korábban kiszámított LU-felbontást felhasználva megoldhatjuk $Az = u$-t, így $z = \begin{bmatrix} -1 & 1{,}5 & -1 \end{bmatrix}^T$, és $Ay = b$-t már megoldottuk, így $y = \begin{bmatrix} -1 & 3 & -1 \end{bmatrix}^T$. Az utolsó lépés ezek után a frissített megoldás kiszámítása:

$$x = y + \frac{v^T y}{1 - v^T z} z = \begin{bmatrix} -1 \\ 3 \\ -1 \end{bmatrix} + \frac{6}{1-3} \begin{bmatrix} -1 \\ 1{,}5 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ -1{,}5 \\ 2 \end{bmatrix}.$$

Így a módosított rendszer megoldását anélkül számítottuk ki, hogy a módosított mátrixot újra felbontottuk volna.

2.4.10 A pontosság javítása

Bár egy lineáris rendszer megoldásában várható pontosság látszólag kőbe vésett, bizonyos esetekben a pontosság javítható a rendszer átskálázásával vagy a kezdetben kiszámított megoldás iteratív javításával. Ezek az eszközök nem mindig vihetők végghez, de kipróbálásuk megérheti, különösen erősen rosszul kondicionált rendszerek esetén.

A 2.10. Példából emlékezhetünk rá, hogy egy lineáris rendszer diagonális skálázása vagy változatlanul hagyja a megoldást (sorkénti skálázás), vagy úgy változtatja meg, hogy a megoldás könnyen visszanyerhető (oszloponkénti skálázás). A gyakorlatban azonban a skálázás befolyásolja a rendszer kondicionáltságát és a Gauss-kiküszöbölésben a főelemek kiválasztását, amelyek mindegyike viszont a kiszámított megoldás pontosságát befolyásolja. Így egy lineáris rendszer sorkénti és oszloponkénti skálázása lehetőséget adhat a numerikus stabilitás és pontosság javítására (vagy rontására).

A pontosság rendszerint javul, ha a mátrix minden eleme körülbelül azonos nagyságrendű – vagy ami még jobb, ha a mátrixelemekben lévő bizonytalanságok is körülbelül azonos méretűek. Néha puszta ránézésre is nyilvánvaló, hogyan skálázzuk a mátrixot úgy, hogy ilyen egyensúly álljon elő: az egyes változókhoz tartozó mértékegységek megválasztásával és az egyes egyenleteknek a viszonylagos fontosságuk és bizonytalanságuk szerinti súlyozásával. Olyan általános, automatikus technikát azonban még soha nem fejlesztettek ki, amely hatékony és megbízható módon optimális skálázást eredményezne. Ráadásul maga a skálázási folyamat is bevezethet kerekítési hibákat, hacsak nem vigyázunk (például ha csak az aritmetikai alap hatványait használjuk skálázási tényezőként).

2.20. Példa. Rossz skálázás. Egyszerű példaként az

$$\begin{bmatrix} 1 & 0 \\ 0 & \epsilon \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ \epsilon \end{bmatrix}$$

lineáris rendszer kondíciószáma $1/\epsilon$, és ezért nagyon rosszul kondicionált, ha ϵ nagyon kicsi. Ez a rossz kondicionáltság azt jelenti, hogy a bemeneti adatok kis perturbációi viszonylag nagy változásokat okozhatnak a megoldásban. Például ha a jobb oldalt a $\begin{bmatrix} 0 & -\epsilon \end{bmatrix}^T$ vektorral perturbáljuk, a megoldás $\begin{bmatrix} 1 & 1 \end{bmatrix}^T$-ról $\begin{bmatrix} 1 & 0 \end{bmatrix}^T$-ra változik. Ha viszont a második sort előbb $1/\epsilon$-nal szorozzuk, akkor a rendszer tökéletesen jól kondicionálttá válik, és ugyanaz a perturbáció most a megoldásban ezzel arányos kis változást eredményez. Így a látszólagos rossz kondicionáltság pusztán a rossz skálázásnak volt köszönhető. Sajnos az, hogy általános mátrixok esetén hogyan korrigáljuk a rossz skálázást, sokkal kevésbé kézenfekvő.

Az *iteratív finomítás* egy másik eszköz, amellyel egy kiszámított megoldás pontossága esetleg javítható. Tegyük fel, hogy az $Ax = b$ lineáris rendszerre kiszámítottunk egy közelítő x_0 megoldást – például valamilyen LU-felbontási alakkal –, és kiszámítjuk a

$$r_0 = b - Ax_0$$

maradékot. Természetesen azt szeretnénk, ha a maradék megfelelően kicsi volna, ha viszont nem az, akkor a korábban kiszámított LU-tényezőkkel megoldhatjuk az

$$As_0 = r_0$$

lineáris rendszert s_0-ra, és az

$$x_1 = x_0 + s_0$$

értéket vesszük új, „jobb” közelítő megoldásnak, mivel

$$Ax_1 = A(x_0 + s_0) = Ax_0 + As_0 = (b - r_0) + r_0 = b.$$

Ez az eljárás ismételhető a megoldás egymás utáni finomítására, konvergenciáig, esetleg olyan megoldást előállítva, amelynek maradéka olyan kicsi, amennyire az alkalmazott aritmetikai pontosságon csak lehet.

Sajnos az iteratív finomítás dupla tárhelyet igényel, mivel mind az eredeti mátrixra, mind annak LU-felbontására szükség van (rendre a maradék kiszámításához és a későbbi rendszerek megoldásához). Ráadásul ahhoz, hogy az iteratív finomítás maximális haszonnal járjon, a maradékot rendszerint a kezdeti megoldás kiszámításához használt pontosságnál nagyobb pontossággal kell kiszámítani (emlékezzünk az 1.17. Példára).

Ezen okok miatt az iteratív finomítás gyakran nem praktikus rutinszerű használatra, bizonyos körülmények között azonban mégis hasznos lehet. Az iteratív finomítással például visszanyerhető a teljes pontosság a rosszul skálázott rendszerek esetén, és olykor stabilizálhatók vele az egyébként potenciálisan instabil megoldási módszerek. Ironikus módon, ha a kezdeti megoldás viszonylag rossz, akkor a maradék elegendően nagy lehet ahhoz, hogy extra pontosság nélkül is kielégítő pontossággal legyen kiszámítható. Az iteratív finomításra a 11.7. Példában térünk vissza.

2.5 Lineáris egyenletrendszerek speciális típusai

Eddig azt feltételeztük, hogy a lineáris rendszer mátrixa általános és *sűrű*, azaz a mátrix bejegyzéseinek lényegében mindegyike nemnulla. Ha a mátrix rendelkezik valamilyen speciális tulajdonsággal, akkor a lineáris rendszer megoldásakor gyakran műveletet és tárhelyet lehet megtakarítani. A kihasználható speciális tulajdonságokra néhány példa a következő:

- Szimmetrikus: $\mathbf{A} = \mathbf{A}^T$, azaz $a_{ij} = a_{ji}$ minden i, j esetén.
- Pozitív definit: $\mathbf{x}^T\mathbf{A}\mathbf{x} > 0$ minden $\mathbf{x} \neq \mathbf{0}$ vektorra.
- Sávos: $a_{ij} = 0$ minden $|i - j| > \beta$ esetén, ahol β az A mátrix sávszélessége. Fontos speciális eset a tridiagonális mátrix, amelyre $\beta = 1$.
- Ritka: az A bejegyzéseinek többsége nulla.

A szimmetrikus és sávos rendszerek kezelésének technikái a sűrű rendszerekre alkalmazott Gauss-kiküszöbölés viszonylag egyszerű változatai. Az általánosabb nemnulla-mintázatú ritka lineáris rendszerek ezzel szemben olyan kifinomultabb algoritmusokat és adatszerkezeteket igényelnek, amelyek elkerülik a mátrixban lévő nullák tárolását és a velük végzett műveleteket (lásd a 11.4.1. szakaszt).

A valós mátrixokra az imént definiált tulajdonságoknak komplex mátrixok esetén is megvannak a megfelelőik, de a komplex esetben a szokásos mátrixtranszponált helyébe a konjugált transzponált lép, amelyet H felső indexszel jelölünk. Ha $z = \alpha + i\beta$ komplex szám, ahol α és β valós számok és $i = \sqrt{-1}$, akkor a komplex konjugáltját a $\bar{z} = \alpha - i\beta$ definíció adja (lásd az 1.3.11. szakaszt). Egy A mátrix konjugált transzponáltját ezek után az $\{A^H\}_{ij} = \bar{a}_{ji}$ képlet adja meg. Természetesen egy valós A mátrixra $A^H = A^T$. Egy komplex mátrix *hermitikus*, ha $A = A^H$, és pozitív definit, ha $x^H Ax > 0$ minden $x \neq 0$ komplex vektorra.

2.5.1 Szimmetrikus pozitív definit rendszerek

Ha az A mátrix szimmetrikus és pozitív definit, akkor az LU-felbontás úgy is elrendezhető, hogy $U = L^T$ legyen, azaz $A = LL^T$, ahol L alsó háromszögmátrix pozitív diagonális bejegyzésekkel (de általában nem egységdiagonálissal). Ezt nevezik az A Cholesky-felbontásának, és egy algoritmus a kiszámítására abból kapható, ha az A és a LL^T megfelelő bejegyzéseit egyenlővé tesszük, majd az L bejegyzéseit a helyes sorrendben állítjuk elő. Például a 2×2-es esetben

$$\begin{bmatrix} a_{11} & a_{21} \\ a_{21} & a_{22} \end{bmatrix} = \begin{bmatrix} \ell_{11} & 0 \\ \ell_{21} & \ell_{22} \end{bmatrix} \begin{bmatrix} \ell_{11} & \ell_{21} \\ 0 & \ell_{22} \end{bmatrix},$$

amiből adódik, hogy

$$\ell_{11} = \sqrt{a_{11}}, \quad \ell_{21} = a_{21}/\ell_{11}, \quad \ell_{22} = \sqrt{a_{22} - \ell_{21}^2}.$$

Az ezáltal nyert általános eljárás egyik lehetséges megszervezése a 2.7. algoritmus, amelyben a Cholesky-faktor L felülírja az eredeti A mátrixot.

A Cholesky-felbontási algoritmusnak számos olyan tulajdonsága van, amely nagyon vonzóvá és népszerűvé teszi szimmetrikus pozitív definit mátrixok esetén:

- A szükséges n négyzetgyökvonás mindegyike pozitív számra történik, így az algoritmus jól definiált.
- Numerikus stabilitás céljából nincs szükség főelemkiválasztásra.

2.7. Algoritmus. Cholesky-felbontás.

```
for k = 1 to n
    akk =
          √
            akk
    for i = k + 1 to n
```

```
        aik = aik/akk
    end
    for j = k + 1 to n
        for i = j to n
            aij = aij - aik · ajk
        end
    end
end
                                        { ciklus az oszlopokon }
                                        { az aktuális oszlop skálázása }
                                        { minden hátralévő oszlopból
                                             kivonjuk az aktuális oszlop
                                             egy megfelelő többszörösét }
```

- Csak az A alsó háromszöge kerül elérésre, ezért a szigorú felső háromszög-részét nem kell tárolni.
- Csak körülbelül $n^3/6$ szorzásra és hozzávetőleg ugyanennyi összeadásra van szükség.

Így a Cholesky-felbontás csak körülbelül feleannyi műveletet és feleannyi tárhelyet igényel, mint egy általános mátrix Gauss-kiküszöböléssel való LU-felbontása. Sajnos ahhoz, hogy a tárhelynek ezt a megtakarítását ki is használjuk, rendszerint szükség van arra, hogy a szimmetrikus mátrix egyik háromszögét egy egydimenziós tömbbe csomagoljuk, ami kényelmetlenebb, mint a mátrixok szokásos kétdimenziós tárolása. Ezért a lineáris algebrai szoftvercsomagok szimmetrikus mátrixokra általában mind a csomagolt tárolású, mind a szabványos kétdimenziós tömbtárolású változatot felkínálják, hogy a felhasználó a kényelem és a tárhelytakarékosság között választhasson.

Bizonyos körülmények között előnyös lehet a Cholesky-felbontást $A = LDL^T$ alakban felírni, ahol L egységdiagonálisú alsó háromszögmátrix, D pedig pozitív diagonális bejegyzésű diagonális mátrix. Egy ilyen felbontás a szabványos Cholesky-algoritmus egy egyszerű változatával számítható ki, és megvan az az előnye, hogy nem igényel négyzetgyökvonást. A LDL^T-felbontásban a D diagonális bejegyzései egyszerűen a LL^T-felbontásban szereplő L diagonális bejegyzéseinek négyzetei.

2.21. Példa. Cholesky-felbontás. Az algoritmus szemléltetésére kiszámítjuk a szimmetrikus pozitív definit

$$\mathbf{A} = \begin{bmatrix} 3 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 3 \end{bmatrix}$$

mátrix Cholesky-felbontását. Mivel az algoritmus csak a mátrix alsó háromszögét érinti, a mátrix alsó háromszögének egymást követő transzformációit mutatjuk be. Az első oszlopot a diagonális bejegyzésének négyzetgyökével, $\sqrt{3} \approx 1{,}7321$-gyel elosztva

$$\begin{bmatrix} 1{,}7321 & & \\ -0{,}5774 & 3 & \\ -0{,}5774 & -1 & 3 \end{bmatrix}$$

adódik. A második oszlopot úgy módosítjuk, hogy kivonjuk belőle a $(2,1)$ bejegyzésnek, $-0{,}5774$-nek, az első oszlop megfelelő részével vett szorzatát, a harmadik oszlopot pedig úgy, hogy kivonjuk belőle a $(3,1)$ bejegyzésnek, szintén $-0{,}5774$-nek, az első oszlop megfelelő részével vett szorzatát, ami

$$\begin{bmatrix} 1{,}7321 & & \\ -0{,}5774 & 2{,}6667 & \\ -0{,}5774 & -1{,}3333 & 2{,}6667 \end{bmatrix}$$

eredményre vezet. A második oszlopot ezután elosztjuk a diagonális bejegyzésének négyzetgyökével, $\sqrt{2{,}6667} \approx 1{,}6330$-cal, amivel

$$\begin{bmatrix} 1{,}7321 & & \\ -0{,}5774 & 1{,}6330 & \\ -0{,}5774 & -0{,}8165 & 2{,}6667 \end{bmatrix}$$

adódik. A harmadik oszlopot úgy módosítjuk, hogy kivonjuk belőle a $(3,2)$ bejegyzésnek, $-0{,}8165$-nek, a második oszlop megfelelő részével vett szorzatát, ami

$$\begin{bmatrix} 1{,}7321 & & \\ -0{,}5774 & 1{,}6330 & \\ -0{,}5774 & -0{,}8165 & 2{,}0000 \end{bmatrix}$$

eredményre vezet. A harmadik diagonális bejegyzés négyzetgyökét véve a végeredmény

$$\mathbf{L} = \begin{bmatrix} 1{,}7321 & & \\ -0{,}5774 & 1{,}6330 & \\ -0{,}5774 & -0{,}8165 & 1{,}4142 \end{bmatrix}.$$

2.5.2 Szimmetrikus indefinit rendszerek

Ha az A mátrix szimmetrikus, de indefinit (azaz $x^T Ax$ x-től függően pozitív és negatív értékeket is felvehet), akkor a Cholesky-felbontás nem alkalmazható, és a numerikus stabilitáshoz általában valamilyen főelemkiválasztásra van szükség. Nyilvánvaló, hogy ha meg akarjuk őrizni a mátrix szimmetriáját, a főelemkiválasztásnak szimmetrikusnak kell lennie, azaz PAP^T alakúnak, ahol P permutációs mátrix.

Szeretnénk olyan $PAP^T = LDL^T$ alakú felbontást kapni, ahol L egységdiagonálisú alsó háromszögmátrix és D diagonális. Sajnos diagonális D-vel ilyen felbontás nem feltétlenül létezik, és létezése esetén is általában nem számítható ki stabilan pusztán szimmetrikus főelemkiválasztással. A legjobb, amit tehetünk, hogy D-t vagy tridiagonálisnak, vagy 1×1-es és 2×2-es diagonális blokkokkal rendelkező blokkdiagonálisnak választjuk. (Blokkmátrixnak nevezünk egy olyan mátrixot, amelynek bejegyzéseit kompatibilis méretű részmátrixokra, azaz „blokkokra” osztjuk. Egy blokkdiagonális mátrixban ezek közül a részmátrixok közül mindegyik nulla, kivéve a fő blokkdiagonálisban lévőket.)

Hatékony algoritmusokat dolgozott ki Aasen a tridiagonális felbontásra, valamint Bunch és Parlett (Bunch és Kaufman, illetve mások későbbi továbbfejlesztéseivel a főelemkiválasztási eljárásban) a blokkdiagonális felbontásra (lásd [198]). Mindkét esetben a főelemkiválasztási eljárás olyan stabil felbontást ad, amely csak körülbelül $n^3/6$ szorzást és hozzávetőleg ugyanennyi összeadást igényel. Mindkét esetben a rákövetkező megoldási szakasznak is csak $\mathcal{O}(n^2)$ munkára van szüksége. Így a szimmetrikus indefinit rendszerek megoldásának költsége hasonló a pozitív definit rendszerek Cholesky-felbontással való megoldásának költségéhez, és csak körülbelül a fele a nemszimmetrikus rendszerek Gauss-kiküszöböléssel való megoldásának költségének.

2.5.3 Sávos rendszerek

A Gauss-kiküszöbölés sávmátrixokra alig különbözik az általános esettől: az egyedüli algoritmikus változtatások a ciklusok tartományaiban vannak. Természetesen olyan adatszerkezetet érdemes használni a sávmátrixhoz, amely elkerüli a sávon kívüli nulla bejegyzések tárolását. Gyakori választás, amikor a sáv sűrű, hogy a mátrixot kétdimenziós tömbben, diagonálisonként tároljuk. Ha numerikus stabilitáshoz szükség van főelemkiválasztásra, akkor az algoritmus kissé bonyolultabbá válik annyiban, hogy a sávszélesség növekedhet (de legfeljebb a kétszeresére). Így egy tetszőleges sávszélességű sávos rendszereket kezelő általános célú megoldó igen hasonló egy általános mátrixokra szolgáló Gauss-kiküszöbölési kódhoz.

Rögzített kis sávszélesség esetén azonban egy sávos rendszermegoldó rendkívül egyszerű lehet, különösen ha a stabilitáshoz nincs szükség főelemkiválasztásra. Tekintsük például a tridiagonális

$$\mathbf{A} = \begin{bmatrix} b_1 & c_1 & 0 & \cdots & 0 \\ a_2 & b_2 & c_2 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & a_{n-1} & b_{n-1} & c_{n-1} \\ 0 & \cdots & 0 & a_n & b_n \end{bmatrix}$$

mátrixot. Ha a stabilitáshoz nincs szükség főelemkiválasztásra – ami a gyakorlatban előforduló tridiagonális rendszerek esetén gyakran teljesül (például ha a mátrix diagonálisan domináns vagy pozitív definit) –, akkor a Gauss-kiküszöbölés a 2.8. algoritmusra redukálódik, és az A mátrix keletkező háromszögű faktorait a következő képletek adják:

$$L = \begin{bmatrix} 1 & 0 & \cdots & \cdots & 0 \\ m_2 & 1 & \ddots & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & m_{n-1} & 1 & 0 \\ 0 & \cdots & 0 & m_n & 1 \end{bmatrix}, \quad U = \begin{bmatrix} d_1 & c_1 & 0 & \cdots & 0 \\ 0 & d_2 & c_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & d_{n-1} & c_{n-1} \\ 0 & \cdots & \cdots & 0 & d_n \end{bmatrix}.$$

Általában egy β sávszélességű sávos rendszer csak $\mathcal{O}(\beta n)$ tárhelyet igényel, a felbontás pedig csak $\mathcal{O}(\beta^2 n)$ munkát; mindkettő jelentős megtakarítást jelent a telt rendsze-

rekhez képest, ha $\beta \ll n$.

2.8. Algoritmus. Tridiagonális LU-felbontás főelemkiválasztás nélkül.

```
d1 = b1
for i = 2 to n
    mi = ai/di-1
    di = bi - mici-1
end
                                    { ciklus az oszlopokon }
                                    { szorzó kiszámítása }
                                    { transzformáció alkalmazása }
```

2.6 Iterációs módszerek lineáris egyenletrendszerekhez

A Gauss-kiküszöbölés egy példa a lineáris rendszerek megoldására szolgáló direkt módszerre, azaz olyan módszerre, amely (pontos aritmetikát feltételezve) egy lineáris rendszer pontos megoldását véges számú lépésben állítja elő. Az iterációs módszerek ezzel szemben a megoldás egy kezdeti becsléséből indulnak ki, és azt egymást követően javítják, amíg a megoldás olyan pontos nem lesz, amilyet kívánunk. Elméletben végtelen sok iterációra lehet szükség ahhoz, hogy a pontos megoldáshoz konvergáljon, a gyakorlatban azonban az iterációkat akkor állítjuk le, amikor a $\|b - Ax\|$ maradék – vagy a hiba valamilyen más mérőszáma – olyan kicsi, amilyet kívánunk. Egyes feladattípusok esetén az iterációs módszereknek jelentős előnyeik lehetnek a direkt módszerekkel szemben. A lineáris egyenletrendszerek iterációs módszereinek tárgyalását a 11. fejezetre halasztjuk, ahol a parciális differenciálegyenletek numerikus megoldását tekintjük, ami olyan ritka lineáris rendszerekre vezet, amelyeket gyakran iterációs módszerekkel lehet a legjobban megoldani.

2.7 Szoftverek lineáris egyenletrendszerekhez

Szinte minden tudományos számítási szoftverkönyvtár tartalmaz különféle típusú lineáris rendszerek megoldására szolgáló rutinokat. A 2.1. táblázat valós, általános, sűrű lineáris rendszerek megoldására, valamint a kondíciószám becslésére szolgáló megfelelő rutinok listáját tartalmazza néhány széles körben elérhető szoftvergyűjteményből. Egyes csomagok a rutinnevekben különböző elő- vagy utótagokat használnak az adattípus jelölésére, tipikusan az `s` az egyszeres pontosságú valós, `d` a dupla pontosságú valós, `c` az egyszeres pontosságú komplex és `z` a dupla pontosságú komplex változatra utal; itt csak az egyszeres pontosságú valós változatokat soroljuk fel. A legtöbb ilyen szubrutinkönyvtárban speciálisabb rutinok is rendelkezésre állnak a lineáris rendszerek bizonyos típusaira, mint a szimmetrikus, pozitív definit, sávos rendszerek vagy ezek kombinációi. Ezek közül a rutinok közül néhányat a 2.2. táblázat sorol fel; más rutinok is rendelkezésre állhatnak, amelyek tárhelytakarékosabbak, vagy egyéb speciális feladatokra készültek.

Az $Ax = b$ lineáris rendszerek megoldására szolgáló hagyományos szoftver olykor egyetlen rutinként van megvalósítva, de felosztható két rutinra is: az egyik a felbontás kiszámítására, a másik a keletkező háromszögű rendszer megoldására. Mindkét esetben a felbontás megismétlésére nincs szükség, ha ugyanazzal a mátrixszal, de más

jobb oldalakkal szeretnénk további megoldásokat. A tipikusan megkövetelt bemenet a következőket tartalmazza: egy kétdimenziós tömb az A mátrixszal, egy egydimenziós tömb a b jobb oldali vektorral (vagy egy kétdimenziós tömb több jobb oldali vektorra), a rendszer egész n rendje, az A-t tartalmazó tömb vezető dimenziója (hogy a szubrutin helyesen tudja értelmezni a tömbindexeket), valamint esetenként némi munkaterület és egy zászló az elvégzendő konkrét feladat jelzésére. Visszatéréskor az x megoldás rendszerint felülírja a b tárhelyét, a felbontás pedig az A tárhelyét. További kimenetként szerepelhet egy állapotjelző zászló a hibák vagy figyelmeztetések jelzésére, valamint a mátrix kondíciószámának egy becslése (vagy olykor a kondíciószám reciproka). A kondíciószám becslésének többletköltsége miatt ez a szolgáltatás általában opcionális.

Forrás	Felbontás	Megoldás	Kondícióbecslés
[152]FMM	decomp	solve	
GSL	gsllinalg	gsllinalg	
	LUdecomp	LUsolve	
HSL	ma21	ma21	
IMSL	lftrg	lfsrg	lfcrg
[262]KMN	sgefs	sgefs	sgefs
[9]LAPACK	sgetrf	sgetrs	sgecon
[116]LINPACK	sgefa	sgesl	sgeco
MATLAB	lu	\	rcond/condest
NAG	f07adf	f07aef	f07agf
[220]NAPACK	fact	solve	con
[377]NR	ludcmp	lubksb	
[297]NUMAL	dec	sol	
SciPy	linalg.	linalg.	numpy.linalg.
	lufactor	lusolve	cond
SLATEC	sgefa	sgesl	sgeco

2.1. táblázat: Szoftverek általános lineáris rendszerek megoldására

Lineáris rendszerek megoldása interaktív környezetben, például MATLAB-ban egyszerűbb, mint hagyományos szoftverrel, mert a csomag belsőleg nyilvántartja az olyan részleteket, mint a vektorok és mátrixok méretei, és sok mátrixművelet be van építve a nyelv szintaxisába és szemantikájába. Például az $Ax = b$ lineáris rendszer megoldását MATLAB-ban a „bal oldali osztás” operátor adja, amelyet fordított törtvonal jelöl, tehát `x = A \ b`. A megoldást belsőleg LU-felbontással, majd előre- és visszahelyettesítéssel számítja a rendszer, de a felhasználónak erről nem kell tudnia. Az LU-felbontás kifejezetten is kiszámítható, ha szükséges, a MATLAB `lu` függvényével: `[L, U] = lu(A)`, illetve ha a mátrix szimmetrikus és pozitív definit, akkor a Cholesky-felbontását az `L = chol(A)` adja.

2.7.1 LINPACK és LAPACK

A LINPACK átfogó szoftvercsomag sokféle lineáris egyenletrendszer megoldására, az általános sűrű rendszereken kívül olyanokat is beleértve, amelyeknek különféle spe-

ciális tulajdonságaik vannak, például szimmetrikusak vagy sávosak. A lineáris rendszerek megoldása annyira alapvető fontosságú a tudományos számításokban, hogy a LINPACK a számítógépek teljesítményének összehasonlítására szolgáló szabványos benchmarkká vált. A LINPACK kézikönyv [116] hasznos forrás gyakorlati tanácsokhoz a lineáris egyenletrendszerek megoldásához.

	Szimmetrikus	Szimmetrikus	Általános
Forrás	pozitív definit	indefinit	sávos
GSL	gsllinalg	gsllinalg	
	choleskydecomp	mcholeskydecomp	
HSL	ma22	ma29	ma35
IMSL	lftds/lfsds	lftsf/lfssf	lftrb/lfsrb
[9]LAPACK	spotrf/spotrs	ssytrf/ssytrs	sgbtrf/sgbtrs
[116]LINPACK	spofa/sposl	ssifa/ssisl	sgbfa/sgbsl
NAG	f07fdf/f07fef	f07mdf/f07mef	f07bdf/f07bef
[220]NAPACK	sfact/ssolve	ifact/isolve	bfact/bsolve
[377]NR	choldc/cholsl		bandec/banbks
[297]NUMAL	chldec2/chlsol2	decsym2/solsym2	decbnd/solbnd
SciPy	linalg.	linalg.	linalg.
	cholesky	ldl	solvebanded
SLATEC	spofa/sposl	ssifa/ssisl	sgbfa/sgbsl

2.2. táblázat: Szoftverek speciális lineáris rendszerek megoldására

A LAPACK nevű utódcsomag a teljes LINPACK-gyűjteményt frissíti, nagyobb teljesítményt nyújtva a modern számítógép-architektúrákon, egyes párhuzamos számítógépeket is beleértve. A LAPACK újabb algoritmusai sok esetben nagyobb pontosságot, robusztusságot és funkcionalitást is elérnek, mint a LINPACK-beli elődeik. A LAPACK a lineáris algebra összes jelentős számítási feladatához egyaránt tartalmaz egyszerű és szakértői meghajtókat, valamint a különféle felbontásokhoz, a háromszögű rendszerek megoldásához, a normabecsléshez, a skálázáshoz és az iteratív finomításhoz szükséges sokféle számítási és kiegészítő rutint. Mind a LINPACK, mind a LAPACK elérhető a Netlibről, és számos egyéb könyvtár és csomag lineáris rendszermegoldóit közvetlenül ezekre építik.

2.7.2 Basic Linear Algebra Subprograms (BLAS)

A LINPACK és a LAPACK magas szintű rutinjai az alacsonyabb szintű Basic Linear Algebra Subprograms (BLAS) csomagon alapulnak. A BLAS-t eredetileg arra tervezték, hogy egységbe zárja a vektorokon végzett alapvető műveleteket, hogy azok egy adott számítógép-architektúrára optimalizálhatók legyenek, miközben a magasabb szintű, őket hívó rutinok hordozhatók maradnak. Az újabb számítógép-architektúrák arra ösztönöztek, hogy magasabb szintű BLAS-t fejlesszenek ki, amely a mátrix-vektor és a mátrix-mátrix műveleteket zárja egységbe a hierarchikus memóriának – például a gyorsítótárnak, a vektorregisztereknek és a lapozásos virtuális memóriának – a jobb kihasználása érdekében. A 2.3. táblázat felsorolja az egyes szintek néhány legfontosabb BLAS-rutinját.

A jó teljesítmény kulcsa az adat-újrafelhasználás, azaz az, hogy egy adott adaton minél több aritmetikai műveletet végezzünk, miközben az a memóriahierarchia leggyorsabban elérhető részében van tárolva. A 3. szintű BLAS-nak nagyobb lehetősége van az adat-újrafelhasználásra, mert $\mathcal{O}(n^3)$ műveletet végez $\mathcal{O}(n^2)$ adaton, míg az alacsonyabb szintű BLAS esetén a műveletek száma az adatok számával arányos. A BLAS általános változatai elérhetők a Netlibről, és számos számítógépgyártó kínál egyedi változatokat, amelyek a saját rendszerükön a legjobb teljesítményre vannak optimalizálva.

Szint	TOMS#	Munka	Példák	Funkció
1	539	O(n)	saxpy	Skalár · vektor plusz vektor
			sdot	Két vektor belső szorzata
			snrm2	Vektor euklideszi normája
2	656	2O(n)	sgemv	Mátrix-vektor szorzás
			strsv	Háromszögű megoldás
			sger	Elsőrendű frissítés
3	679	3O(n)	sgemm	Mátrix-mátrix szorzás
			strsm	Többszörös háromszögű megoldás
			ssyrk	k-adrendű frissítés

2.3. táblázat: Példák az alapvető lineáris algebrai szubrutinokra (BLAS)

2.8 Történeti jegyzetek és további olvasnivaló

Gauss 1810-ben fogalmazta meg a kiküszöbölési módszerét, de nem egy mátrix felbontásaként, hanem egy kvadratikus alak egyszerűsítéseként írta le. A mátrixokat ugyanis csak 1855-ben vezette be Cayley, és a Gauss-kiküszöbölés mátrixfelbontásként való értelmezését csak az 1940-es években ismerte fel Dwyer, von Neumann és mások (lásd [431]). A digitális számítások korai úttörőit – például von Neumannt [487] és Turingot [471] (a mátrix kondíciószámának fogalma Turingtól származik) – súlyosan aggasztotta, hogy a nagy lineáris rendszerek Gauss-kiküszöböléssel történő megoldásakor felhalmozódó kerekítési hiba használhatatlanná teszi-e az eredményeket, és kezdetben jelentős pesszimizmus övezte ezt a kérdést. A számítási tapasztalatok hamarosan megmutatták azonban, hogy a módszer a gyakorlatban meglepően stabil és pontos, és a hibaelemzések végül követték a tapasztalatot, hogy magyarázatot adjanak erre a szerencsére (lásd különösen Wilkinson munkáit [499, 500, 501]).

Mint kiderült, a részleges főelemkiválasztással végzett Gauss-kiküszöbölés műveletszáma rosszabb az optimálisnál [444], a legrosszabb esetben instabil [499], és elméleti értelemben nem valósítható meg hatékonyan párhuzamosan [485]. Mégis a gyakorlatban – még párhuzamos számítógépeken is – következetesen eredményes, és a tudományos számítások egyik fő igáslova. A legtöbb numerikus algoritmus Murphy törvényének engedelmeskedik – „ha valami elromolhat, el is romlik” –, ám úgy tűnik, a Gauss-kiküszöbölés boldog kivétel. E figyelemre méltó algoritmus egyes „rejtélyeinek” további tárgyalásához lásd [461, 467]-et; „számítási ellenpéldákhoz” lásd [156, 510]-et.

A lineáris algebra hátteréhez az olvasó Strang kiváló tankönyveit [436, 441] veheti kézbe. Haladóbb mátrixelmélethez lásd [241, 242]-t. További példák, feladatok és a számítási lineáris algebra gyakorlati alkalmazásai megtalálhatók [238, 276, 314]-ben. A mátrixszámítások enciklopédikus referenciája [198]; tankönyvi tárgyalások közé tartoznak [40, 96, 104, 185, 220, 257, 426, 430, 466, 493, 496]. A lineáris rendszerek megoldásának egyik nagy hatású korai műve – és az elsők egyike, amely magas színvonalú szoftvert is tartalmaz – a [153]. A mátrixszámítások hasznos, Fortran és MATLAB nyelvű gyakorlati kézikönyve a [82].

A lineáris rendszerek és a lineáris algebra számos egyéb feladatának hibaelemzéséről és perturbációelméletéről szóló átfogó tárgyaláshoz lásd [237, 433]-at. A kondíciószám-becslésről áttekintést ad [235]. A lineáris algebrában a (normaszerű helyett) komponensenkénti perturbációelmélet részletes áttekintését adja [236]. A LINPACK és a LAPACK dokumentációja [116], illetve [9]. A BLAS (Basic Linear Algebra Subprograms) könyvtárról lásd [114, 115, 300]-at. A számítási környezetnek a Gauss-kiküszöbölés és más mátrixszámítások teljesítményére gyakorolt hatását vizsgáló egyik legkorábbi cikk [330] volt. Az erről a mára igen terjedelmessé vált témáról egy ízelítőhöz lásd [105, 117, 118, 357]-et.

Áttekintő kérdések

- **2.1.** Igaz vagy hamis: Ha egy A mátrix nemszinguláris, akkor az $Ax = b$ lineáris egyenletrendszer megoldásainak száma a b jobb oldali vektor konkrét választásától függ.
- **2.2.** Igaz vagy hamis: Ha egy mátrix determinánsa nagyon kicsi, akkor a mátrix majdnem szinguláris.
- **2.3.** Igaz vagy hamis: Szimmetrikus A mátrixra mindig $\|A\|_1 = \|A\|_\infty$.
- **2.4.** Igaz vagy hamis: Ha egy háromszögmátrixnak van nullával egyenlő eleme a főátlóban, akkor a mátrix szükségképpen szinguláris.
- **2.5.** Igaz vagy hamis: Ha egy tetszőleges mátrixnak van nullával egyenlő eleme a főátlóban, akkor a mátrix szükségképpen szinguláris.
- **2.6.** Igaz vagy hamis: Egy alulhatározott $Ax = b$ lineáris egyenletrendszernek, ahol A egy $m \times n$-es mátrix $m < n$-nel, mindig van megoldása.
- **2.7.** Igaz vagy hamis: Két felső háromszögmátrix szorzata felső háromszögmátrix.
- **2.8.** Igaz vagy hamis: Két szimmetrikus mátrix szorzata szimmetrikus.
- **2.9.** Igaz vagy hamis: Egy nemszinguláris felső háromszögmátrix inverze felső háromszögmátrix.
- **2.10.** Igaz vagy hamis: Ha egy $n \times n$-es A mátrix sorai lineárisan összefüggők, akkor az oszlopai is lineárisan összefüggők.
- **2.11.** Igaz vagy hamis: Az $Ax = b$ lineáris egyenletrendszernek akkor és csak akkor van megoldása, ha az $m \times n$-es A mátrix és az $m \times (n+1)$-es $[\, A\, b\,]$ kibővített mátrix rangja azonos.
- **2.12.** Igaz vagy hamis: Ha A tetszőleges $n \times n$-es mátrix és P tetszőleges $n \times n$-es permutációs mátrix, akkor $PA = AP$.
- **2.13.** Igaz vagy hamis: Ha sorcserék megengedettek, akkor az LU-felbontás mindig létezik, még szinguláris A mátrixra is.
- **2.14.** Igaz vagy hamis: Ha egy lineáris rendszer jól kondicionált, akkor a Gauss-

kiküszöbölésben nincs szükség főelemkiválasztásra.

- **2.15.** Igaz vagy hamis: Ha egy mátrix szinguláris, akkor nem lehet LU-felbontása.
- **2.16.** Igaz vagy hamis: Ha egy nemszinguláris szimmetrikus mátrix nem pozitív definit, akkor nem lehet Cholesky-felbontása.
- **2.17.** Igaz vagy hamis: Egy szimmetrikus pozitív definit mátrix mindig jól kondicionált.
- **2.18.** Igaz vagy hamis: A főelemkiválasztás nélküli Gauss-kiküszöbölés csak akkor hiúsul meg, ha a mátrix rosszul kondicionált vagy szinguláris.
- **2.19.** Igaz vagy hamis: Ha egy mátrix LU-felbontását már kiszámítottuk egy lineáris rendszer megoldásához, akkor az ugyanazzal a mátrixszal, de eltérő jobb oldali vektorokkal adott további lineáris rendszerek a mátrix újrafelbontása nélkül megoldhatók.
- **2.20.** Igaz vagy hamis: LU-felbontáson és háromszögű megoldáson alapuló explicit mátrixinvertálásban a munka többségét a felbontás adja.
- **2.21.** Igaz vagy hamis: Ha x tetszőleges n-dimenziós vektor, akkor $\|x\|_1 \geq \|x\|_\infty$.
- **2.22.** Igaz vagy hamis: Egy szinguláris mátrix normája nulla.
- **2.23.** Igaz vagy hamis: Ha $\|A\| = 0$, akkor $A = 0$.
- **2.24.** Igaz vagy hamis: $\|A\|_1 = \|A^T\|_\infty$.
- **2.25.** Igaz vagy hamis: Ha A tetszőleges $n \times n$-es nemszinguláris mátrix, akkor $\text{cond}(A) = \text{cond}(A^{-1})$.
- **2.26.** Igaz vagy hamis: Nemszinguláris lineáris egyenletrendszer megoldásakor a részleges főelemkiválasztással végzett Gauss-kiküszöbölés általában kis maradékot ad, még ha a mátrix rosszul kondicionált is.
- **2.27.** Igaz vagy hamis: A részleges főelemkiválasztással végzett Gauss-kiküszöbölésben a szorzótényezők abszolút értékben 1-gyel korlátosak, ezért az egymást követő redukált mátrixok elemei abszolút értékben nem növekedhetnek.
- **2.28.** Lehet-e egy $Ax = b$ lineáris egyenletrendszernek pontosan két különböző megoldása?
- **2.29.** Meghatározható-e egy $Ax = b$ lineáris rendszer megoldásainak száma valaha is pusztán az A mátrixból, a b jobb oldali vektor ismerete nélkül?
- **2.30.** Egy $Ax = b$ négyzetes lineáris egyenletrendszer megoldásakor melyik volna a súlyosabb nehézség: ha A sorai lineárisan összefüggők, vagy ha A oszlopai lineárisan összefüggők? Magyarázd meg.
- **2.31.** (a) Mondj ki egy definiáló tulajdonságot egy A szinguláris mátrixra. (b) Tegyük fel, hogy az $Ax = b$ lineáris rendszernek két különböző x és y megoldása van. Az (a)-ban megadott tulajdonsággal bizonyítsd be, hogy A szükségképpen szinguláris.
- **2.32.** Adott egy nemszinguláris $Ax = b$ lineáris egyenletrendszer. Az alábbi műveletek mindegyikének milyen hatása van az x megoldásvektorra? (a) $[\,A\ b\,]$ sorainak permutálása; (b) A oszlopainak permutálása; (c) az egyenlet mindkét oldalának balról való szorzása egy M nemszinguláris mátrixszal.
- **2.33.** Tegyük fel, hogy egy $Ax = b$ lineáris egyenletrendszer mindkét oldalát megszorozzuk egy nem nulla α skalárral. (a) Megváltozik-e ettől a valódi x megoldás? (b) Megváltozik-e ettől egy adott x-re az $r = b - Ax$ maradékvektor? (c) Milyen következtetés vonható le egy számított megoldás minőségének megítéléséről?
- **2.34.** Tegyük fel, hogy egy $Ax = b$ lineáris egyenletrendszer mindkét oldalát

balról megszorozzuk egy nemszinguláris diagonális mátrixszal. (a) Megváltozik-e ettől a valódi x megoldás? (b) Befolyásolhatja-e ez a rendszer kondicionáltságát? (c) Befolyásolhatja-e ez a Gauss-kiküszöbölésben a főelemek választását?

- **2.35.** Adj meg egy elemi eliminációs mátrixot, amely az alábbi vektor utolsó két komponensét nullává teszi:

$$\begin{bmatrix} 3 \\ 2 \\ -1 \\ 4 \end{bmatrix}.$$

- **2.36.** (a) Adj meg egy 4×4-es permutációs mátrixot, amely bármely 4-dimenziós vektor $2.$ és $4.$ komponensét felcseréli. (b) Adj meg egy 4×4-es permutációs mátrixot, amely bármely 4-dimenziós vektor komponenseinek sorrendjét megfordítja.
- **2.37.** Szinguláris mátrix és egzakt aritmetika használata esetén egy lineáris rendszer Gauss-kiküszöböléssel történő megoldásában hol fog a megoldási folyamat megakadni (a) részleges főelemkiválasztással? (b) főelemkiválasztás nélkül?
- **2.38.** (a) Mi a különbség a Gauss-kiküszöbölésben a részleges és a teljes főelemkiválasztás között? (b) Mondj a másikhoz képest egy-egy előnyt mindkét típusú főelemkiválasztásra.
- **2.39.** Tekintsük az alábbi A mátrixot, amelynek LU-felbontását Gauss-kiküszöböléssel szeretnénk kiszámítani:

$$A = \begin{bmatrix} 4 & -8 & 1 \\ 6 & 5 & 7 \\ 0 & -10 & -3 \end{bmatrix}.$$

Mi lesz a kezdeti főelem, ha (a) nem használunk főelemkiválasztást? (b) részleges főelemkiválasztást használunk? (c) teljes főelemkiválasztást használunk?

- **2.40.** Adj két okot arra, hogy a Gauss-kiküszöbölés numerikusan stabil megvalósításához miért lényeges a főelemkiválasztás.
- **2.41.** Ha A egy rosszul kondicionált mátrix, és az LU-felbontását részleges főelemkiválasztással végzett Gauss-kiküszöböléssel számítjuk, azt várnád-e, hogy a rossz kondicionáltság L-ben, U-ban vagy mindkettőben tükröződik? Miért?
- **2.42.** (a) Mi az alábbi mátrix inverze?

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & m_1 & 1 & 0 \\ 0 & m_2 & 0 & 1 \end{bmatrix}$$

(b) Hogyan merülhet fel egy ilyen mátrix a számítási gyakorlatban?

- **2.43.** (a) Felírható-e minden $n \times n$-es nemszinguláris A mátrix $A = LU$ szorzat alakban, ahol L alsó háromszögmátrix és U felső háromszögmátrix? (b) Ha igen, milyen algoritmus valósítja ezt meg? Ha nem, adj ellenpéldát a szemléltetésre.

- **2.44.** Adott egy $n \times n$-es nemszinguláris A mátrix és egy további $n \times n$-es B mátrix. Mi a legjobb módja az $n \times n$-es $A^{-1}B$ mátrix kiszámításának?
- **2.45.** Ha A és B $n \times n$-es mátrixok, A nemszinguláris, és c egy n-dimenziós vektor, hogyan számítanád ki hatékonyan az $A^{-1}Bc$ szorzatot?
- **2.46.** Ha A egy $n \times n$-es mátrix és x egy n-dimenziós vektor, az alábbi számítások közül melyik igényel kevesebb munkát? Magyarázd meg. (a) $y = (x\,x^T)\,A$; (b) $y = x\,(x^T\,A)$.
- **2.47.** Hogyan viszonyul egy $n \times n$-es háromszögű lineáris egyenletrendszer megoldásához szükséges számítási munka egy általános $n \times n$-es rendszer megoldásához szükségesével?
- **2.48.** Tegyük fel, hogy egy nemszinguláris A mátrix $A = LU$ LU-felbontását már kiszámítottad. Hogyan használnád fel az $A^T x = b$ lineáris rendszer megoldásához?
- **2.49.** Ha L egy nemszinguláris alsó háromszögmátrix, P egy permutációs mátrix, és b egy adott vektor, hogyan oldanád meg az alábbi lineáris rendszerek mindegyikét? (a) $LPx = b$; (b) $PLx = b$.
- **2.50.** Az $\mathbb{R}^2$ síkban lehetséges-e olyan $x \neq 0$ vektor, amelyre $\|x\|_1 = \|x\|_\infty$? Ha igen, adj példát.
- **2.51.** Az $\mathbb{R}^2$ síkban lehetséges-e olyan x és y vektor, amelyekre $\|x\|_1 > \|y\|_1$, de $\|x\|_\infty < \|y\|_\infty$? Ha igen, adj példát.
- **2.52.** Általában melyik mátrixnormát könnyebb kiszámítani, $\|A\|_1$-et vagy $\|A\|_2$-t? Miért?
- **2.53.** (a) Egy mátrix determinánsának nagysága jó mutatója-e annak, hogy a mátrix majdnem szinguláris-e? (b) Ha igen, miért? Ha nem, mi a közelszingularitás jobb mutatója?
- **2.54.** (a) Hogyan definiáljuk egy A mátrix kondíciószámát egy adott mátrixnormára? (b) Hogyan használjuk a kondíciószámot az $Ax = b$ lineáris rendszer számított megoldásának pontosságbecslésére?
- **2.55.** Miért nem triviális feladat egy általános mátrix kondíciószámának kiszámítása?
- **2.56.** Adj példát egy 3×3-as A mátrixra – az egységmátrixtól különbözőre –, amelyre $\mathrm{cond}(A) = 1$.
- **2.57.** (a) Mi az alábbi mátrix kondíciószáma az 1-normát használva?

$$\begin{bmatrix} 4 & 0 & 0 \\ 0 & -6 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

(b) Eltér-e a válaszod, ha a ∞-normát használjuk?

- **2.58.** Tegyük fel, hogy az $n \times n$-es A mátrix tökéletesen jól kondicionált, azaz $\mathrm{cond}(A) = 1$. Az alábbi mátrixok közül melyek szükségképpen szintén ezzel a tulajdonsággal rendelkeznek? (a) cA, ahol c tetszőleges nem nulla skalár; (b) DA, ahol D egy nemszinguláris diagonális mátrix; (c) PA, ahol P tetszőleges permutációs mátrix; (d) BA, ahol B tetszőleges nemszinguláris mátrix; (e) A^{-1}, azaz A inverze; (f) A^T, azaz A transzponáltja.
- **2.59.** Legyen $A = \mathrm{diag}(1/2)$ egy $n \times n$-es diagonális mátrix, amelynek minden diagonális eleme $1/2$. (a) Mi $\det(A)$ értéke? (b) Mi $\mathrm{cond}(A)$ értéke? (c) Milyen

következtetést lehet levonni ezekből az eredményekből?

- **2.60.** Tegyük fel, hogy az $n \times n$-es A mátrix egzaktul szinguláris, de a lebegőpontos ábrázolása, $fl(A)$, nemszinguláris. Ebben az esetben mit várnál a $\text{cond}(fl(A))$ kondíciószám nagyságrendjének?
- **2.61.** Sorold be az alábbi mátrixok mindegyikét, hogy jól vagy rosszul kondicionált-e:

$$(a) \quad \begin{bmatrix} 10^{10} & 0 \\ 0 & 10^{-10} \end{bmatrix}$$
$$(b) \quad \begin{bmatrix} 10^{10} & 0 \\ 0 & 10^{10} \end{bmatrix}$$
$$(c) \quad \begin{bmatrix} 10^{-10} & 0 \\ 0 & 10^{-10} \end{bmatrix}$$
$$(d) \quad \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

- **2.62.** Az alábbiak közül melyek jó mutatói annak, hogy egy mátrix majdnem szinguláris? (a) Determinánsa kicsi. (b) Normája kicsi. (c) Normája nagy. (d) Kondíciószáma nagy.
- **2.63.** (a) Egy $Ax = b$ lineáris rendszer megoldásakor mit értünk egy $\hat{x}$ közelítő megoldás maradékán? (b) Kis relatív maradék mindig azt jelenti-e, hogy a megoldás pontos? Miért? (c) Nagy relatív maradék mindig azt jelenti-e, hogy a megoldás pontatlan? Miért?
- **2.64.** Egy 10 decimális jegy precíziójú lebegőpontos rendszerben, ha egy olyan lineáris rendszert oldunk meg részleges főelemkiválasztással végzett Gauss-kiküszöböléssel, amelynek mátrixa 10^3 kondíciószámú, és amelynek bemeneti adatai teljes gépi precízióval pontosak, körülbelül hány értékes jegyet várnál a megoldásban?
- **2.65.** Tegyük fel, hogy egy $Ax = b$ lineáris egyenletrendszert egy olyan számítógépen oldasz meg, amelynek lebegőpontos számrendszere 12 decimális jegy precíziójú, és hogy a feladat adatai teljes gépi precízióval pontosak. Körülbelül milyen nagy lehet az A mátrix kondíciószáma, mielőtt a számított x megoldás egyetlen értékes jegyet sem tartalmaz?
- **2.66.** Milyen körülmények között jelenti-e egy kis $r = b - Ax$ maradékvektor, hogy x az $Ax = b$ lineáris rendszer pontos megoldása?
- **2.67.** Legyen A egy tetszőleges négyzetes mátrix, és c egy tetszőleges skalár. Az alábbi állítások közül melyek kell szükségképpen teljesüljenek? (a) $\|cA\| = |c| \cdot \|A\|$. (b) $\text{cond}(cA) = |c| \cdot \text{cond}(A)$.
- **2.68.** (a) Mi a fő különbség a Gauss-kiküszöbölés és a Gauss–Jordan-kiküszöbölés között? (b) Mondj a másikhoz képest egy-egy előnyt mindkét típusú kiküszöbölésre.
- **2.69.** Rangsorold az alábbi módszereket az n-ed rendű általános lineáris egyenletrendszer megoldásához szükséges munka mennyisége szerint: (a) Gauss–Jordan-kiküszöbölés; (b) részleges főelemkiválasztással végzett Gauss-kiküszöbölés; (c) Cramer-szabály; (d) explicit mátrixinvertálás, majd mátrix-vektor szorzás.
- **2.70.** (a) Mennyi tárhely szükséges egy egyrangú $n \times n$-es mátrix hatékony táro-

lásához? (b) Hány aritmetikai művelet szükséges egy n-dimenziós vektor és egy egyrangú $n \times n$-es mátrix szorzatának hatékony kiszámításához?

- **2.71.** Egy $Ax = b$ lineáris rendszer megoldásához a közönséges Gauss-kiküszöbölés és a Gauss–Jordan-kiküszöbölés összehasonlításakor (a) melyiknek költségesebb a felbontása? (b) melyiknek költségesebb a visszahelyettesítése? (c) melyiknek nagyobb az összköltsége?
- **2.72.** A lineáris rendszerek megoldására szolgáló alábbi kiküszöbölési algoritmusok mindegyikére: van-e olyan főelemválasztási stratégia, amely biztosítani tudja, hogy minden szorzótényező abszolút értéke legfeljebb 1? (a) Gauss-kiküszöbölés; (b) Gauss–Jordan-kiküszöbölés.
- **2.73.** Egy A mátrix mely két tulajdonsága együtt implikálja, hogy A-nak van Cholesky-felbontása?
- **2.74.** Sorolj fel három előnyt, amelyet a Cholesky-felbontás az LU-felbontáshoz képest nyújt.
- **2.75.** Egy $n \times n$-es szimmetrikus pozitív definit mátrix Cholesky-felbontásának kiszámításához hány négyzetgyökre van szükség?
- **2.76.** Legyen $A = \{a_{ij}\}$ egy $n \times n$-es szimmetrikus pozitív definit mátrix. (a) Mi a Cholesky-tényezőjének $(1, 1)$ eleme? (b) Mi a Cholesky-tényezőjének $(n, 1)$ eleme?
- **2.77.** Mi az alábbi mátrix Cholesky-felbontása?

$$\begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix}$$

- **2.78.** (a) Lehetséges-e általában egy szimmetrikus indefinit lineáris rendszert hasonló költséggel megoldani, mint amennyibe egy szimmetrikus pozitív definit lineáris rendszer Cholesky-felbontással történő megoldása kerül? (b) Ha igen, milyen algoritmus valósítja ezt meg? Ha nem, miért?
- **2.79.** Adj két okot arra, hogy a lineáris rendszerek megoldására szolgáló iteratív finomítás miért gyakran nem valósítható meg a gyakorlatban.
- **2.80.** Tegyük fel, hogy egy $n \times n$-es $Ax = b$ lineáris rendszert LU-felbontással és visszahelyettesítéssel már megoldottál. Mi a további költsége (a nagyságrendi becslés elegendő) egy új rendszer megoldásának (a) ugyanazzal a mátrixszal, de más jobb oldali vektorral? (b) ha a mátrixot egy egyrangú mátrix hozzáadásával módosítjuk? (c) ha a mátrix teljesen megváltozik?

Feladatok

- **2.1.** A 2.2. szakaszban négy definiáló tulajdonságot adtunk meg egy szinguláris mátrixra. Mutasd meg, hogy ez a négy tulajdonság valóban ekvivalens.
- **2.2.** Tegyük fel, hogy egy $n \times n$-es A mátrix minden sorösszege nulla. Mutasd meg, hogy A szükségképpen szinguláris.
- **2.3.** Tegyük fel, hogy A egy szinguláris $n \times n$-es mátrix. Bizonyítsd be, hogy ha az $Ax = b$ lineáris rendszernek van legalább egy x megoldása, akkor végtelen sok megoldása van.
- **2.4.** (a) Mutasd meg, hogy az alábbi mátrix szinguláris.

$$A = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 1 & 3 & 2 \end{bmatrix}$$

(b) Ha $b = [2 \quad 4 \quad 6]^T$, hány megoldása van az $Ax = b$ rendszernek?

- **2.5.** Mi az alábbi mátrix inverze?

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & -1 & 0 \\ 1 & -2 & 1 \end{bmatrix}$$

- **2.6.** Legyen A olyan $n \times n$-es mátrix, amelyre $A^2 = O$, azaz a nullmátrix. Mutasd meg, hogy A szükségképpen szinguláris.
- **2.7.** Legyen

$$A = \begin{bmatrix} 1 & 1+\epsilon \\ 1-\epsilon & 1 \end{bmatrix}.$$

(a) Mi A determinánsa? (b) Lebegőpontos aritmetikában ϵ mely értéktartományára lesz a determináns számított értéke nulla? (c) Mi A LU-felbontása? (d) Lebegőpontos aritmetikában ϵ mely értéktartományára lesz U számított értéke szinguláris?

- **2.8.** Legyen A és B tetszőleges két $n \times n$-es mátrix. (a) Bizonyítsd be, hogy $(AB)^T = B^T A^T$. (b) Ha A és B mindkettő nemszinguláris, bizonyítsd be, hogy $(AB)^{-1} = B^{-1}A^{-1}$.
- **2.9.** Ha A tetszőleges nemszinguláris valós mátrix, mutasd meg, hogy $(A^{-1})^T = (A^T)^{-1}$. Ennek következtében az A^{-T} jelölés egyértelműen használható e mátrix jelölésére. Hasonlóan, ha A tetszőleges nemszinguláris komplex mátrix, akkor $(A^{-1})^H = (A^H)^{-1}$, és az A^{-H} jelölés egyértelműen használható e mátrix jelölésére.
- **2.10.** Legyen P egy tetszőleges permutációs mátrix. (a) Bizonyítsd be, hogy $P^{-1} = P^T$. (b) Bizonyítsd be, hogy P kifejezhető páronkénti cserék szorzataként.
- **2.11.** Írj részletes algoritmust egy $Lx = b$ alsó háromszögű lineáris rendszer előrehelyettesítéssel történő megoldására.
- **2.12.** Ellenőrizd, hogy egy n-ed rendű alsó háromszögű rendszer előrehelyettesítéssel történő megoldásának műveletszámában (szorzások vagy összeadások számában) a domináns tag $n^2/2$.
- **2.13.** Hogyan oldanád meg a következő alakú particionált lineáris rendszert?

$$\begin{bmatrix} L_1 & O \\ B & L_2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} b \\ c \end{bmatrix},$$

ahol L_1 és L_2 nemszinguláris alsó háromszögmátrixok, és a megoldás- és jobb oldali vektorokat ennek megfelelően particionáljuk. Mutasd be a konkrét lépéseket a megadott almátrixok és vektorok segítségével.

- **2.14.** Bizonyítsd be az elemi eliminációs mátrixok 2.4.3. szakaszban felsorolt négy tulajdonságát.
- **2.15.** (a) Bizonyítsd be, hogy két alsó háromszögmátrix szorzata alsó háromszögmátrix. (b) Bizonyítsd be, hogy egy nemszinguláris alsó háromszögmátrix inverze alsó háromszögmátrix.
- **2.16.** (a) Mi az alábbi mátrix LU-felbontása?

$$\begin{bmatrix} 1 & a \\ c & b \end{bmatrix}$$

(b) Milyen feltétel mellett szinguláris ez a mátrix?

- **2.17.** Írd fel az alábbi mátrix LU-felbontását (mutasd be mind az L-t, mind az U-t explicit módon):

$$\begin{bmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}.$$

- **2.18.** Bizonyítsd be, hogy az

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

mátrixnak nincs LU-felbontása, azaz nem létezik olyan L alsó háromszögmátrix és U felső háromszögmátrix, amelyekre $A = LU$.

- **2.19.** Legyen A egy $n \times n$-es nemszinguláris mátrix. Tekintsük az alábbi algoritmust: 1. Végigfutva A 1-től n-ig terjedő oszlopain, permutáljuk a sorokat szükség szerint úgy, hogy minden oszlopban a főátlón vagy az alatta lévő elemek közül a főátlóbeli elem legyen az abszolút értékben legnagyobb. Az eredmény PA valamilyen P permutációs mátrixszal. 2. Ezután főelemkiválasztás nélküli Gauss-kiküszöböléssel számítsuk ki PA LU-felbontását. (a) Numerikusan stabile ez az algoritmus? (b) Ha igen, magyarázd meg miért. Ha nem, adj ellenpéldát a szemléltetésre.
- **2.20.** Bizonyítsd be, hogy ha részleges főelemkiválasztással végzett Gauss-kiküszöbölést alkalmazunk egy oszloponként diagonálisan domináns A mátrixra, akkor sorcserék nem fognak bekövetkezni.
- **2.21.** Ha A, B és C $n \times n$-es mátrixok, B és C nemszinguláris, és b egy n-dimenziós vektor, hogyan implementálnád az

$$x = B^{-1}(2A + I)(C^{-1} + A)b$$

képletet anélkül, hogy bármelyik mátrixinverzet kiszámítanád?

- **2.22.** Ellenőrizd, hogy egy n-ed rendű mátrix Gauss-kiküszöböléssel történő LU-felbontásának műveletszámában (szorzások vagy összeadások számában) a domináns tag $n^3/3$.

- **2.23.** Ellenőrizd, hogy egy n-ed rendű mátrix Gauss-kiküszöböléssel történő inverzének kiszámításához tartozó műveletszámban (szorzások vagy összeadások számában) a domináns tag n^3.
- **2.24.** Ellenőrizd, hogy egy n-ed rendű mátrixon végrehajtott Gauss–Jordan-kiküszöbölés műveletszámában (szorzások vagy összeadások számában) a domináns tag $n^3/2$.
- **2.25.** (a) Ha u és v nem nulla n-dimenziós vektorok, bizonyítsd be, hogy a uv^T $n\times n$-es külső szorzat mátrix rangja 1. (b) Ha A egy olyan $n\times n$-es mátrix, amelyre $\operatorname{rank}(A) = 1$, bizonyítsd be, hogy léteznek olyan nem nulla u és v n-dimenziós vektorok, amelyekre $A = uv^T$.
- **2.26.** Egy $n \times n$-es A mátrixot *elemi*nek nevezünk, ha az egységmátrixtól egy egyrangú mátrixban tér el, azaz $A = I - uv^T$ valamilyen u és v n-dimenziós vektorokkal. (a) Ha A elemi, milyen feltétel biztosítja u-ra és v-re, hogy A nemszinguláris? (b) Ha A elemi és nemszinguláris, bizonyítsd be, hogy A^{-1} is elemi, megmutatva, hogy $A^{-1} = I - \sigma uv^T$ valamilyen σ skalárral. Mi a konkrét σ érték u és v függvényében? (c) Elemi-e egy a 2.4.3. szakaszban definiált elemi eliminációs mátrix? Ha igen, mik u, v és σ ebben az esetben?
- **2.27.** Bizonyítsd be, hogy a 2.4.9. szakaszban megadott Sherman–Morrison-formula,

$$(A - uv^T)^{-1} = A^{-1} + A^{-1}u(1 - v^T A^{-1}u)^{-1}v^T A^{-1},$$

helyes. (*Útmutatás:* Szorozd meg mindkét oldalt $A - uv^T$-tal.)

- **2.28.** Bizonyítsd be, hogy a 2.4.9. szakaszban megadott Woodbury-formula,

$$(A - UV^T)^{-1} = A^{-1} + A^{-1}U(I - V^T A^{-1}U)^{-1}V^T A^{-1},$$

helyes. (*Útmutatás:* Szorozd meg mindkét oldalt $A - UV^T$-tal.)

- **2.29.** $p = 1, 2$ és ∞ esetén bizonyítsd be, hogy a vektor p-normák teljesítik a 2.3.1. szakasz végén megadott három tulajdonságot.
- **2.30.** $p = 1$ és ∞ esetén bizonyítsd be, hogy a mátrix p-normák teljesítik a 2.3.2. szakasz végén megadott öt tulajdonságot.
- **2.31.** Legyen A egy szimmetrikus pozitív definit mátrix. Mutasd meg, hogy az

$$\|x\|_A = (x^T Ax)^{1/2}$$

függvény teljesíti a vektornorma 2.3.1. szakasz végén megadott három tulajdonságát. Ezt a vektornormát az A mátrix által *indukáltnak* mondjuk.

- **2.32.** Mutasd meg, hogy egy $m \times n$-es A mátrix alábbi függvényei teljesítik a mátrixnorma 2.3.2. szakasz végén megadott első három tulajdonságát, és ezért mátrixnormák az ott említett általánosabb értelemben. (a)

$$\|A\|_{\max} = \max_{i,j} |a_{ij}|$$

Figyeljük meg, hogy ez egyszerűen A-nak az $\mathbb{R}^{mn}$-beli vektorként tekintett ∞-normája. (b)

$$\|A\|_F = \left(\sum_{i,j} |a_{ij}|^2 \right)^{1/2}$$

Figyeljük meg, hogy ez egyszerűen A-nak az $\mathbb{R}^{mn}$-beli vektorként tekintett 2-normája. Ezt *Frobenius-mátrixnormának* nevezzük.

- **2.33.** Bizonyítsd be vagy adj ellenpéldát: Ha A nemszinguláris mátrix, akkor $\|A^{-1}\| = \|A\|^{-1}$.
- **2.34.** Tegyük fel, hogy A pozitív definit mátrix. (a) Mutasd meg, hogy A szükségképpen nemszinguláris. (b) Mutasd meg, hogy A^{-1} szükségképpen pozitív definit.
- **2.35.** Tegyük fel, hogy az A mátrixnak van $A = BB^T$ alakú felbontása, ahol B nemszinguláris. Mutasd meg, hogy A szükségképpen szimmetrikus és pozitív definit.
- **2.36.** Vezess le egy algoritmust, amely egy $n \times n$-es szimmetrikus pozitív definit A mátrix LL^T Cholesky-felbontásának kiszámításához A és LL^T megfelelő elemeinek egyenlővé tételével kapható.
- **2.37.** Tegyük fel, hogy az

$$B = \begin{bmatrix} \alpha & a^T \\ a & A \end{bmatrix}$$

$(n+1)$-ed rendű szimmetrikus mátrix pozitív definit. (a) Mutasd meg, hogy az α skalár szükségképpen pozitív, és az $n \times n$-es A mátrix szükségképpen pozitív definit. (b) Mi B Cholesky-felbontása az alkotó almátrixok segítségével?

- **2.38.** Tegyük fel, hogy az

$$B = \begin{bmatrix} A & a \\ a^T & \alpha \end{bmatrix}$$

$(n+1)$-ed rendű szimmetrikus mátrix pozitív definit. (a) Mutasd meg, hogy az α skalár szükségképpen pozitív, és az $n \times n$-es A mátrix szükségképpen pozitív definit. (b) Mi B Cholesky-felbontása az alkotó almátrixok segítségével?

- **2.39.** Ellenőrizd, hogy egy n-ed rendű szimmetrikus pozitív definit mátrix Cholesky-felbontásának műveletszámában (szorzások vagy összeadások számában) a domináns tag $n^3/6$.
- **2.40.** Legyen A egy β sávszélességű sávos mátrix, és tegyük fel, hogy a $PA = LU$ LU-felbontást részleges főelemkiválasztással végzett Gauss-kiküszöböléssel számítjuk. Mutasd meg, hogy az U felső háromszögű tényező sávszélessége legfeljebb 2β.

- **2.41.** Legyen A egy nemszinguláris tridiagonális mátrix. (a) Mutasd meg, hogy általában A^{-1} sűrű. (b) Hasonlítsd össze az $Ax = b$ lineáris rendszer Gauss-kiküszöböléssel és visszahelyettesítéssel történő megoldásához, valamint a rendszer explicit mátrixinvertálással történő megoldásához szükséges munkát és tárhelyet.

Ez a példa egy további okot szemléltet arra, hogy az explicit mátrixinvertálás miért rendszerint rossz ötlet.

- **2.42.** (a) Készíts algoritmust egy nemszinguláris $n \times n$-es háromszögmátrix inverzének helyben történő kiszámítására, azaz további tömbtárhely nélkül. (b) Lehetséges-e egy általános nemszinguláris $n \times n$-es mátrix inverzét helyben kiszámítani? Ha igen, vázolj egy erre szolgáló algoritmust, ha nem, magyarázd meg miért. E feladat céljaira feltételezheted, hogy főelemkiválasztásra nincs szükség.
- **2.43.** Tegyük fel, hogy egy $Cz = d$ lineáris rendszert kell megoldanod, ahol C egy komplex $n \times n$-es mátrix, d és z pedig komplex n-dimenziós vektorok, de a lineáris egyenletrendszer-megoldód csak valós rendszereket kezel. Legyen $C = A + iB$ és $d = b + ic$, ahol A, B, b és c valósak, és $i = \sqrt{-1}$. Mutasd meg, hogy a $z = x + iy$ megoldást a

$$\begin{bmatrix} A & -B \\ B & A \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} b \\ c \end{bmatrix}$$

$2n \times 2n$-es valós lineáris rendszer adja. Jó módja-e ez a feladat megoldásának? Miért?

Számítógépes feladatok

- **2.1.** (a) Mutasd meg, hogy az

$$\mathbf{A} = \begin{bmatrix} 0{,}1 & 0{,}2 & 0{,}3 \\ 0{,}4 & 0{,}5 & 0{,}6 \\ 0{,}7 & 0{,}8 & 0{,}9 \end{bmatrix}$$

mátrix szinguláris. Írd le az $Ax = b$ rendszer megoldásainak halmazát, ha

$$b = \begin{bmatrix} 0{,}1 \\ 0{,}3 \\ 0{,}5 \end{bmatrix}.$$

(b) Ha részleges főelemkiválasztással végzett Gauss-kiküszöböléssel akarnánk megoldani ezt a rendszert egzakt aritmetikával, hol akadna meg a folyamat? (c) Mivel A néhány eleme nem ábrázolható egzaktul egy bináris lebegőpontos rendszerben, a mátrix a számítógépbe bevitel után már nem egzaktul szinguláris, így a rendszer Gauss-kiküszöböléssel történő megoldása nem feltétlenül hiúsul meg. Oldd meg ezt a rendszert számítógépen egy Gauss-kiküszöbölést végző könyvtári rutinnal. Hasonlítsd össze a számított megoldást az (a) részben adott megoldáshalmaz leírásával. Ha a szoftvered tartalmaz kondícióbecslőt, mi a cond(A) becsült értéke? Mennyi értékes jegyet várnál e becslés alapján a megoldásban?

- **2.2.** (a) Egy Gauss-kiküszöbölést végző könyvtári rutinnal oldd meg az $Ax = b$ rendszert, ahol

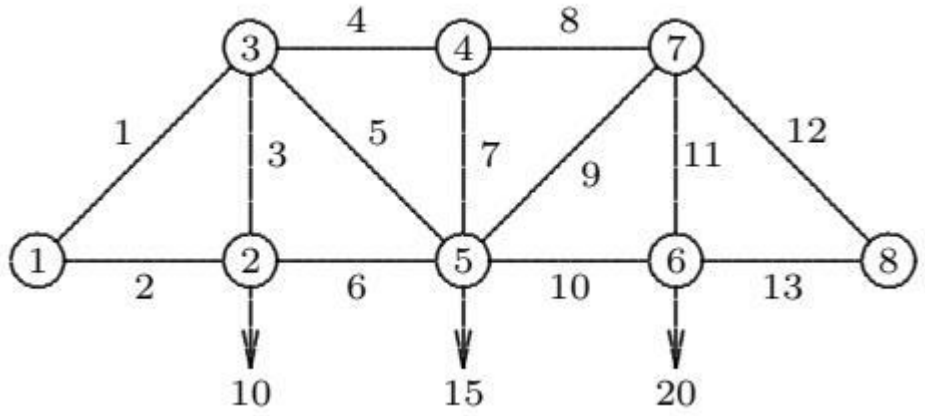
$$A = \begin{bmatrix} 2 & 4 & -2 \\ 4 & 9 & -3 \\ -2 & -1 & 7 \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ 8 \\ 10 \end{bmatrix}.$$

(b) Az A mátrix már kiszámított LU-felbontását felhasználva – a mátrix újrafelbontása nélkül – oldd meg az $Ay = c$ rendszert, ahol

$$c = \begin{bmatrix} 4 \\ 8 \\ -6 \end{bmatrix}.$$

(c) Ha az A mátrix úgy változik, hogy $a_{1,2} = 2$, a Sherman–Morrison-féle frissítési technikával – a mátrix újrafelbontása nélkül – számítsd ki az új x megoldást az eredeti b jobb oldali vektort használva.

- **2.3.** Az alábbi ábra egy síkbeli rácsszerkezetet ábrázol, amely 13 rúdból (a számozott egyenesek) és 10 csomópontból (a számozott körök) áll. A jelzett terhelések, tonnában, a 2., az 5. és a 6. csomópontra hatnak, és meg szeretnénk határozni a rácsszerkezet egyes rúdjaira ható eredő erőt.



Ahhoz, hogy a rácsszerkezet statikus egyensúlyban legyen, minden csomópontban se vízszintes, se függőleges irányú eredő erő ne lépjen fel. Így a rúderőket úgy határozhatjuk meg, hogy minden csomópontban a balról és a jobbról ható vízszintes erőket, hasonlóképpen a felfelé és a lefelé ható függőleges erőket egyenlővé tesszük. A nyolc csomópontra ez 16 egyenletet adna, ami több, mint a 13 meghatározandó ismeretlen erő. Ahhoz, hogy a rácsszerkezet statikailag határozott legyen, azaz hogy egyetlen megoldás létezzen, feltételezzük, hogy az 1. csomópont vízszintesen és függőlegesen is mereven rögzített, a 8. csomópont pedig függőlegesen rögzített. A rúderőket vízszintes és függőleges komponensekre felbontva és $\alpha = \sqrt{2}/2$-t bevezetve a rúderőkre, f_i-re, a következő egyenletrendszert kapjuk:

Egy könyvtári rutinnal oldd meg ezt a lineáris egyenletrendszert az f rúderővektorra. Megjegyzendő, hogy e rendszer mátrixa meglehetősen ritka, így érdemes lehet sávos

rendszer-megoldóval vagy általánosabb ritka megoldóval is kísérletezned, bár ez a konkrét feladatpéldány túl kicsi ahhoz, hogy ezek egy általános megoldóhoz képest jelentős előnyt jelentsenek.

- **2.4.** Írj rutint egy A mátrix kondíciószámának becslésére. Használhatod akár az 1-normát, akár a ∞-normát (vagy próbáld ki mindkettőt, és hasonlítsd össze az eredményeket). Ki kell majd számítanod $\|A\|$-t, ami könnyű, és meg kell becsülnöd $\|A^{-1}\|$-t, ami nagyobb kihívás. Ahogy a 2.3.3. szakaszban tárgyaltuk, $\|A^{-1}\|$ becslésének egy módja, hogy olyan y vektort választunk, amelyre a $\|z\|/\|y\|$ arány nagy, ahol z az $Az = y$ megoldása. Próbálj ki két különböző megközelítést y választására: (a) Válaszd y-t az $A^T y = c$ rendszer megoldásaként, ahol c egy olyan vektor, amelynek minden komponense ± 1, és az egyes komponensekhez tartozó előjelet az alábbi heurisztikával választjuk. Az $\mathbf{A} = \mathbf{LU}$ felbontást felhasználva az $\mathbf{A}^T\mathbf{y} = \mathbf{c}$ rendszert két lépésben oldjuk meg: egymás után megoldjuk az $\mathbf{U}^T\mathbf{v} = \mathbf{c}$ és az $\mathbf{L}^T\mathbf{y} = \mathbf{v}$ háromszögű rendszereket. Az első háromszögű megoldás minden lépésében c megfelelő komponensét 1-nek vagy -1-nek választjuk, attól függően, hogy melyik választás teszi a kapott v-komponenst abszolút értékben nagyobbá. (Ehhez egy saját háromszögű megoldó rutint kell majd írnod e logika megvalósítására.) Ezután oldd meg a második háromszögű rendszert a szokott módon y-ra. Az ötlet az, hogy az A-ban jelen lévő rossz kondicionáltság U-ban fog tükröződni, viszonylag nagy v-t eredményezve. A viszonylag jól kondicionált egységháromszögű L mátrix aztán megőrzi ezt a viszonyt, viszonylag nagy y-t eredményezve. (b) Válassz néhány – mondjuk öt – különböző y vektort véletlenszerűen, és azt használd, amelyik a legnagyobb $\|z\|/\|y\|$ arányt adja. (Ehhez közönséges háromszögű megoldó rutint használhatsz.)

Az A szükséges LU-felbontásához könyvtári rutint használhatsz. Teszteld mindkét megközelítést az alábbi mátrixokon:

$$\mathbf{A}_1 = \begin{bmatrix} 10 & -7 & 0 \\ -3 & 2 & 6 \\ 5 & -1 & 5 \end{bmatrix},$$

$$\mathbf{A}_2 = \begin{bmatrix} -73 & 78 & 24 \\ 92 & 66 & 25 \\ -80 & 37 & 10 \end{bmatrix}.$$

Hogyan viszonyulnak egymáshoz a két módszerrel kapott eredmények? A becsléseid minőségének ellenőrzéséhez számítsd ki $\mathbf{A}^{-1}$-et explicit módon, hogy meghatározd a valódi normáját (ehhez a számításhoz is felhasználhatod a már kiszámított LU-felbontást). Ha van hozzáférésed olyan lineáris egyenletrendszer-szoftverhez, amely már tartalmaz kondícióbecslőt, hogyan viszonyulnak az eredményeid az övéhez?

- **2.5.** (a) Egy egyszeres pontosságú, Gauss-kiküszöbölést végző rutinnal oldd meg az $Ax = b$ rendszert, ahol

$$A = \begin{bmatrix} 21{,}0 & 67{,}0 & 88{,}0 & 73{,}0 \\ 76{,}0 & 63{,}0 & 7{,}0 & 20{,}0 \\ 0{,}0 & 85{,}0 & 56{,}0 & 54{,}0 \\ 19{,}3 & 43{,}0 & 30{,}2 & 29{,}4 \end{bmatrix},$$

$$b = \begin{bmatrix} 141{,}0 \\ 109{,}0 \\ 218{,}0 \\ 93{,}7 \end{bmatrix}.$$

(b) Számítsd ki az $r = b - Ax$ maradékot dupla pontosságú aritmetikával, ha elérhető (de a végső eredményt egyszeres pontosságú r vektorba tárolva). Megjegyzendő, hogy a megoldó rutin elpusztíthatja az A-t tartalmazó tömböt, így külön másolatot kell majd megőrizned a maradék számításához. (Ha a használt számítási környezetben csak egyetlen precízió érhető el, akkor végezd el a teljes feladatot abban a precízióban.) (c) Oldd meg az $Az = r$ lineáris rendszert a „javított" $x + z$ megoldás megkapásához. Megjegyzendő, hogy A-t nem kell újrafelbontani. (d) Ismételd a (b) és (c) lépéseket, amíg további javulást nem tapasztalsz.

- **2.6.** Egy $n \times n$-es H Hilbert-mátrix elemei $h_{ij} = 1/(i + j - 1)$, így az alakja

$$\begin{bmatrix} 1 & 1/2 & 1/3 & \cdots \\ 1/2 & 1/3 & 1/4 & \cdots \\ 1/3 & 1/4 & 1/5 & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}.$$

$n = 2, 3, \ldots$ esetén generáld az n-ed rendű Hilbert-mátrixot, valamint a $b = Hx$ n-dimenziós vektort, ahol x az az n-dimenziós vektor, amelynek minden komponense 1. Egy Gauss-kiküszöbölést (vagy Cholesky-felbontást, mivel a Hilbert-mátrix szimmetrikus és pozitív definit) végző könyvtári rutinnal oldd meg a kapott $Hx = b$ lineáris rendszert, $\hat{x}$ közelítő megoldást kapva. Számítsd ki az $\mathbf{r} = \mathbf{b} - \mathbf{H}\hat{x}$ maradék és a $\Delta\mathbf{x} = \hat{x} - \mathbf{x}$ hiba ∞-normáját, ahol $\mathbf{x}$ a csupa egyes vektor. Milyen nagyra veheted n-et, mielőtt a hiba 100 százalékossá válik (azaz a megoldásban nincs egyetlen értékes jegy sem)? Emellett egy kondícióbecslővel határozd meg cond(H)-t minden n-re. Próbáld meg jellemezni a kondíciószámot n függvényében. Ahogy n változik, hogyan viszonyul a számított megoldás komponenseiben lévő helyes számjegyek száma a mátrix kondíciószámához?

- **2.7.** (a) Mi történik, ha részleges főelemkiválasztással végzett Gauss-kiküszöbölést alkalmazunk a következő alakú mátrixra?

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 1 \\ -1 & 1 & 0 & 0 & 1 \\ -1 & -1 & 1 & 0 & 1 \\ -1 & -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & -1 & 1 \end{bmatrix}$$

Nőnek-e a transzformált mátrix elemei? Mi történik, ha ehelyett teljes főelemkiválasztást használunk? (Megjegyzendő, hogy az (a) rész nem igényel számítógépet.)

(b) Egy részleges főelemkiválasztással végzett Gauss-kiküszöbölést alkalmazó könyvtári rutinnal oldj meg ilyen alakú, különböző méretű lineáris rendszereket, olyan jobb oldali vektorokat választva, amelyek esetén a megoldás ismert. Hogyan viselkedik a hiba, a maradék és a kondíciószám, ahogy a rendszerek egyre nagyobbak lesznek? Ez a mesterségesen megkonstruált rendszer a 2.4.5. szakaszban idézett, legrosszabb esetben fellépő növekedési tényezőt illusztrálja, és nem jellemző a részleges főelemkiválasztással végzett Gauss-kiküszöbölés szokásos viselkedésére.

- **2.8.** Egy $Ax = b$ lineáris rendszer mindkét oldalának egy D nemszinguláris diagonális mátrixszal való szorzása, hogy egy új $DAx = Db$ rendszert kapjunk, egyszerűen átskálázza a rendszer sorait, és elméletben nem változtatja meg a megoldást. Az ilyen skálázás azonban befolyásolja a mátrix kondíciószámát és a Gauss-kiküszöbölésben a főelemek választását, így véges precíziójú aritmetikában befolyásolhatja a megoldás pontosságát. Megjegyzendő, hogy a skálázás némi kerekítési hibát vihet a mátrixba, hacsak D elemei nem a használt lebegőpontos aritmetikai rendszer alapjának hatványai (miért?).

Véletlenszerűen választott A mátrixú lineáris rendszer és olyan b jobb oldali vektor használatával, amely mellett a megoldás ismert, kísérletezz különféle D skálázó mátrixokkal, hogy milyen hatással vannak a DA mátrix kondíciószámára és a $DAx = Db$ lineáris rendszer megoldására egy erre szolgáló könyvtári rutin által adott megoldásra. Mindenképpen próbálj ki elég torz skálázásokat is, amelyeknél D diagonális elemeinek nagyságrendje jelentősen különbözik egymástól (a cél egy rosszul megválasztott mértékegységekkel adott rendszer szimulálása). Hasonlítsd össze mind a relatív maradékokat, mind a különféle skálázások által adott hibát. Találsz-e olyan skálázást, amely nagyon rossz pontosságot ad? Ebben az esetben a maradék továbbra is kicsi marad-e?

- **2.9.** (a) Főelemkiválasztás nélküli Gauss-kiküszöböléssel oldd meg az

$$\begin{bmatrix} \epsilon & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1+\epsilon \\ 2 \end{bmatrix}$$

lineáris rendszert $\epsilon = 10^{-2k}$ esetén, $k = 1, \dots, 10$. Az egzakt megoldás $x = \begin{bmatrix} 1 & 1 \end{bmatrix}^T$, ϵ értékétől függetlenül. Hogyan viselkedik a számított megoldás pontossága, ahogy ϵ értéke csökken?

(b) Ismételd meg az (a) részt, továbbra is főelemkiválasztás nélküli Gauss-kiküszöbölést használva, de ezúttal a megoldás javítására egy lépés iteratív finomítást is alkalmazz, a maradékot ugyanolyan precízióval számítva, mint a számítások többi részét. Most hogyan viselkedik a számított megoldás pontossága, ahogy ϵ értéke csökken?

- **2.10.** Tekintsük az

$$\begin{bmatrix} 1 & 1+\epsilon \\ 1-\epsilon & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1+(1+\epsilon)\epsilon \\ 1 \end{bmatrix}$$

lineáris rendszert, ahol ϵ egy rögzítendő kis paraméter. Az egzakt megoldás nyilvánvalóan

$$x = \begin{bmatrix} 1 \\ \epsilon \end{bmatrix}$$

ϵ bármely értékére.

Egy Gauss-kiküszöbölésen alapuló könyvtári rutinnal oldd meg ezt a rendszert. Kísérletezz ϵ különféle értékeivel, különösen a saját számítógéped $\sqrt{\epsilon_{\text{mach}}}$-ához közeli értékekkel. ϵ minden kipróbált értékére becsüld meg a mátrix kondíciószámát és a megoldás minden komponensében lévő relatív hibát. Milyen pontosan van meghatározva az egyes komponensek? Hogyan viszonyul az egyes komponensekre kapott pontosság a mátrix kondíciószáma és a 2.3.4. szakaszban megadott hibakorlátok alapján várhatóhoz? Milyen következtetéseket tudsz levonni ebből a kísérletből?

- **2.11.** (a) Írj programokat, amelyek megvalósítják a főelemkiválasztás nélküli, a részleges főelemkiválasztással és a teljes főelemkiválasztással végzett Gauss-kiküszöbölést. (b) Generálj több véletlen mátrixú lineáris rendszert (azaz a mátrix elemeinek előállítására használj véletlenszám-generátort), valamint olyan jobb oldali vektorokat, amelyek mellett a megoldás ismert, és hasonlítsd össze a három implementáció pontosságát, maradékait és teljesítményét. (c) Tudsz-e alkotni egy (nem véletlen) mátrixot, amelyre a teljes főelemkiválasztás lényegesen pontosabb, mint a részleges főelemkiválasztás?
- **2.12.** Írj rutint tridiagonális lineáris egyenletrendszerek megoldására a 2.5.3. szakaszban megadott algoritmussal, és teszteld néhány mintapéldán. Írd le, hogyan változna a rutinod, ha részleges főelemkiválasztást vennél be. Írd le, hogyan változna a rutinod, ha a rendszer pozitív definit volna, és LU-felbontás helyett a Cholesky-felbontást számítanád ki.
- **2.13.** Egy háromszögmátrix determinánsa a főátlóbeli elemeinek szorzatával egyenlő. Használd ezt a tényt egy olyan rutin kidolgozásához, amely egy tetszőleges $n \times n$-es A mátrix determinánsát számítja ki az LU-felbontása segítségével. Használhatsz egy részleges főelemkiválasztással végzett Gauss-kiküszöbölést megvalósító könyvtári rutint az LU-felbontás megszerzésére, vagy tervezheted saját rutinodat. Hogyan tudod meghatározni a determináns megfelelő előjelét? A túlcsordulás vagy alulcsordulás veszélyének elkerülésére érdemes lehet a determináns tényleges értéke helyett a logaritmusának kiszámítását fontolóra venned.
- **2.14.** Írj programokat, amelyek megvalósítják a $C = AB$ mátrixszorzást – ahol A egy $m \times n$-es és B egy $n \times k$-s mátrix – két különböző módon: (a) Számítsd ki az A sorainak és B oszlopainak mk darab belső szorzatát. (b) C minden oszlopát A oszlopainak lineáris kombinációjaként állítsd elő.

BLAS-terminológiában (lásd a 2.7.2. szakaszt) az első implementáció az `sdot`-ot, a második pedig az `saxpy`-t használja. Hasonlítsd össze e két implementáció teljesítményét

a saját számítógépeden. Előfordulhat, hogy elég nagy mátrixokat kell kipróbálnod, mielőtt a teljesítménykülönbségek jelentőssé válnának. Tudj meg annyit a számítógéprendszeredről, amennyit csak tudsz (pl. a gyorsítótár mérete és kezelési stratégiája), és használd fel ezt az információt a megfigyelt eredmények magyarázatára.

- **2.15.** Valósítsd meg a Gauss-kiküszöbölést a hármas egymásba ágyazott ciklus mind a hat különböző sorrendjével, és hasonlítsd össze a teljesítményüket a saját számítógépeden. E feladat céljaira a numerikus stabilitás érdekében végzett főelemkiválasztást figyelmen kívül hagyhatod, de mindenképpen olyan tesztmátrixokat használj, amelyek nem igényelnek főelemkiválasztást. Előfordulhat, hogy elég nagy rendszert kell kipróbálnod, mielőtt a teljesítménykülönbségek jelentőssé válnának. Tudj meg annyit a számítógéprendszeredről, amennyit csak tudsz (pl. a gyorsítótár mérete és kezelési stratégiája), és használd fel ezt az információt a megfigyelt eredmények magyarázatára.
- **2.16.** Háromszögű lineáris rendszerek megoldásához mind az előre-, mind a visszahelyettesítés olyan egymásba ágyazott ciklusokat tartalmaz, amelyek két indexe bármelyik sorrendben futhat. Valósítsd meg mind az előre-, mind a visszahelyettesítést a két indexsorrend mindegyikével (összesen négy algoritmust), és hasonlítsd össze teljesítményüket különböző méretű háromszögű tesztmátrixokon. Előfordulhat, hogy elég nagy rendszert kell kipróbálnod, mielőtt a teljesítménykülönbségek jelentőssé válnának. Az indexsorrendek legjobb választása ugyanaz-e mindkét algoritmusnál? Tudj meg annyit a számítógéprendszeredről, amennyit csak tudsz (pl. a gyorsítótár mérete és kezelési stratégiája), és használd fel ezt az információt a megfigyelt eredmények magyarázatára.
- **2.17.** Tekintsünk egy vízszintes, egyik végén befogott, a hossza fennmaradó részén szabad tartót. A tartón ható erők egy diszkrét modellje egy $Ax = b$ lineáris egyenletrendszert ad, ahol az $n \times n$-es A mátrix a következő sávos alakú:

$$\begin{bmatrix} 9 & -4 & 1 & 0 & \cdots & \cdots & 0 \\ -4 & 6 & -4 & 1 & \ddots & & \vdots \\ 1 & -4 & 6 & -4 & 1 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & 1 & -4 & 6 & -4 & 1 \\ \vdots & & \ddots & 1 & -4 & 5 & -2 \\ 0 & \cdots & \cdots & 0 & 1 & -2 & 1 \end{bmatrix},$$

a b n-dimenziós vektor a tartóra ható ismert terhelés (a saját súlyával együtt), a x n-dimenziós vektor pedig a tartó meghatározandó, eredő lehajlását jelöli. A tartót egyenletesen terheltnek tekintjük, úgy, hogy a terhelésvektor minden komponensére $b_i = 1/n^4$.

(a) $n = 100$ esetén oldd meg ezt a lineáris rendszert mind egy sűrű lineáris rendszerekhez szolgáló standard könyvtári rutinnal, mind egy sávos (vagy általánosabb ritka) rendszerekhez tervezett könyvtári rutinnal. Hogyan viszonyul egymáshoz a két rutin a megoldás kiszámításához szükséges időben? Mennyire egyeznek meg a kapott válaszok egymással?

(b) Ellenőrizd, hogy az A mátrixnak van $A = RR^T$ UL-felbontása, ahol R a következő alakú felső háromszögmátrix:

$\lceil 2 \rceil$	-2	1	0		0]
0	-21□.	-2	1	□.	:
:	□.	□	□.	□	0
:		□.		-2	
:			□.	1	$\begin{bmatrix} -2 \\ 1 \end{bmatrix}$
				0	1

$n = 1000$ esetén oldd meg a lineáris rendszert e felbontással (két háromszögű megoldásra lesz szükség). Emellett oldd meg a rendszert az eredeti alakjában is egy sávos (vagy általános ritka) rendszer-megoldóval, ahogy az (a) részben. Mennyire egyeznek meg a kapott válaszok egymással? Melyik megközelítés tűnik pontosabbnak? Mi A kondíciószáma, és milyen pontosságot sugall, hogy várnunk kellene? Próbáld ki az iteratív finomítást, hogy lássad, a kevésbé pontos módszernél javul-e a pontosság vagy a maradék.